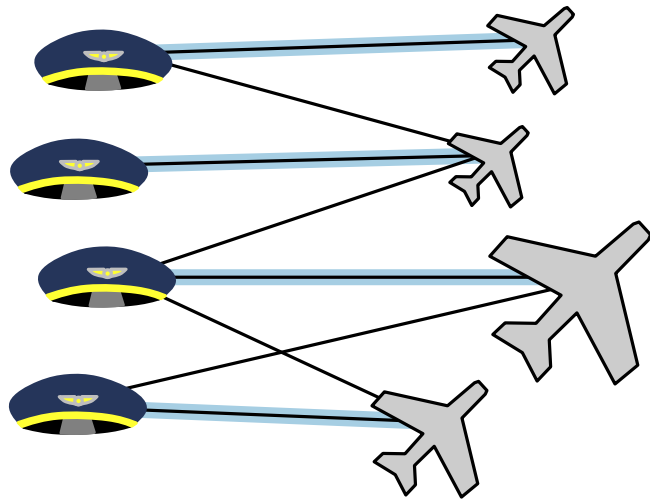
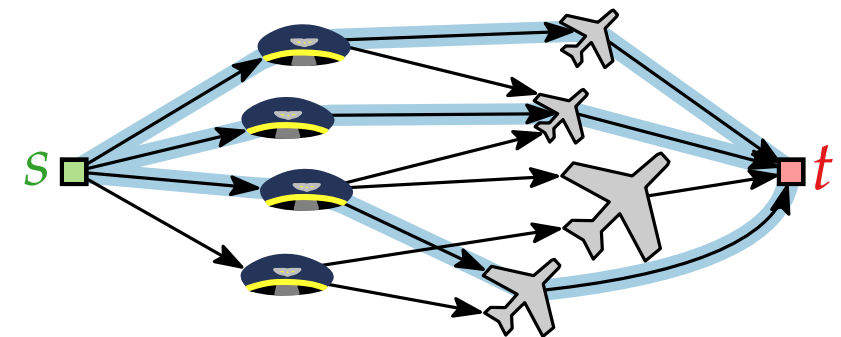


Fortgeschrittene Algorithmen

9. Vorlesung Graphalgorithmen III: Paarungen / Matchings



Teil I:
Allgemeine Flussnetzwerke
& Ganzzahligkeitssatz



Allgemeine Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Ein **größter S - T -Fluss** ist ein S - T -Fluss, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



Allgemeine Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; \ell; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Ein **größter S - T -Fluss** ist ein S - T -Fluss, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



Allgemeine Flussnetzwerke

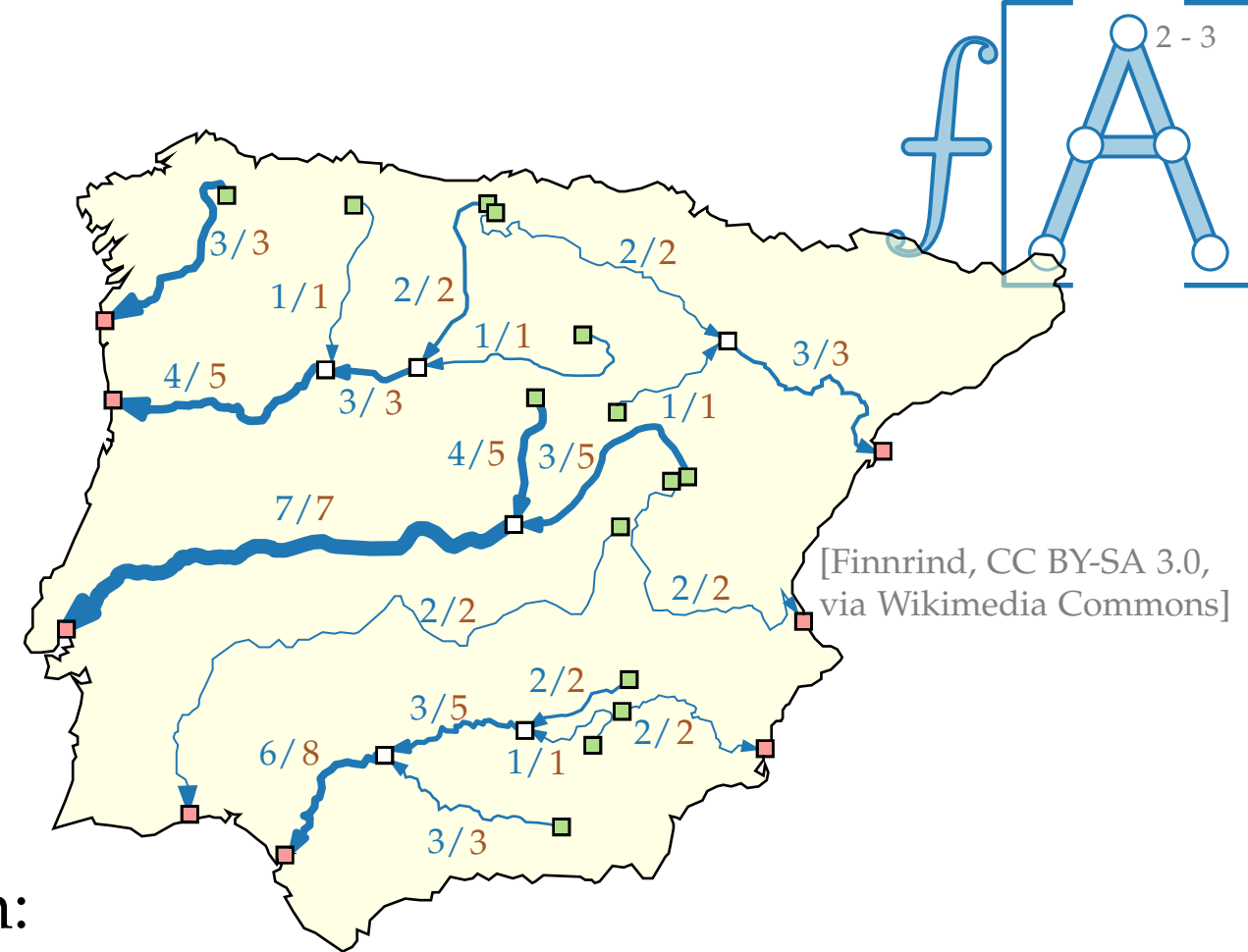
Flussnetzwerk $(G = (V, E); S; T; \ell; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *untere Kantenschranken* $\ell: E \rightarrow \mathbb{R}_0^+$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Ein **größter S-T-Fluss** ist ein **S-T-Fluss**, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



Allgemeine Flussnetzwerke

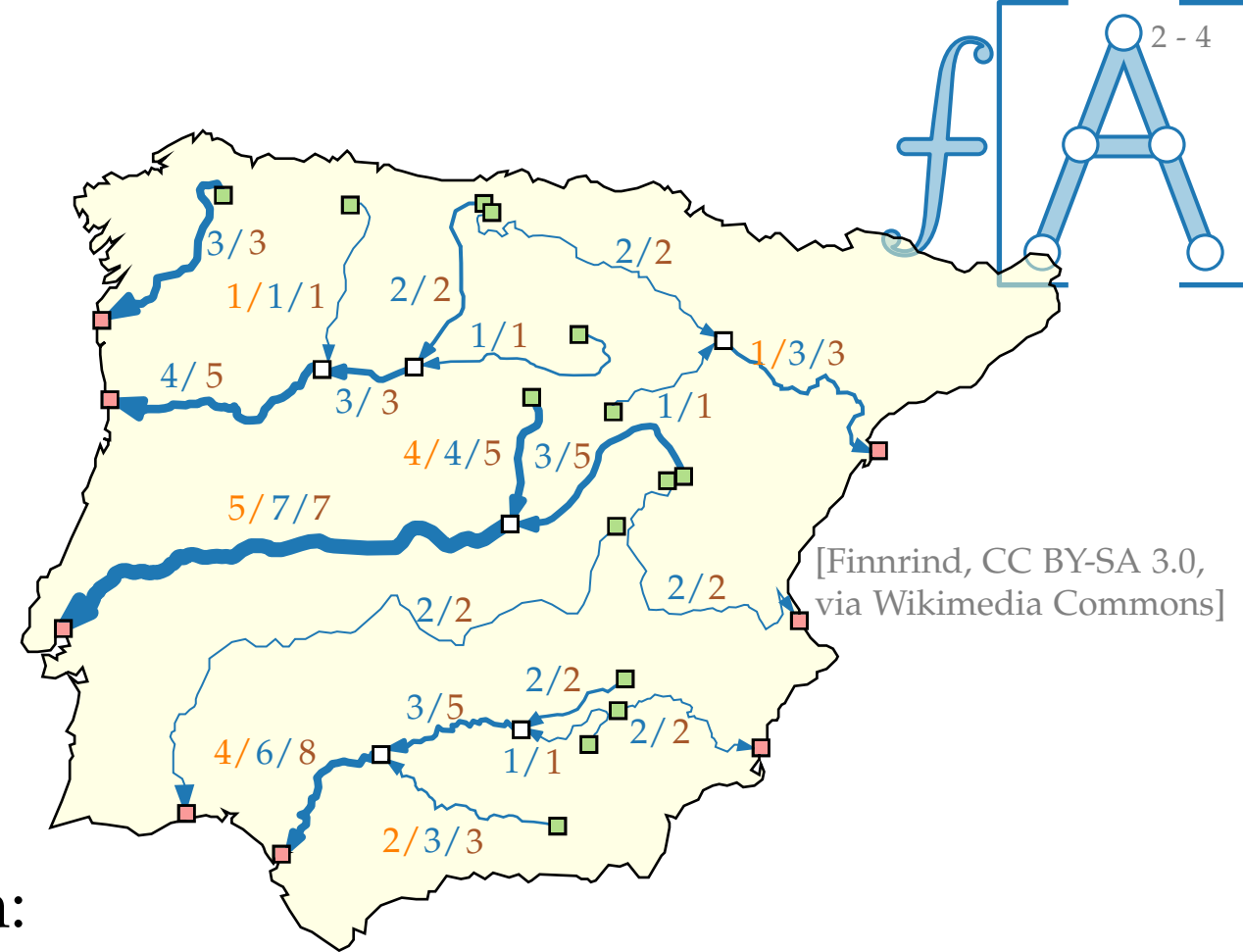
Flussnetzwerk $(G = (V, E); S; T; \ell; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *untere Kantenschranken* $\ell: E \rightarrow \mathbb{R}_0^+$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Ein **größter S-T-Fluss** ist ein **S-T-Fluss**, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



Allgemeine Flussnetzwerke

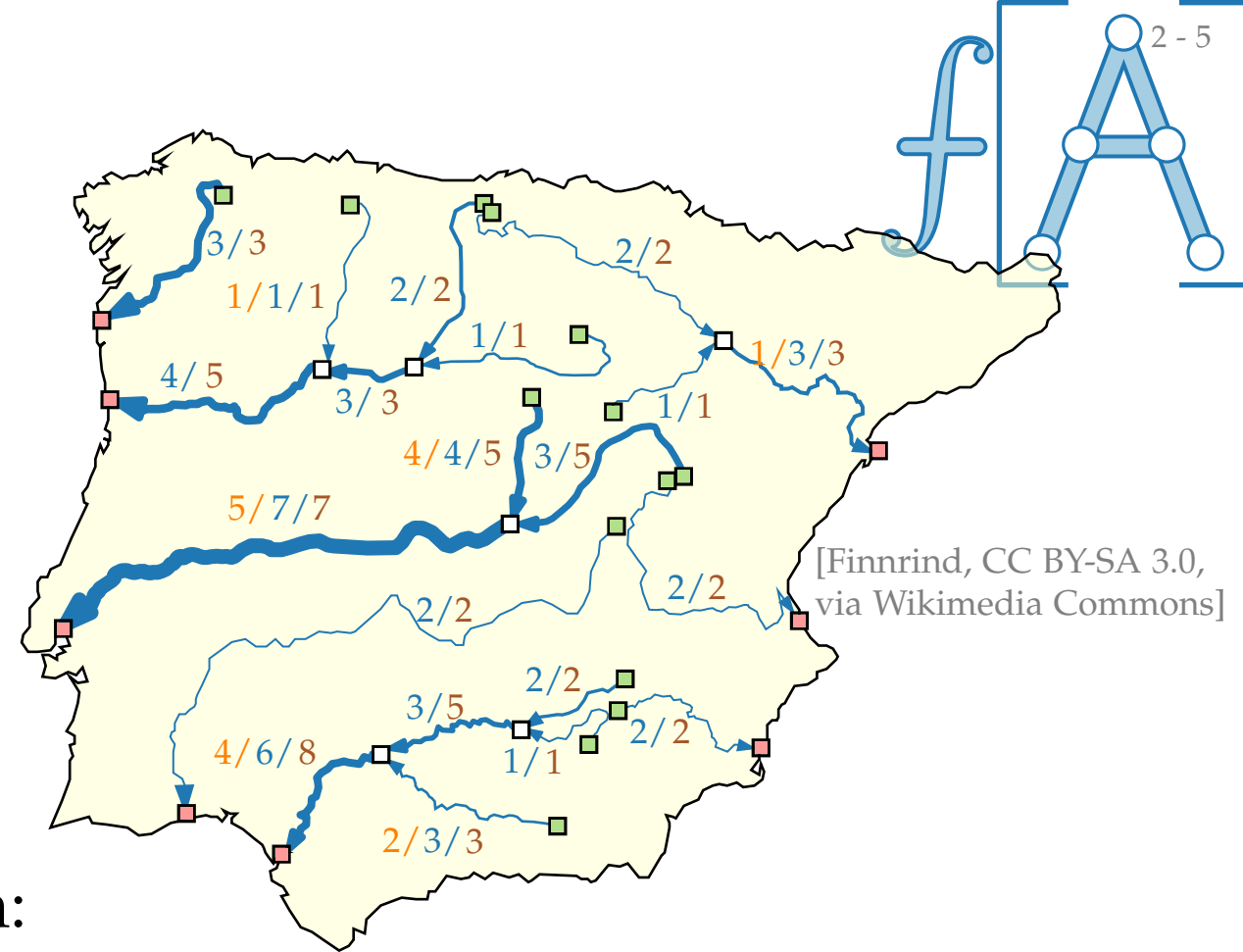
Flussnetzwerk $(G = (V, E); S; T; \ell; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *untere Kantenschranken* $\ell: E \rightarrow \mathbb{R}_0^+$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$\begin{aligned} \ell(u, v) &\leq f(u, v) \leq c(u, v) & \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) &= 0 & \forall v \in V \setminus (S \cup T) \end{aligned}$$

Ein **größter S-T-Fluss** ist ein **S-T-Fluss**, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



Allgemeine Flussnetzwerke

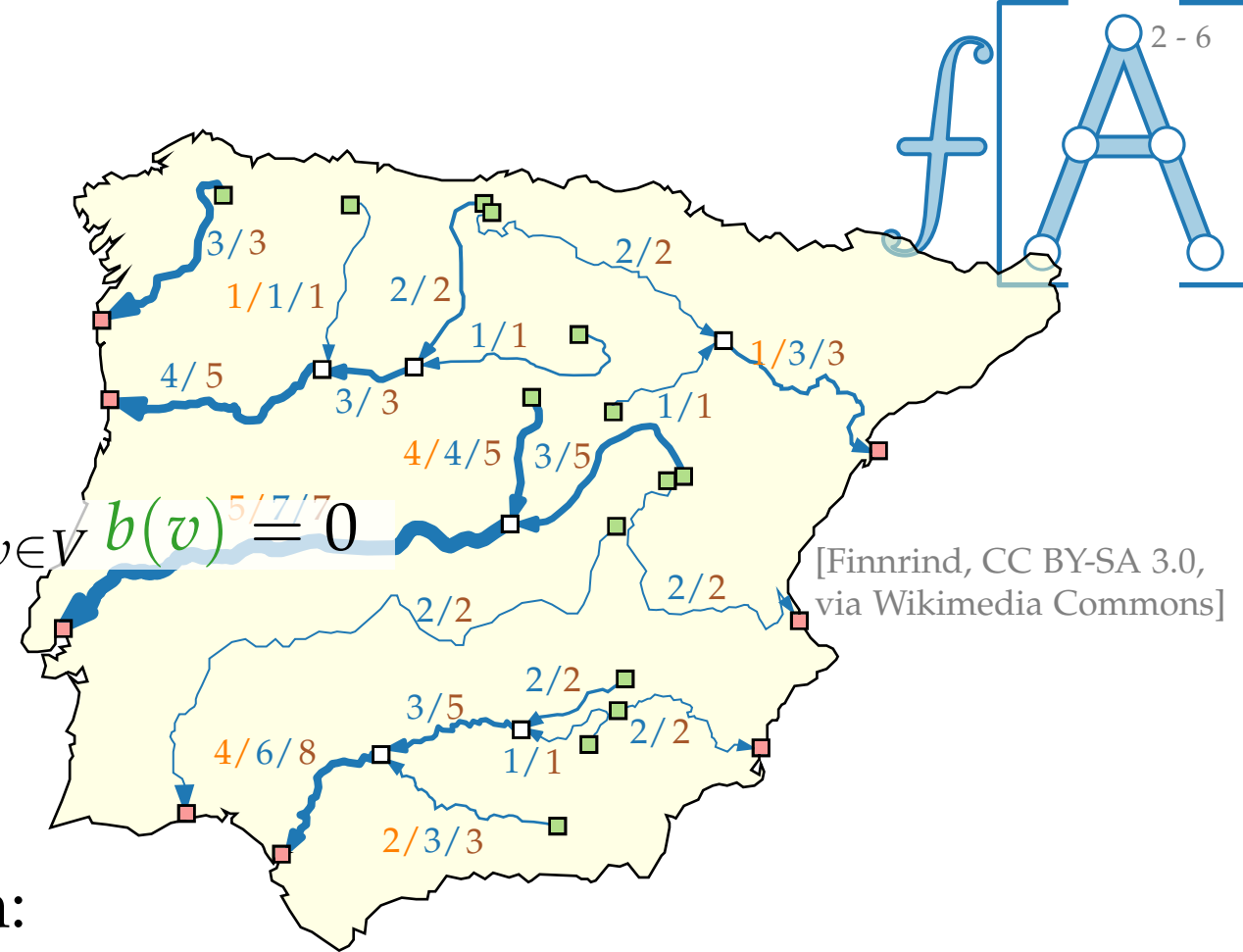
Flussnetzwerk $(G = (V, E); b; \ell; c; \text{cost})$ mit

- gerichteter Graph $G = (V, E)$
- *Knotenproduktion/-verbrauch* $b: V \rightarrow \mathbb{R}$ mit $\sum_{v \in V} b(v) = 0$
- *untere Kantenschranken* $\ell: E \rightarrow \mathbb{R}_0^+$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$\begin{aligned} \ell(u, v) &\leq f(u, v) \leq c(u, v) & \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) &= 0 & \forall v \in V \setminus (S \cup T) \end{aligned}$$

Ein **größter S-T-Fluss** ist ein **S-T-Fluss**, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



Allgemeine Flussnetzwerke

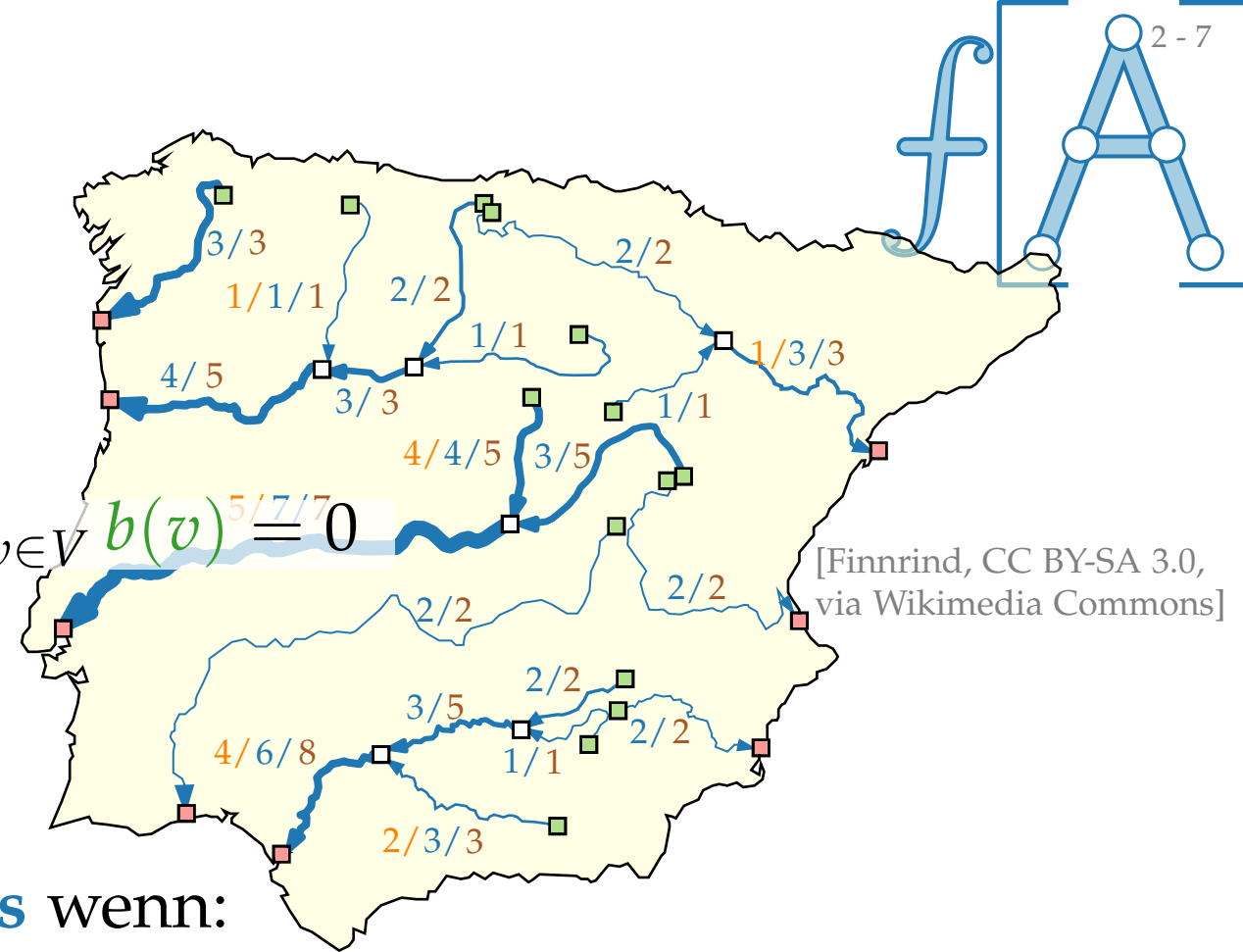
Flussnetzwerk $(G = (V, E); b; \ell; c; \text{cost})$ mit

- gerichteter Graph $G = (V, E)$
- *Knotenproduktion/-verbrauch* $b: V \rightarrow \mathbb{R}$ mit $\sum_{v \in V} b(v) = 0$
- *untere Kantenschranken* $\ell: E \rightarrow \mathbb{R}_0^+$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **zulässiger Fluss** wenn:

$$\begin{aligned} \ell(u, v) &\leq f(u, v) \leq c(u, v) & \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) &= b(v) & \forall v \in V \end{aligned}$$

Ein **größter** S - T -Fluss ist ein S - T -Fluss, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



Allgemeine Flussnetzwerke

Flussnetzwerk $(G = (V, E); b; \ell; c; \text{cost})$ mit

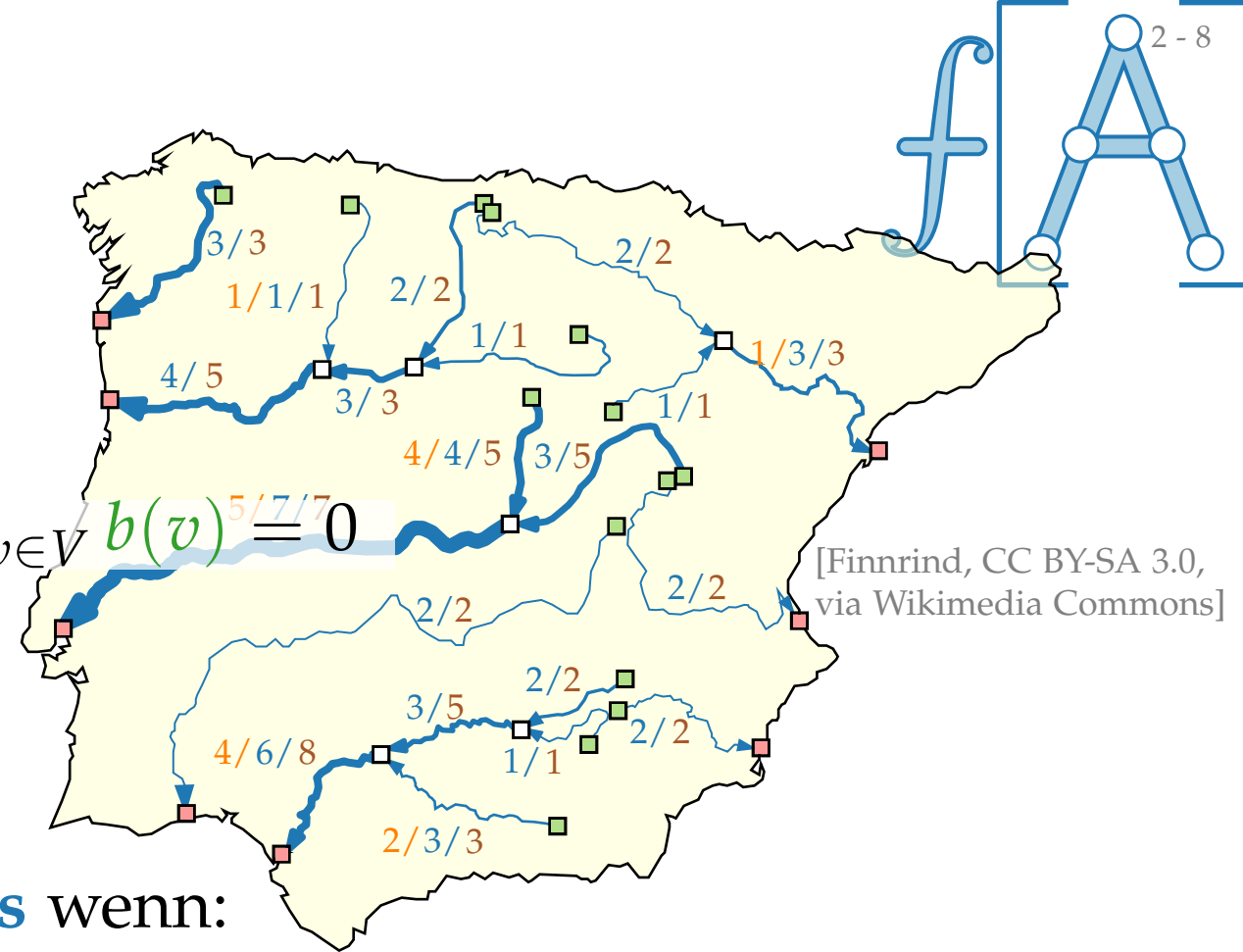
- gerichteter Graph $G = (V, E)$
- *Knotenproduktion/-verbrauch* $b: V \rightarrow \mathbb{R}$ mit $\sum_{v \in V} b(v) = 0$
- *untere Kantenschranken* $\ell: E \rightarrow \mathbb{R}_0^+$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **zulässiger Fluss** wenn:

$$\begin{aligned} \ell(u, v) &\leq f(u, v) \leq c(u, v) & \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) &= b(v) & \forall v \in V \end{aligned}$$

- *Kostenfunktion* $\text{cost}: E \rightarrow \mathbb{R}_0^+$

Ein **größter S - T -Fluss** ist ein S - T -Fluss, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



Allgemeine Flussnetzwerke

Flussnetzwerk $(G = (V, E); b; \ell; c; \text{cost})$ mit

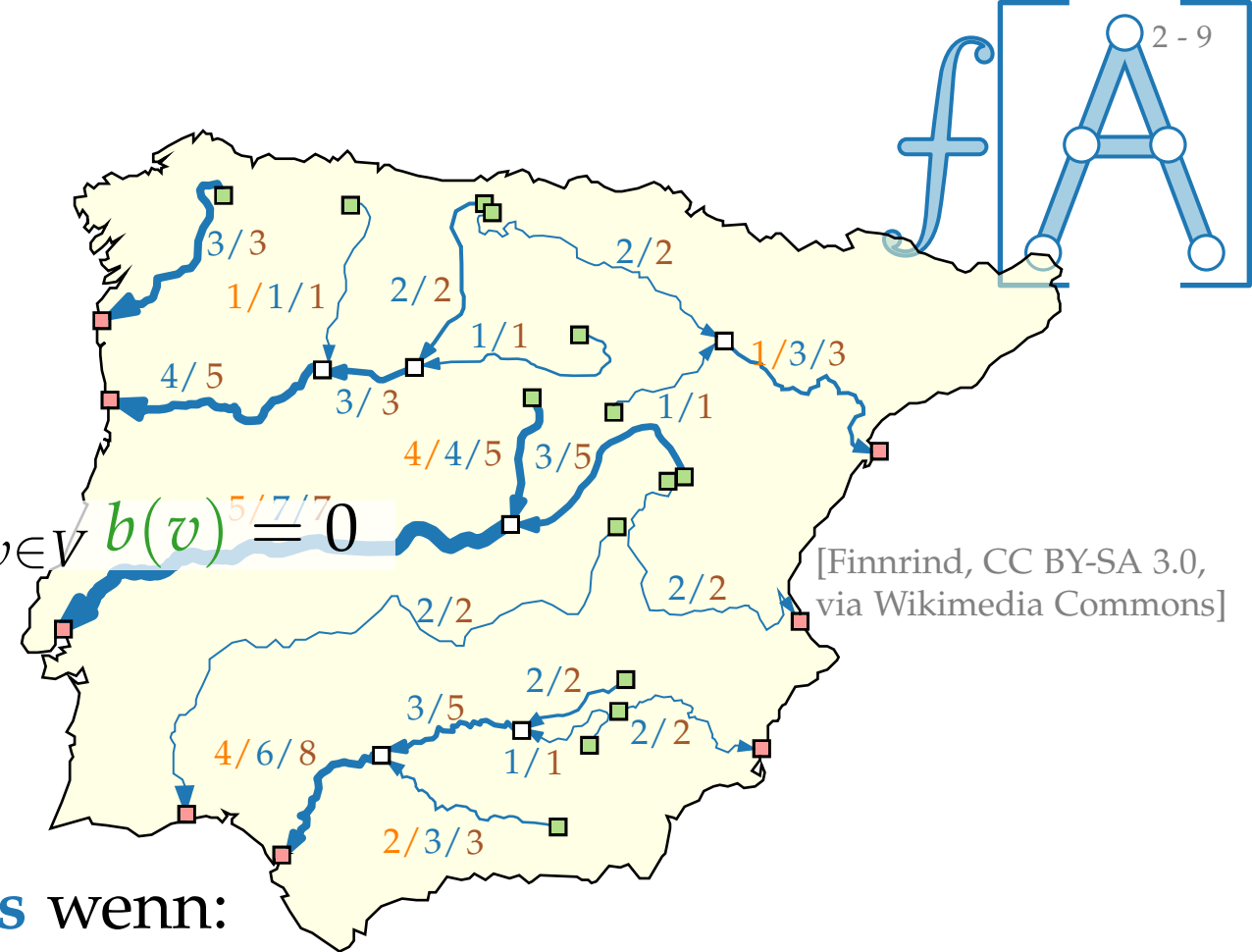
- gerichteter Graph $G = (V, E)$
- *Knotenproduktion/-verbrauch* $b: V \rightarrow \mathbb{R}$ mit $\sum_{v \in V} b(v) = 0$
- *untere Kantenschranken* $\ell: E \rightarrow \mathbb{R}_0^+$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **zulässiger Fluss** wenn:

$$\begin{aligned} \ell(u, v) &\leq f(u, v) \leq c(u, v) & \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) &= b(v) & \forall v \in V \end{aligned}$$

- *Kostenfunktion* $\text{cost}: E \rightarrow \mathbb{R}_0^+$ und $\text{cost}(f) := \sum_{(u, v) \in E} \text{cost}(u, v) \cdot f(u, v)$

Ein **größter S - T -Fluss** ist ein S - T -Fluss, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



Allgemeine Flussnetzwerke

Flussnetzwerk $(G = (V, E); b; \ell; c; \text{cost})$ mit

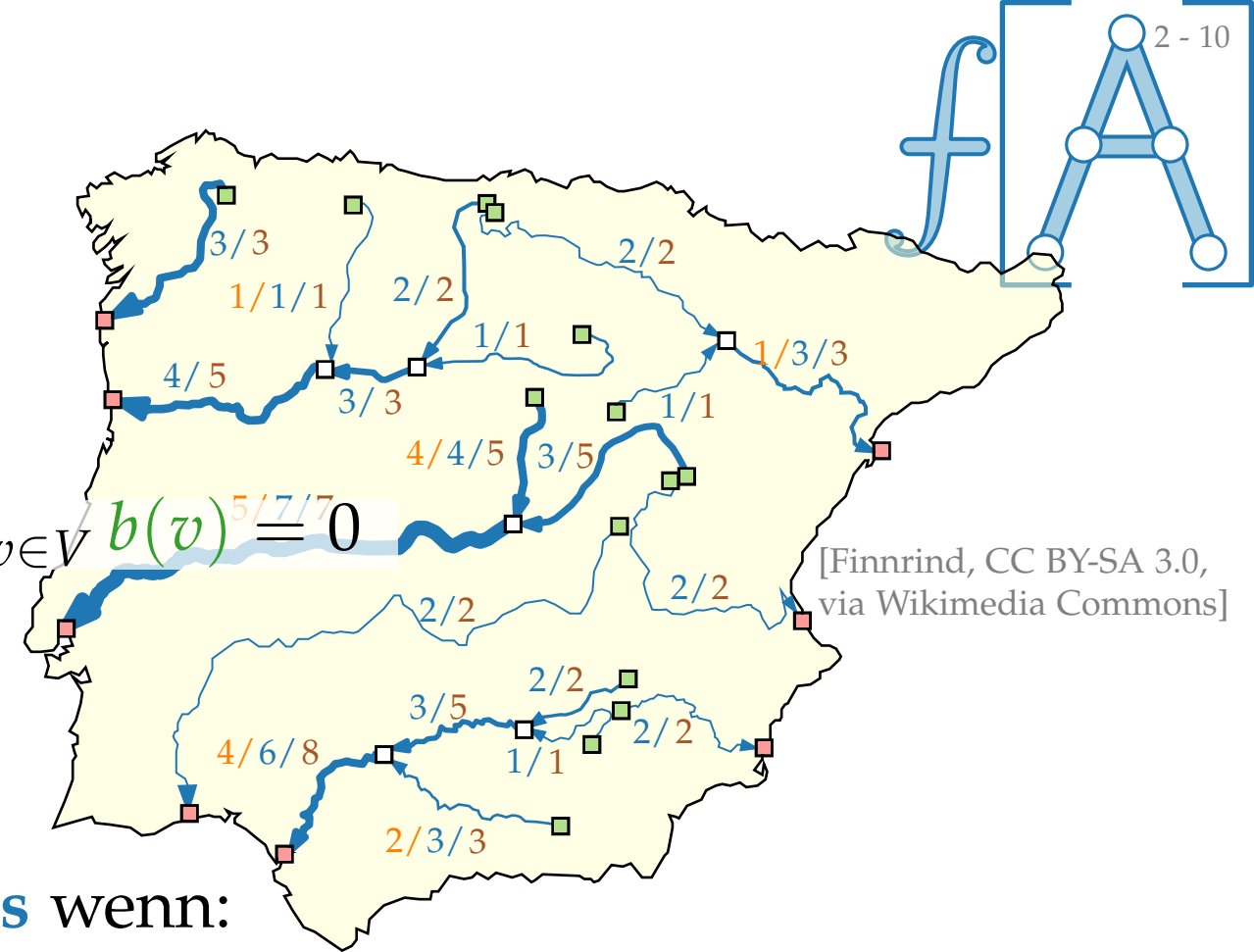
- gerichteter Graph $G = (V, E)$
- *Knotenproduktion/-verbrauch* $b: V \rightarrow \mathbb{R}$ mit $\sum_{v \in V} b(v) = 0$
- *untere Kantenschranken* $\ell: E \rightarrow \mathbb{R}_0^+$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **zulässiger Fluss** wenn:

$$\begin{aligned} \ell(u, v) &\leq f(u, v) \leq c(u, v) & \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) &= b(v) & \forall v \in V \end{aligned}$$

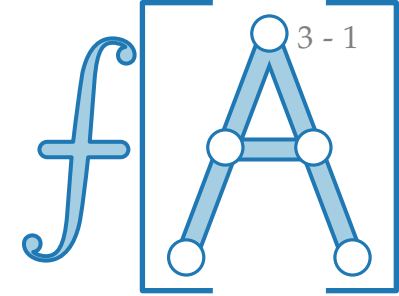
- *Kostenfunktion* $\text{cost}: E \rightarrow \mathbb{R}_0^+$ und $\text{cost}(f) := \sum_{(u, v) \in E} \text{cost}(u, v) \cdot f(u, v)$

Ein **kostenminimaler Fluss** ist ein zulässiger Fluss f , der $\text{cost}(f)$ minimiert.



[Finnrind, CC BY-SA 3.0,
via Wikimedia Commons]

Allgemeine Flussnetzwerke – Algorithmen



Polynomial Algorithms

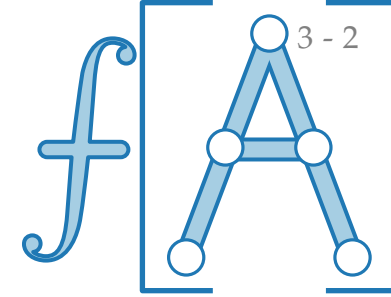
#	Due to	Year	Running Time
1	Edmonds and Karp	1972	$O((n + m') \log U S(n, m, nC))$
2	Rock	1980	$O((n + m') \log U S(n, m, nC))$
3	Rock	1980	$O(n \log C M(n, m, U))$
4	Bland and Jensen	1985	$O(m \log C M(n, m, U))$
5	Goldberg and Tarjan	1987	$O(nm \log (n^2/m) \log (nC))$
6	Goldberg and Tarjan	1988	$O(nm \log n \log (nC))$
7	Ahuja, Goldberg, Orlin and Tarjan	1988	$O(nm \log \log U \log (nC))$

Strongly Polynomial Algorithms

#	Due to	Year	Running Time
1	Tardos	1985	$O(m^4)$
2	Orlin	1984	$O((n + m')^2 \log n S(n, m))$
3	Fujishige	1986	$O((n + m')^2 \log n S(n, m))$
4	Galil and Tardos	1986	$O(n^2 \log n S(n, m))$
5	Goldberg and Tarjan	1987	$O(nm^2 \log n \log (n^2/m))$
6	Goldberg and Tarjan	1988	$O(nm^2 \log^2 n)$
7	Orlin (this paper)	1988	$O((n + m') \log n S(n, m))$

$S(n, m)$	$= O(m + n \log n)$	Fredman and Tarjan [1984]
$S(n, m, C)$	$= O(\min(m + n\sqrt{\log C}, (m \log \log C)))$	Ahuja, Mehlhorn, Orlin and Tarjan [1990] Van Emde Boas, Kaas and Zijlstra[1977]
$M(n, m)$	$= O(\min(nm + n^{2+\epsilon}, nm \log n))$ where ϵ is any fixed constant.	King, Rao, and Tarjan [1991]
$M(n, m, U)$	$= O(nm \log (\frac{n}{m}\sqrt{\log U} + 2))$	Ahuja, Orlin and Tarjan [1989]

Allgemeine Flussnetzwerke – Algorithmen



3 - 2

Polynomial Algorithms

#	Due to	Year	Running Time
1	Edmonds and Karp	1972	$O((n + m') \log U S(n, m, nC))$
2	Rock	1980	$O((n + m') \log U S(n, m, nC))$
3	Rock	1980	$O(n \log C M(n, m, U))$
4	Bland and Jensen	1985	$O(m \log C M(n, m, U))$
5	Goldberg and Tarjan	1987	$O(nm \log (n^2 / m) \log (nC))$
6	Goldberg and Tarjan	1988	$O(nm \log n \log (nC))$
7	Ahuja, Goldberg, Orlin and Tarjan	1988	$O(nm \log \log U \log (nC))$

Strongly Polynomial Algorithms

#	Due to	Year	Running Time
1	Tardos	1985	$O(m^4)$
2	Orlin	1984	$O((n + m')^2 \log n S(n, m))$
3	Fujishige	1986	$O((n + m')^2 \log n S(n, m))$
4	Galil and Tardos	1986	$O(n^2 \log n S(n, m))$
5	Goldberg and Tarjan	1987	$O(nm^2 \log n \log (n^2 / m))$
6	Goldberg and Tarjan	1988	$O(nm^2 \log^2 n)$
7	Orlin (this paper)	1988	$O((n + m') \log n S(n, m))$

$S(n, m)$	=	$O(m + n \log n)$	Fredman and Tarjan [1984]
$S(n, m, C)$	=	$O(\min(m + n\sqrt{\log C}, (m \log \log C)))$	Ahuja, Mehlhorn, Orlin and Tarjan [1990] Van Emde Boas, Kaas and Zijlstra[1977]
$M(n, m)$	=	$O(\min(nm + n^{2+\epsilon}, nm \log n))$ where ϵ is any fixed constant.	King, Rao, and Tarjan [1991]
$M(n, m, U)$	=	$O(nm \log (\frac{n}{m} \sqrt{\log U} + 2))$	Ahuja, Orlin and Tarjan [1989]

Theorem.

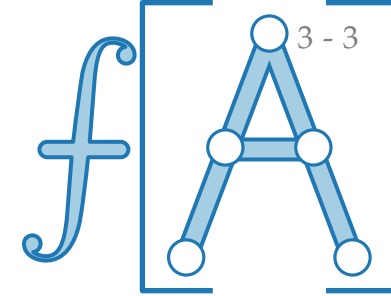
[Orlin 1991]

Ein kostenminimaler Fluss kann in $O(n^2 \log^2 n + m^2 \log n)$ Zeit gefunden werden.



James B. Orlin
*1953

Allgemeine Flussnetzwerke – Algorithmen



Polynomial Algorithms

#	Due to	Year	Running Time
1	Edmonds and Karp	1972	$O((n + m') \log U S(n, m, nC))$
2	Rock	1980	$O((n + m') \log U S(n, m, nC))$
3	Rock	1980	$O(n \log C M(n, m, U))$
4	Bland and Jensen	1985	$O(m \log C M(n, m, U))$
5	Goldberg and Tarjan	1987	$O(nm \log (n^2 / m) \log (nC))$
6	Goldberg and Tarjan	1988	$O(nm \log n \log (nC))$
7	Ahuja, Goldberg, Orlin and Tarjan	1988	$O(nm \log \log U \log (nC))$

Strongly Polynomial Algorithms

#	Due to	Year	Running Time
1	Tardos	1985	$O(m^4)$
2	Orlin	1984	$O((n + m')^2 \log n S(n, m))$
3	Fujishige	1986	$O((n + m')^2 \log n S(n, m))$
4	Galil and Tardos	1986	$O(n^2 \log n S(n, m))$
5	Goldberg and Tarjan	1987	$O(nm^2 \log n \log (n^2 / m))$
6	Goldberg and Tarjan	1988	$O(nm^2 \log^2 n)$
7	Orlin (this paper)	1988	$O((n + m') \log n S(n, m))$

$S(n, m)$	$= O(m + n \log n)$	Fredman and Tarjan [1984]
$S(n, m, C)$	$= O(\text{Min}(m + n\sqrt{\log C}, (m \log \log C)))$	Ahuja, Mehlhorn, Orlin and Tarjan [1990] Van Emde Boas, Kaas and Zijlstra[1977]
$M(n, m)$	$= O(\text{min}(nm + n^{2+\epsilon}, nm \log n))$ where ϵ is any fixed constant.	King, Rao, and Tarjan [1991]
$M(n, m, U)$	$= O(nm \log (\frac{n}{m} \sqrt{\log U} + 2))$	Ahuja, Orlin and Tarjan [1989]

[Orlin 1991]

Theorem.

[Orlin 1991]

Ein kostenminimaler Fluss kann in $O(n^2 \log^2 n + m^2 \log n)$ Zeit gefunden werden.



James B. Orlin
*1953

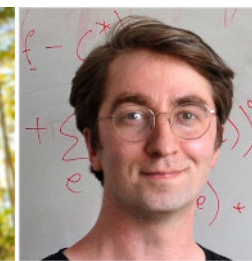
Theorem.

[Chen et al. 2022]

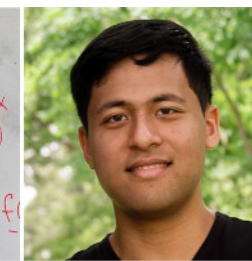
Ein kostenminimaler Fluss kann in $O(m^{1+o(1)})$ Zeit gefunden werden.



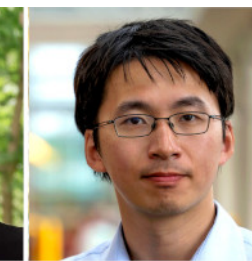
Li
Chen



Rasmus
Kyng



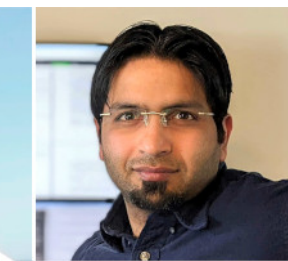
Yang
Liu



Richard
Peng

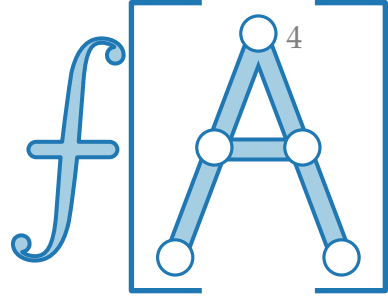


Maximilian
Probst
Gutenberg



Sushant
Sachdeva

Das Max-Flow-Min-Cut-Theorem



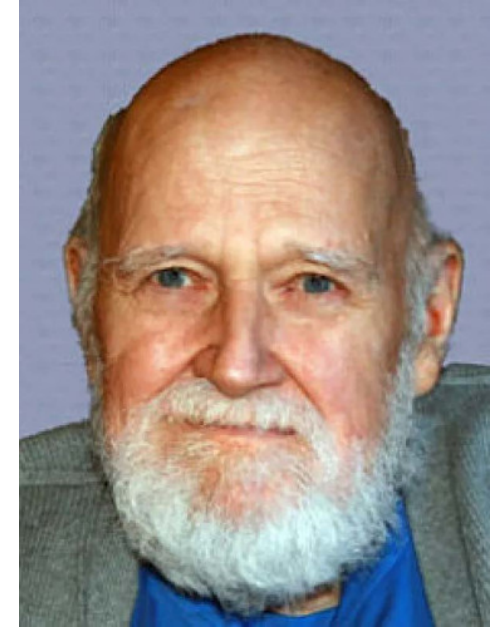
Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$. Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S,T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

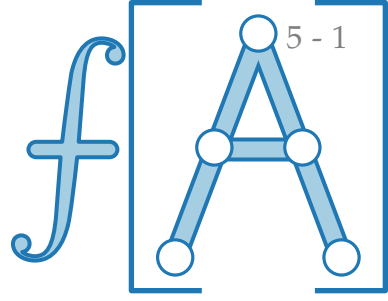
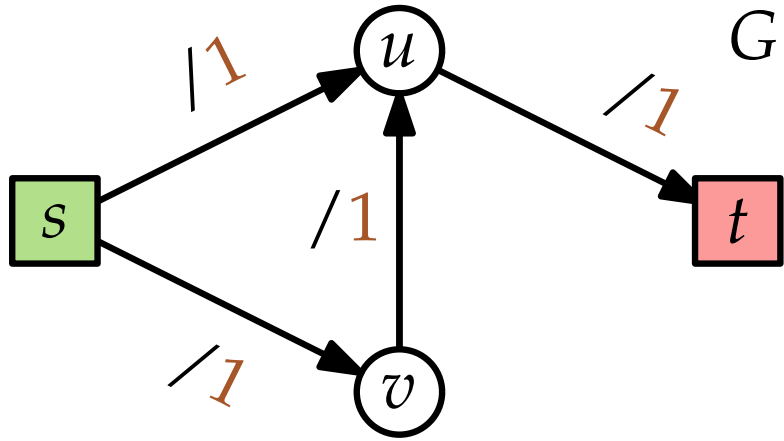
Lester R. Ford Jr.
*1927 Houston, TX
†2017



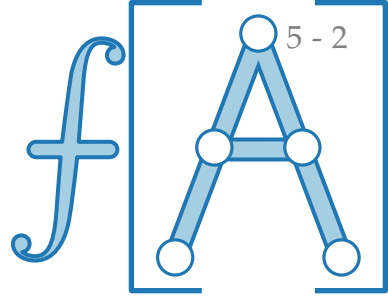
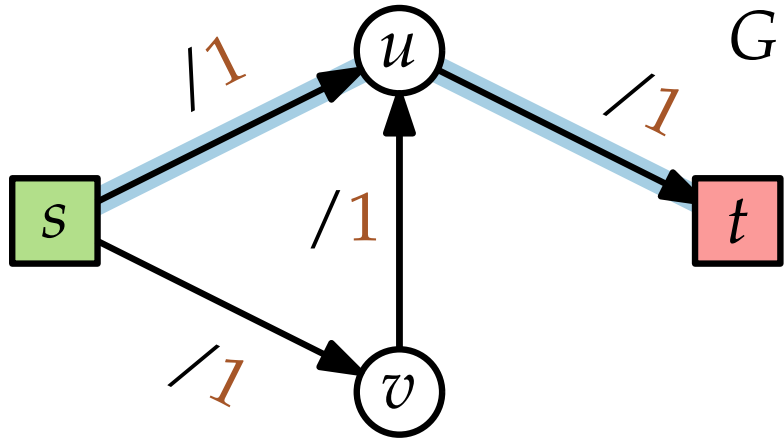
Delbert R. Fulkerson
*1924 Tamm, IL
†1976 Ithaca, NY



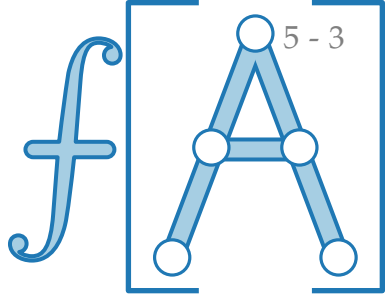
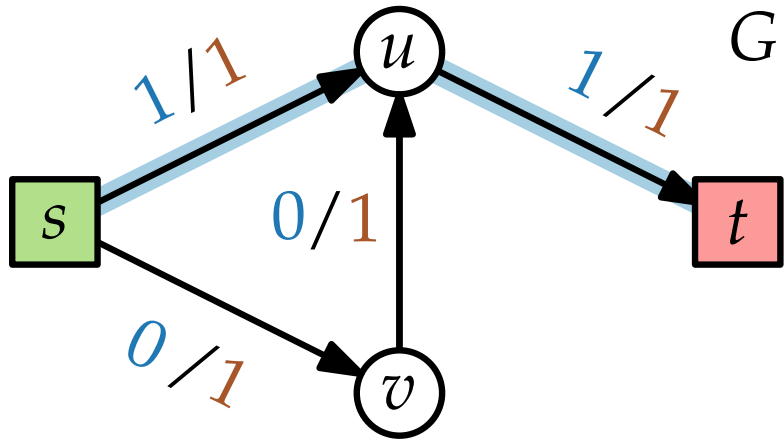
Ganzzahligkeitssatz



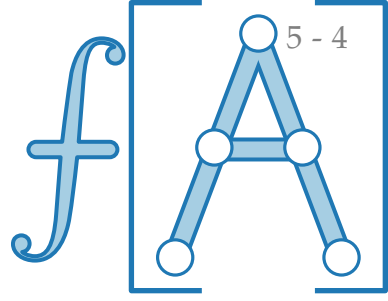
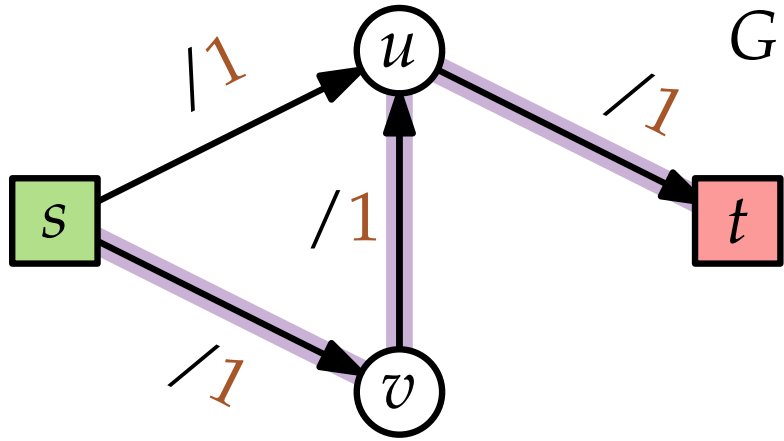
Ganzzahligkeitssatz



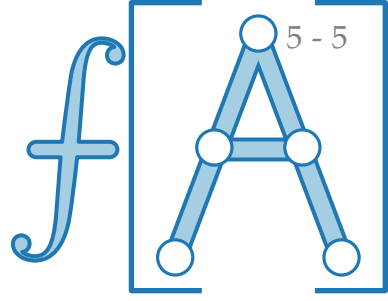
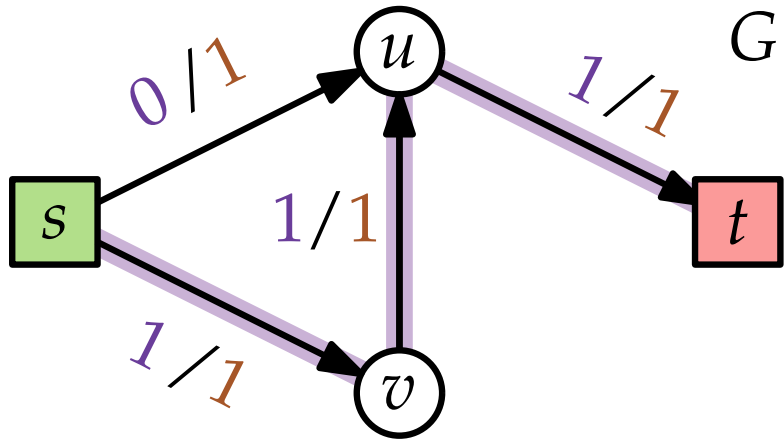
Ganzzahligkeitssatz



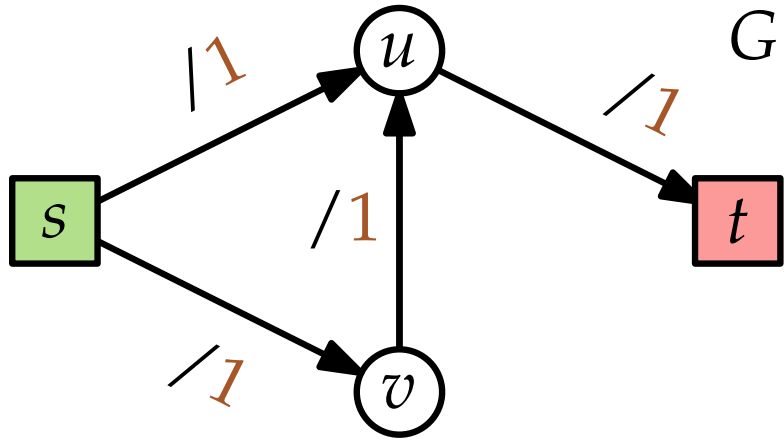
Ganzzahligkeitssatz



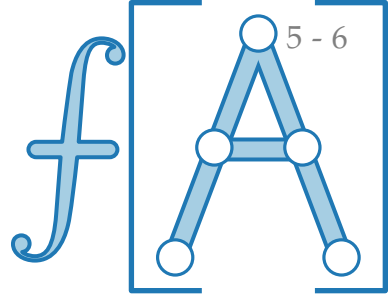
Ganzzahligkeitssatz



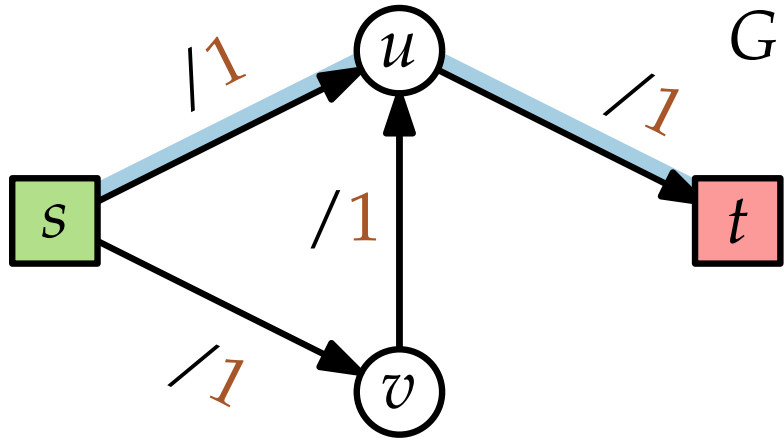
Ganzzahligkeitssatz



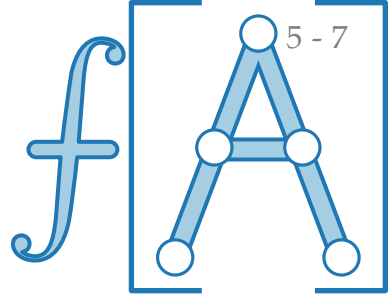
Gibt es *noch* einen maximalen Fluss?



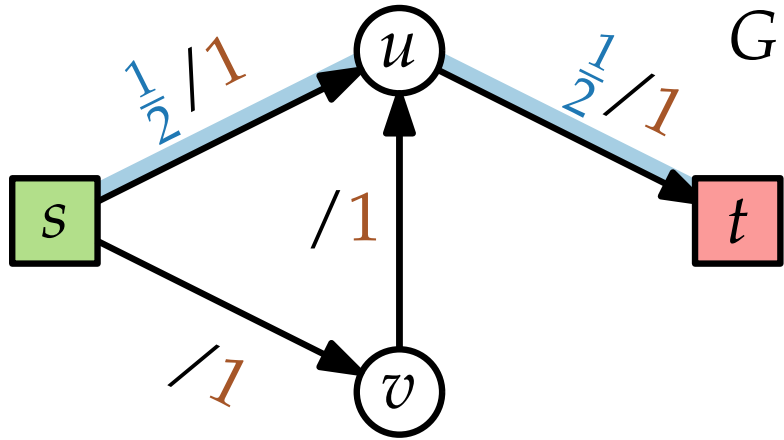
Ganzzahligkeitssatz



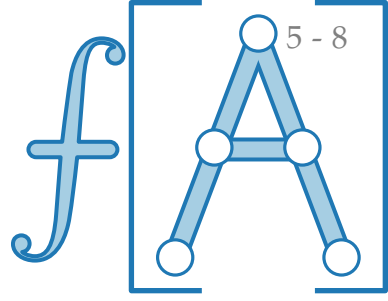
Gibt es *noch* einen maximalen Fluss?



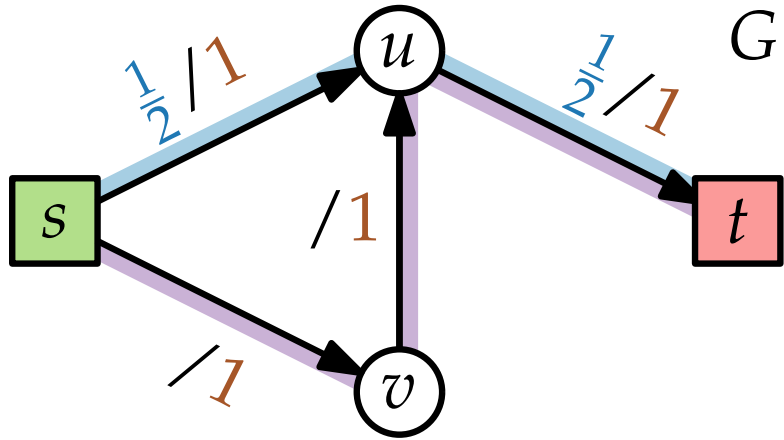
Ganzzahligkeitssatz



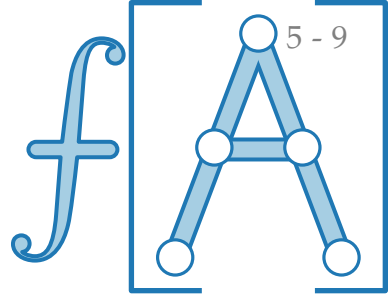
Gibt es *noch* einen maximalen Fluss?



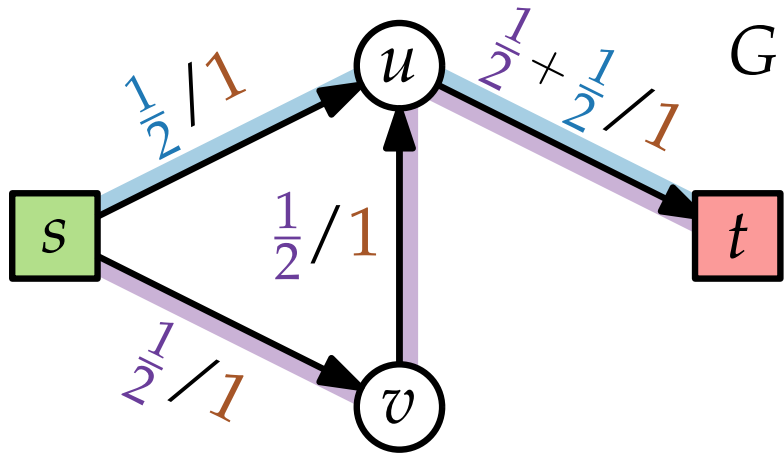
Ganzzahligkeitssatz



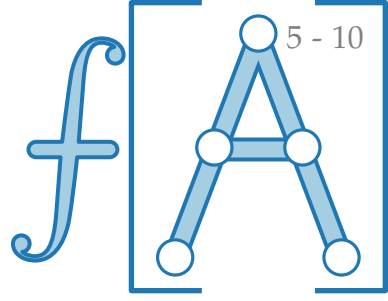
Gibt es *noch* einen maximalen Fluss?



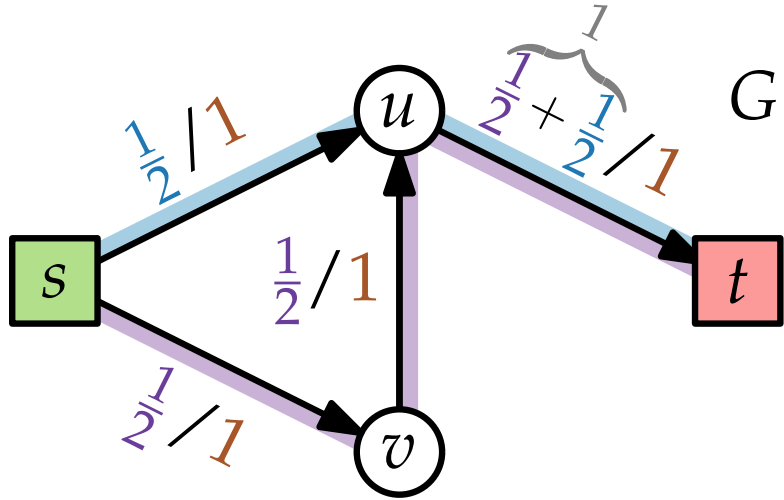
Ganzzahligkeitssatz



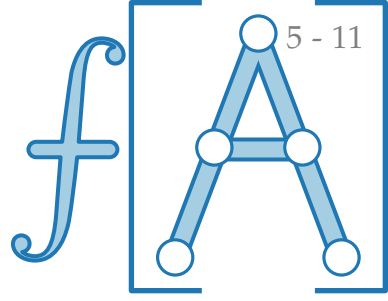
Gibt es *noch* einen maximalen Fluss?



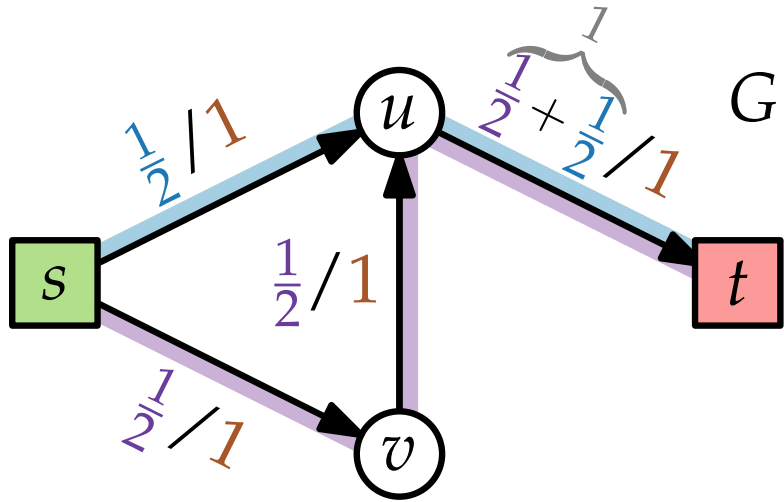
Ganzzahligkeitssatz



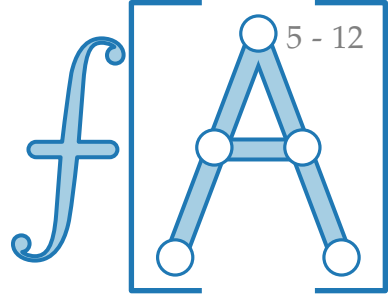
Gibt es *noch* einen maximalen Fluss?



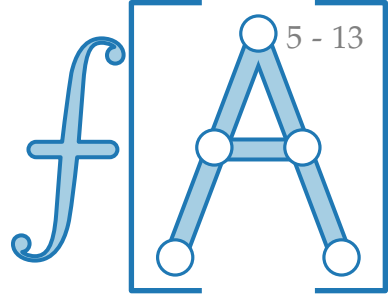
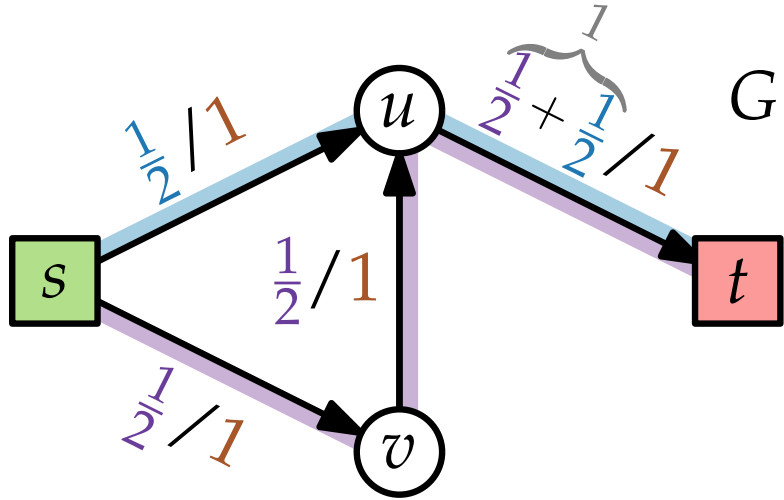
Ganzzahligkeitssatz



Gibt es *noch* einen maximalen Fluss? Ja!



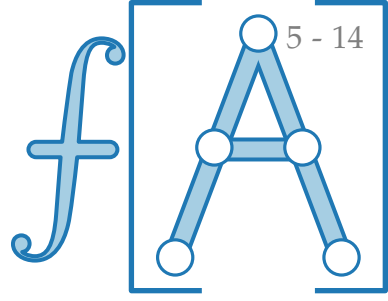
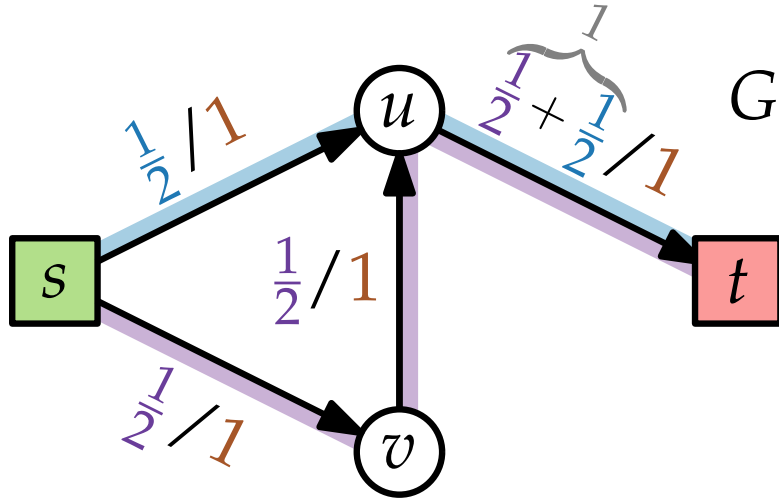
Ganzzahligkeitssatz



Gibt es *noch* einen maximalen Fluss? Ja!

- Wenn es mind. *zwei* verschiedene maximale Flüsse gibt, so gibt es ein ganzes *Kontinuum* maximaler Flüsse.

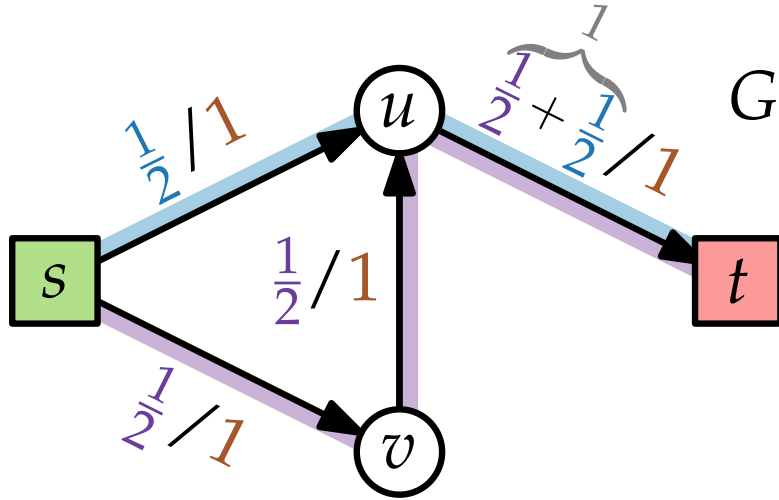
Ganzzahligkeitssatz



Gibt es *noch* einen maximalen Fluss? Ja!

- Wenn es mind. *zwei* verschiedene maximale Flüsse gibt, so gibt es ein ganzes *Kontinuum* maximaler Flüsse.
- Aber:

Ganzzahligkeitssatz

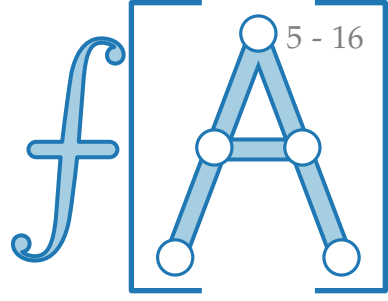
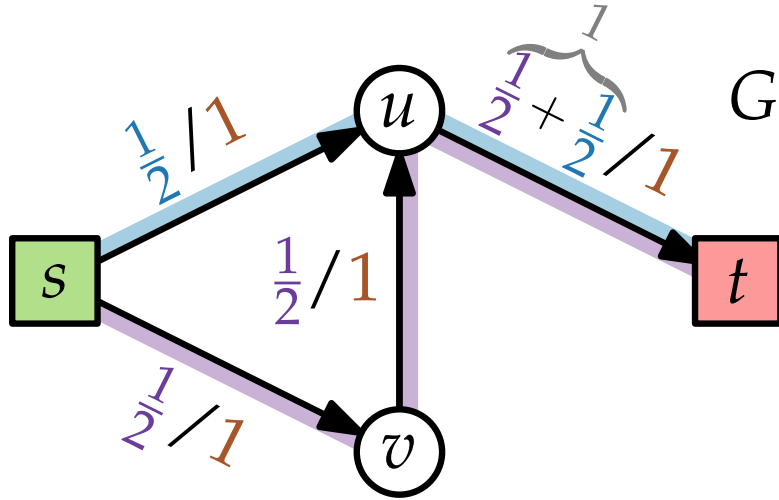


Gibt es *noch* einen maximalen Fluss? Ja!

- Wenn es mind. *zwei* verschiedene maximale Flüsse gibt, so gibt es ein ganzes *Kontinuum* maximaler Flüsse.
- Aber:

Korollar. Sind alle Kapazitäten ganzzahlig, d.h. $c: E \rightarrow \mathbb{N}$,

Ganzzahligkeitssatz

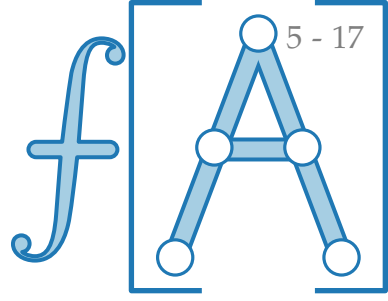
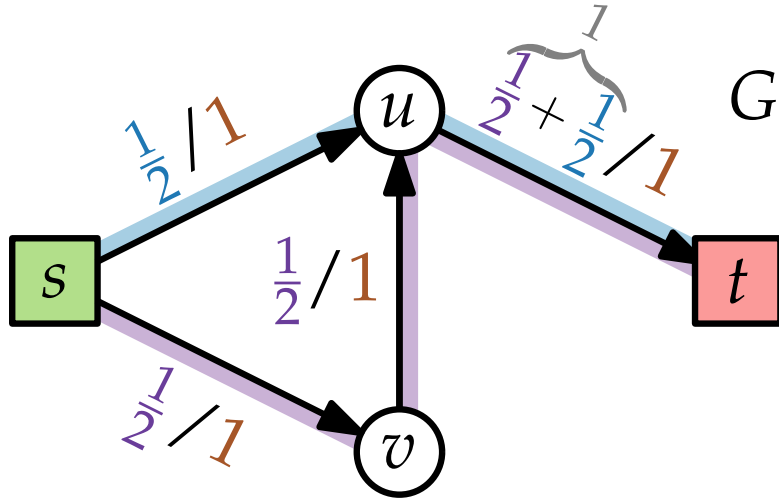


Gibt es *noch* einen maximalen Fluss? Ja!

- Wenn es mind. *zwei* verschiedene maximale Flüsse gibt, so gibt es ein ganzes *Kontinuum* maximaler Flüsse.
- Aber:

Korollar. Sind alle Kapazitäten ganzzahlig, d.h. $c: E \rightarrow \mathbb{N}$, so existiert ein maximaler Fluss, der ganzzahlig ist.

Ganzzahligkeitssatz



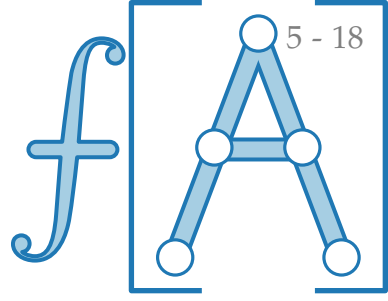
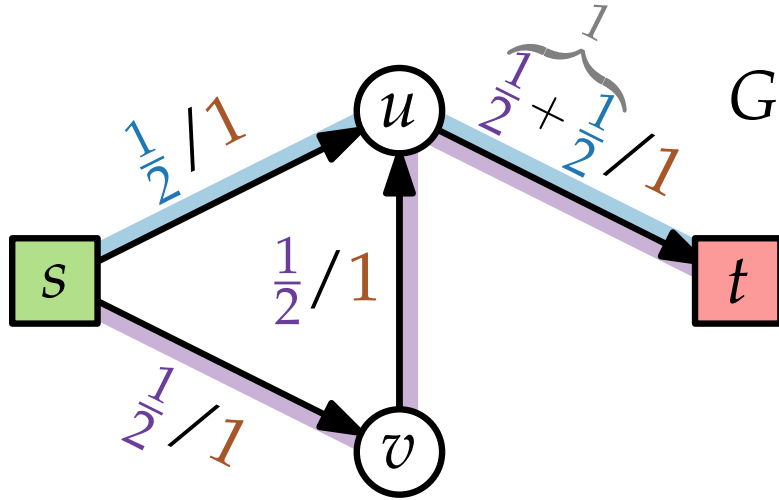
Gibt es *noch* einen maximalen Fluss? Ja!

- Wenn es mind. *zwei* verschiedene maximale Flüsse gibt, so gibt es ein ganzes *Kontinuum* maximaler Flüsse.
- Aber:

Korollar. Sind alle Kapazitäten ganzzahlig, d.h. $c: E \rightarrow \mathbb{N}$, so existiert ein maximaler Fluss, der ganzzahlig ist.

Beweis. ?

Ganzzahligkeitssatz

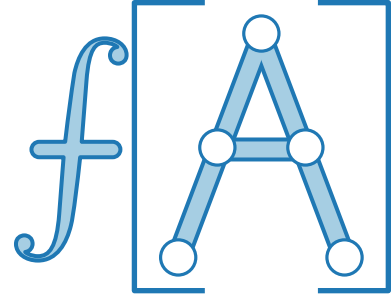


Gibt es *noch* einen maximalen Fluss? Ja!

- Wenn es mind. *zwei* verschiedene maximale Flüsse gibt, so gibt es ein ganzes *Kontinuum* maximaler Flüsse.
- Aber:

Korollar. Sind alle Kapazitäten ganzzahlig, d.h. $c: E \rightarrow \mathbb{N}$, so existiert ein maximaler Fluss, der ganzzahlig ist.

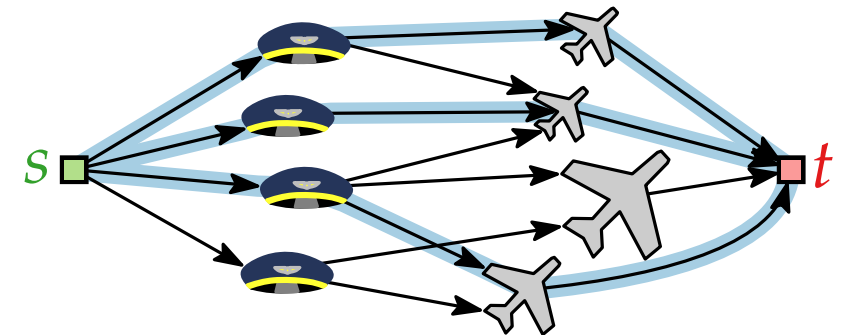
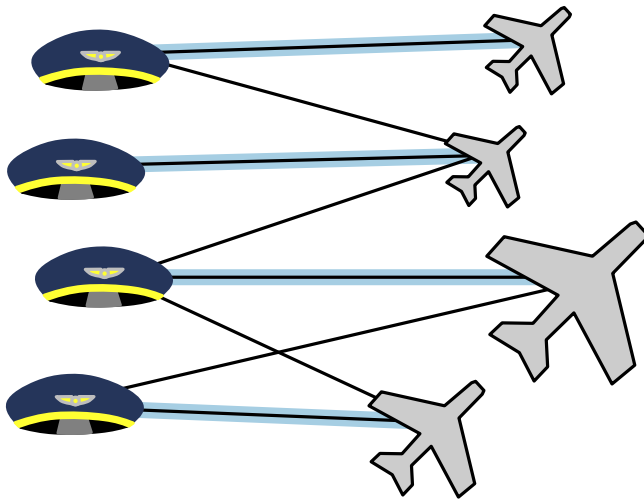
Beweis. Wende FORDFULKERSON/EDMONDSKARP/PUSH-RELABEL an! $\square$



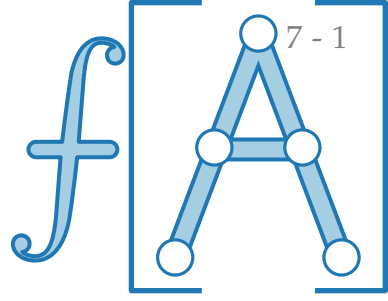
Fortgeschrittene Algorithmen

9. Vorlesung Graphalgorithmen III: Paarungen / Matchings

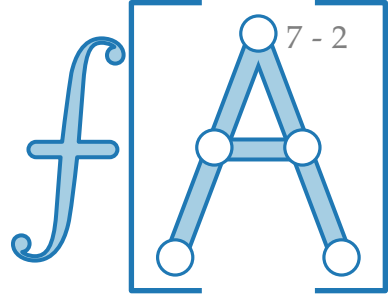
Teil II: Paarungen



Motivation – Zuordnung von Piloten zu Flügen

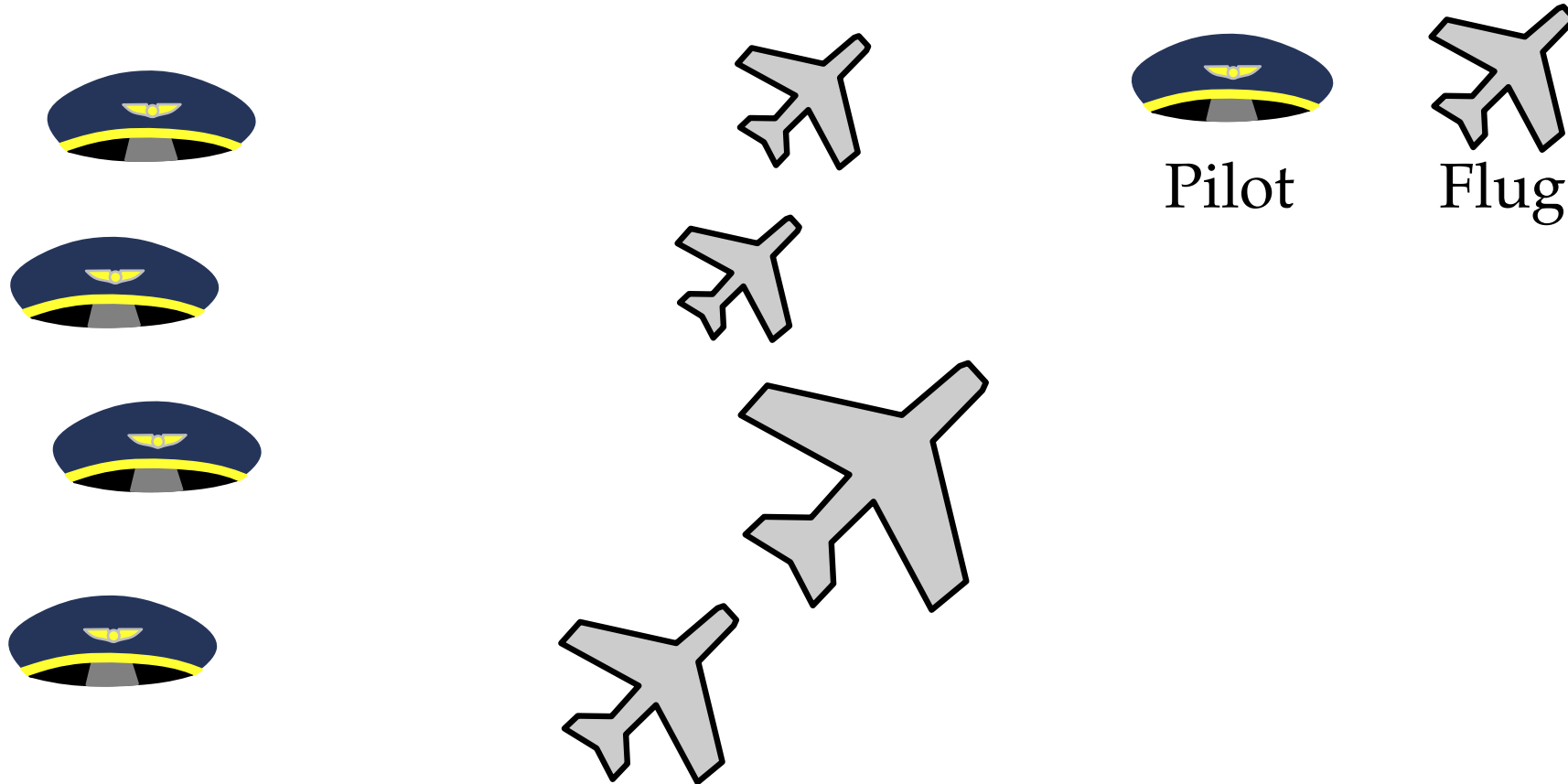
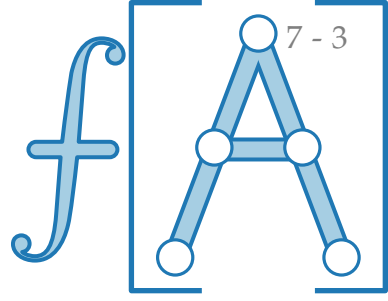


Motivation – Zuordnung von Piloten zu Flügen

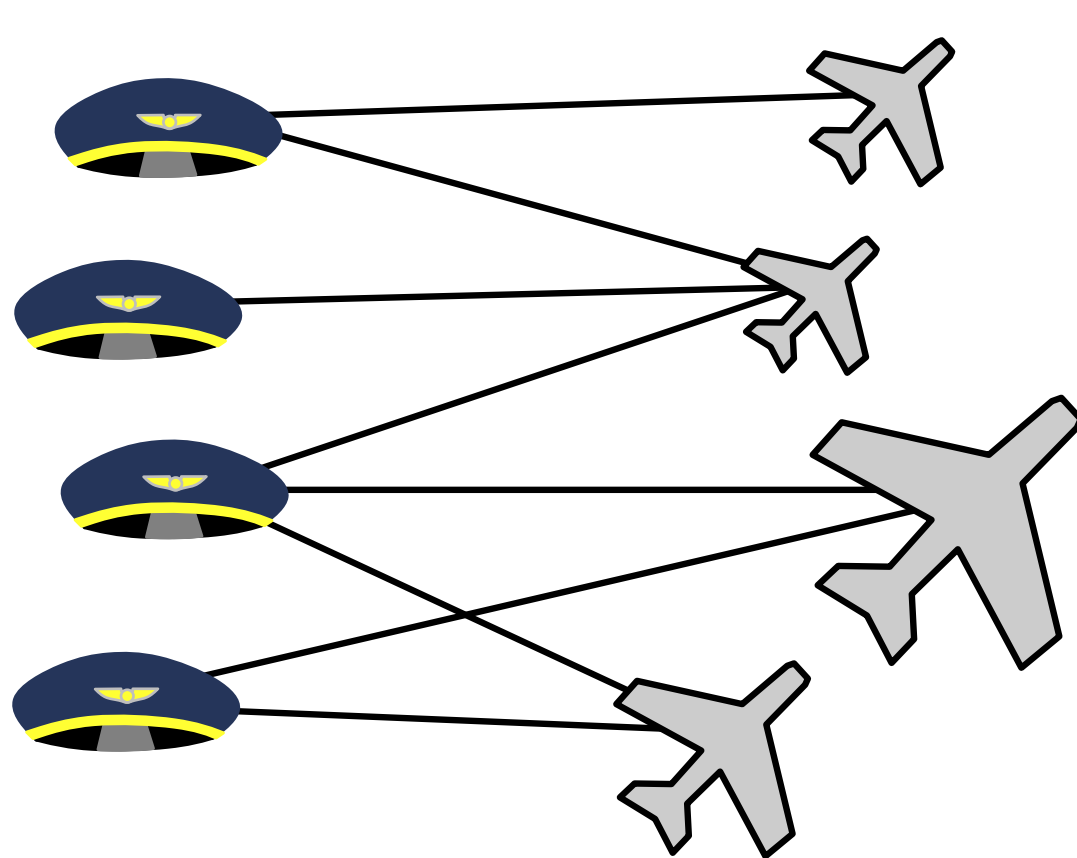
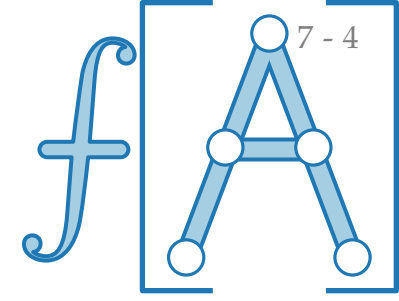


Pilot

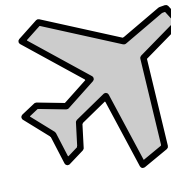
Motivation – Zuordnung von Piloten zu Flügen



Motivation – Zuordnung von Piloten zu Flügen



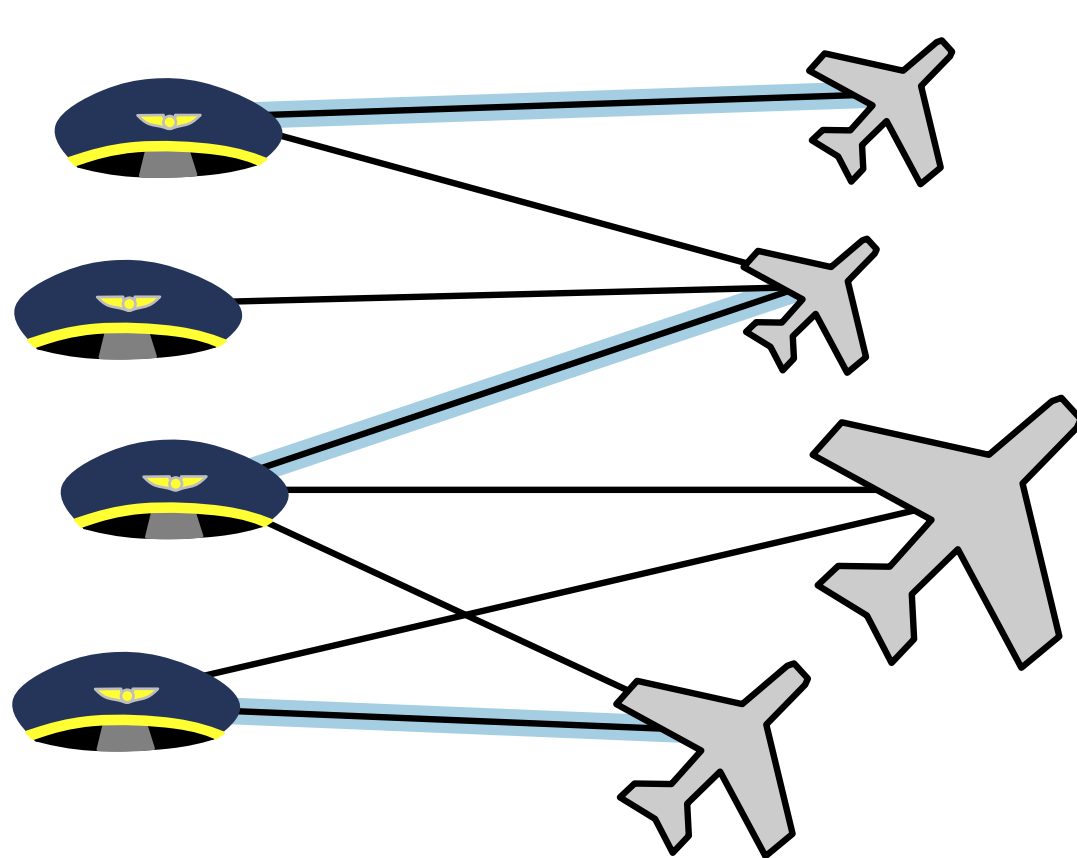
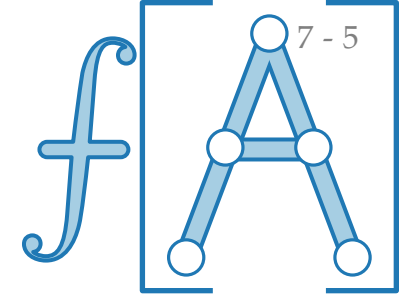
Pilot



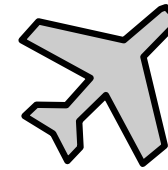
Flug

/ Pilot darf Flugzeug fliegen

Motivation – Zuordnung von Piloten zu Flügen



Pilot

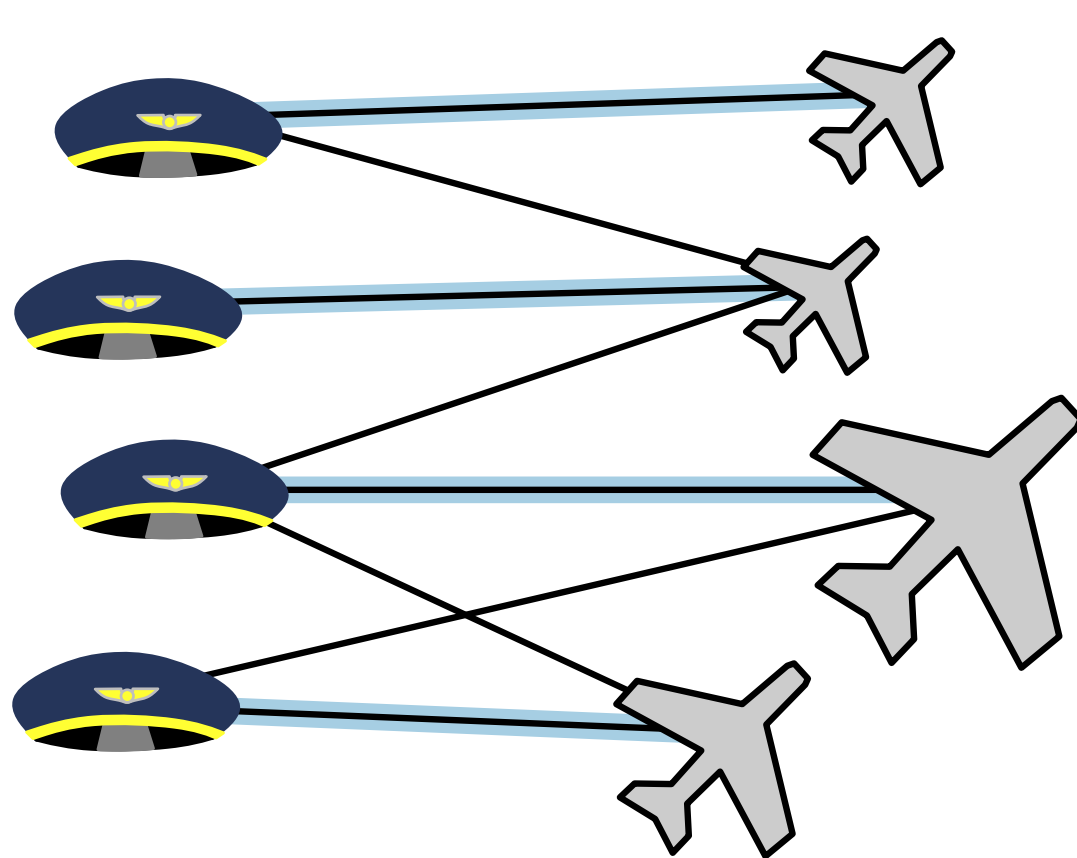
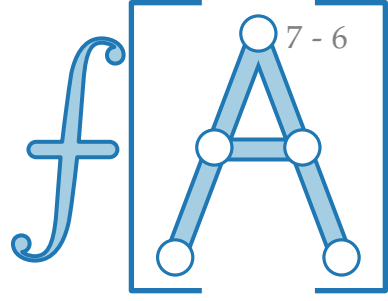


Flug

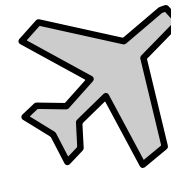
/ Pilot darf Flugzeug fliegen

/ Zuordnung der Piloten

Motivation – Zuordnung von Piloten zu Flügen



Pilot

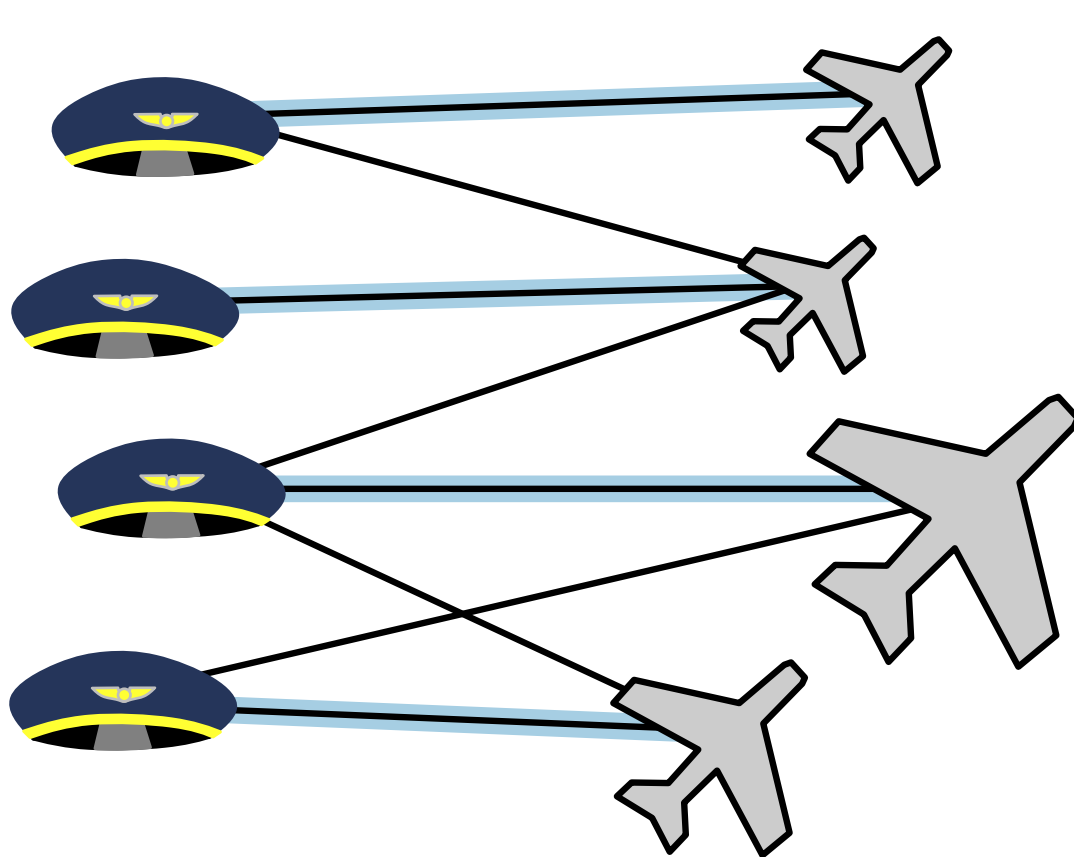
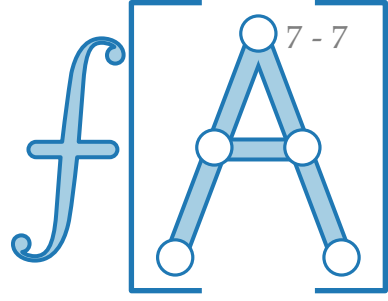


Flug

/ Pilot darf Flugzeug fliegen

/ Zuordnung der Piloten

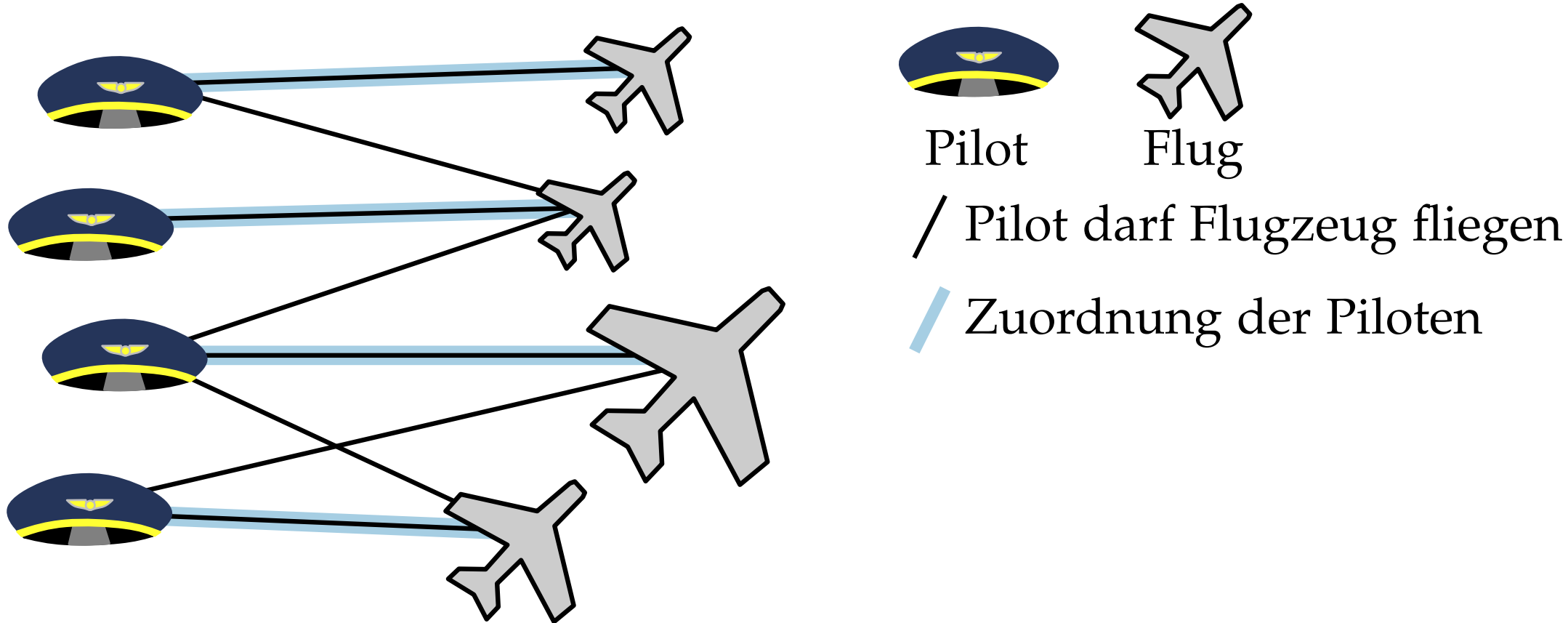
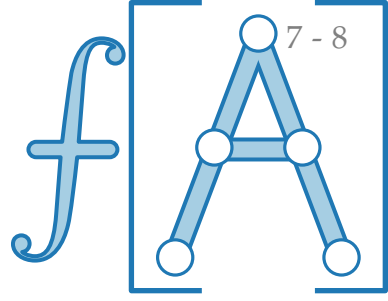
Motivation – Zuordnung von Piloten zu Flügen



 Pilot  Flug
/ Pilot darf Flugzeug fliegen
/ Zuordnung der Piloten

Ziel: Finde eine Zuordnung, sodass:

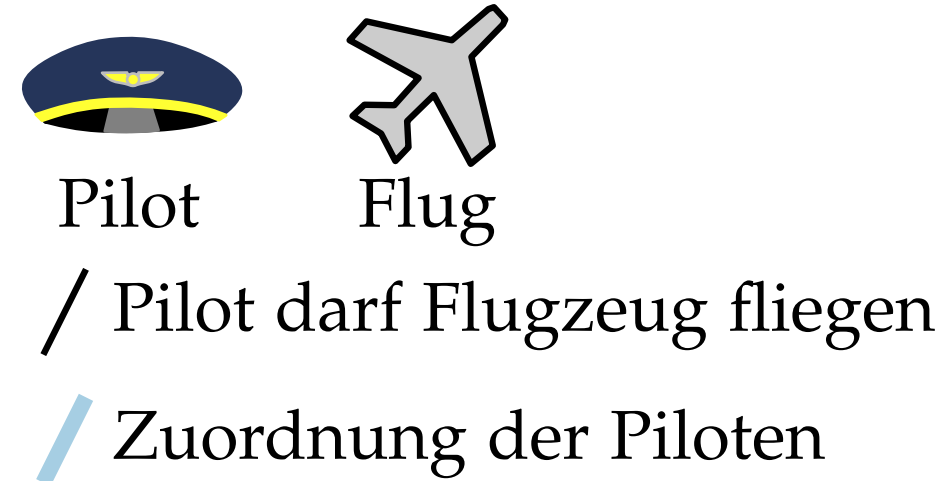
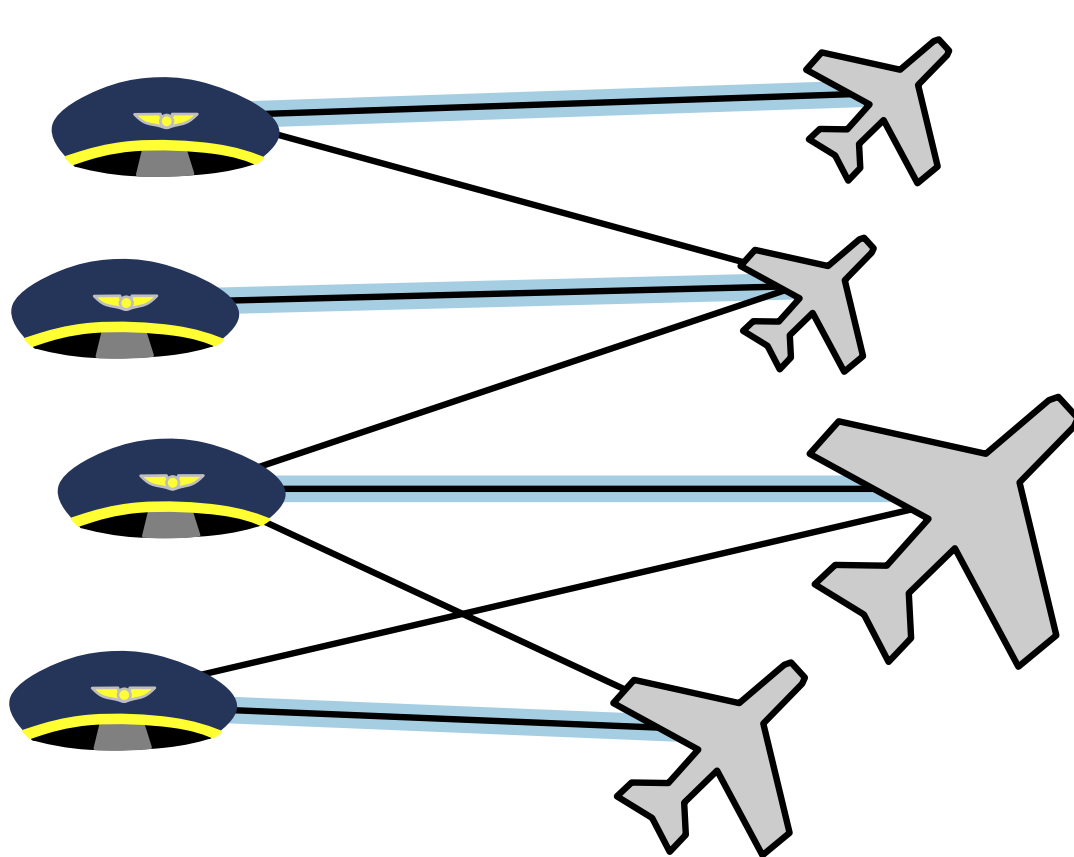
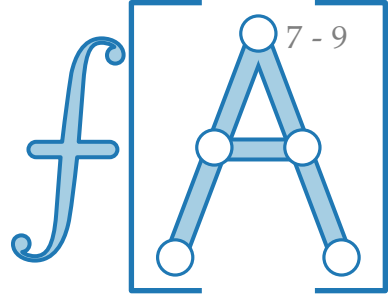
Motivation – Zuordnung von Piloten zu Flügen



Ziel: Finde eine Zuordnung, sodass:

- Jeder Pilot maximal ein Flugzeug fliegt, jedes Flugzeug von maximal einem Pilot geflogen wird.

Motivation – Zuordnung von Piloten zu Flügen

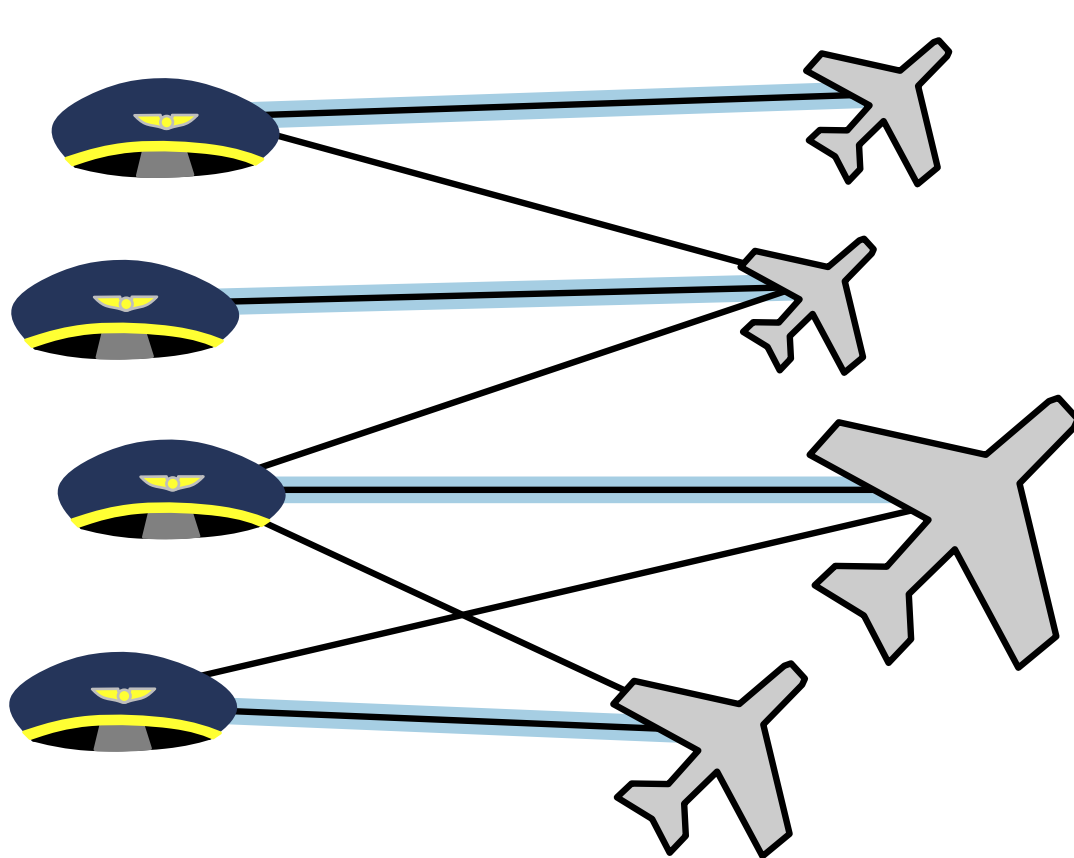
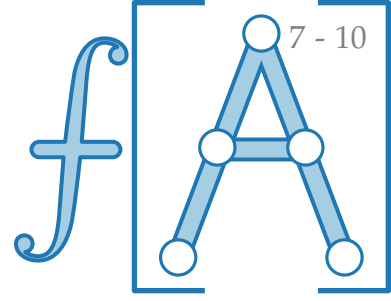


Ziel: Finde eine Zuordnung, sodass:

- Jeder Pilot maximal ein Flugzeug fliegt, jedes Flugzeug von maximal einem Pilot geflogen wird.

Paarung

Motivation – Zuordnung von Piloten zu Flügen



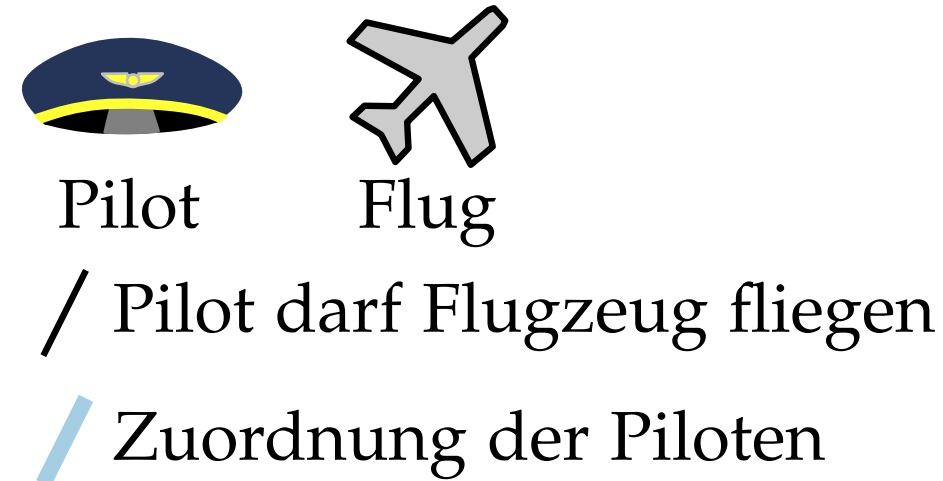
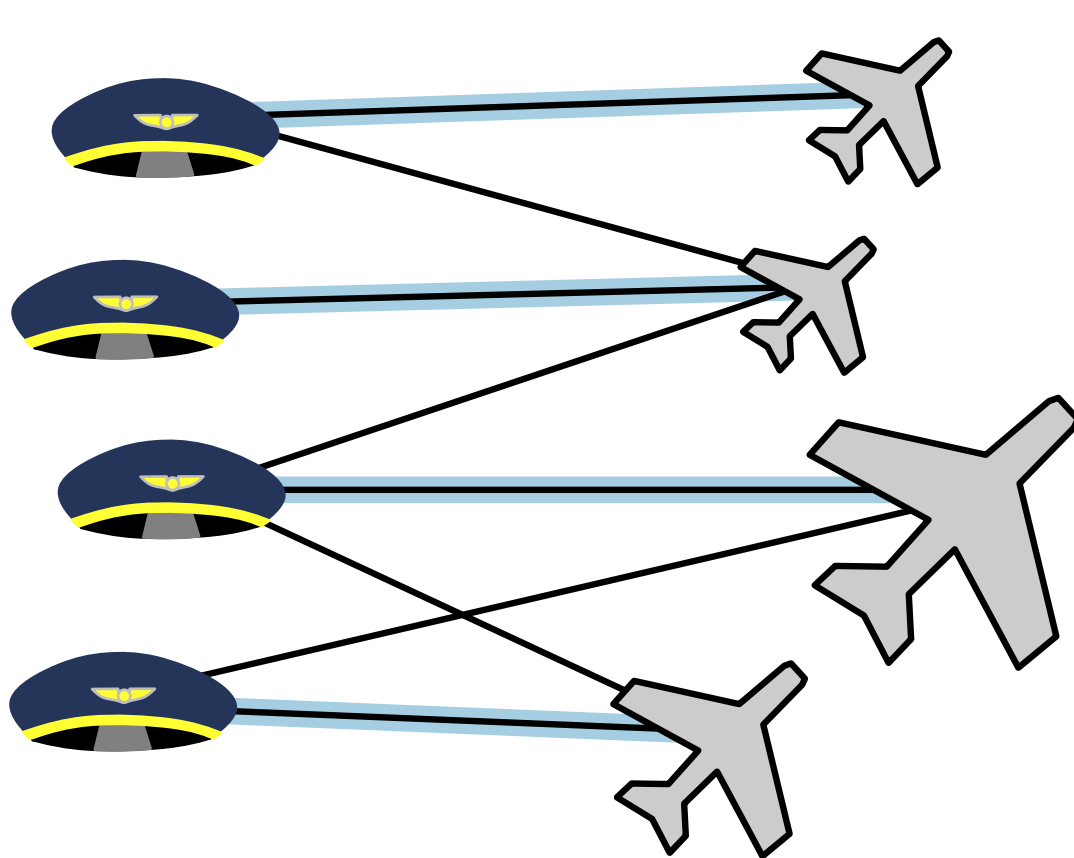
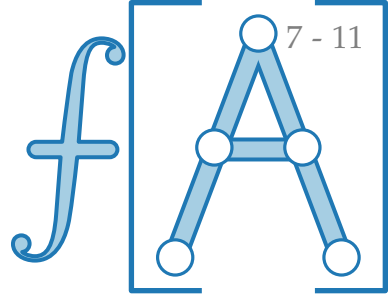
Pilot Flug
/ Pilot darf Flugzeug fliegen
/ Zuordnung der Piloten

Ziel: Finde eine Zuordnung, sodass:

- Jeder Pilot maximal ein Flugzeug fliegt, jedes Flugzeug von maximal einem Pilot geflogen wird.
- Möglichst viele Flugzeuge besetzt (bzw. Piloten beschäftigt) sind.

Paarung

Motivation – Zuordnung von Piloten zu Flügen



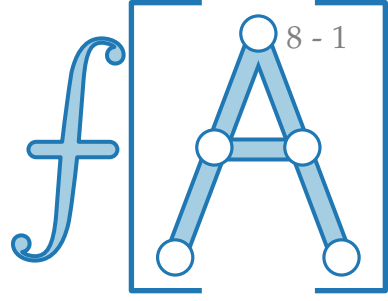
Ziel: Finde eine Zuordnung, sodass:

- Jeder Pilot maximal ein Flugzeug fliegt, jedes Flugzeug von maximal einem Pilot geflogen wird.
- Möglichst viele Flugzeuge besetzt (bzw. Piloten beschäftigt) sind.

Paarung

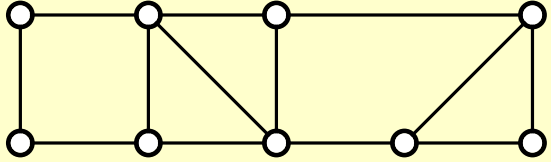
größte

Paarungen (Matchings)

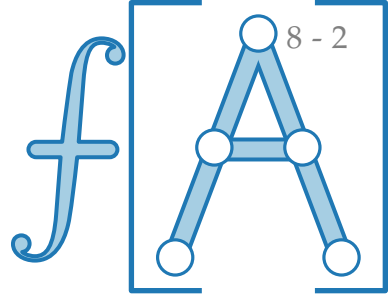


Def.

Sei $G = (V, E)$ ein ungerichteter Graph.



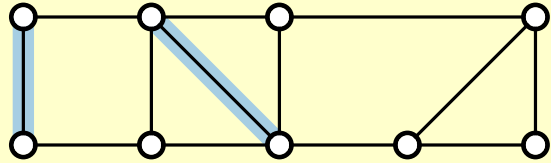
Paarungen (Matchings)



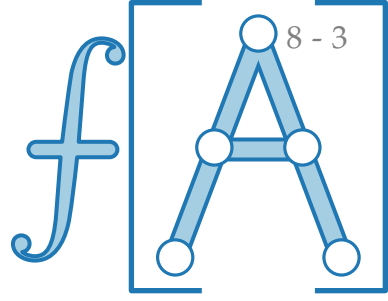
Def.

Sei $G = (V, E)$ ein ungerichteter Graph.

$P \subseteq E$ ist eine **Paarung** (engl. *matching*),
wenn je zwei Kanten in P keinen gleichen
Endpunkt haben.

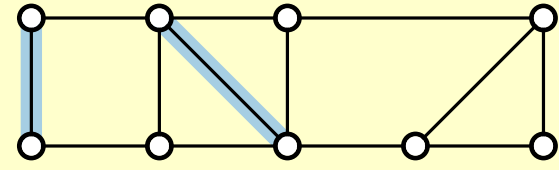


Paarungen (Matchings)

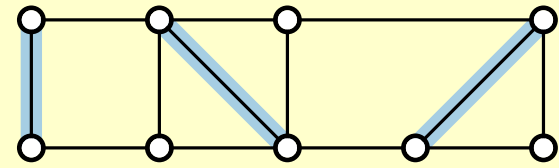


Def.

Sei $G = (V, E)$ ein ungerichteter Graph.

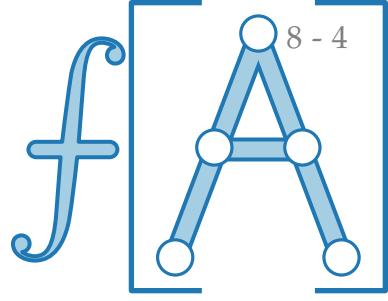


$P \subseteq E$ ist eine **Paarung** (engl. *matching*), wenn je zwei Kanten in P keinen gleichen Endpunkt haben.



Falls für jede Kante $e \notin P$ gilt, dass $P \cup \{e\}$ keine Paarung ist, so ist P **nicht erweiterbar** (engl. *maximal*).

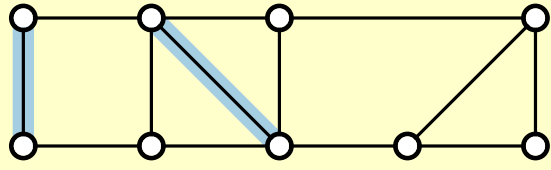
Paarungen (Matchings)



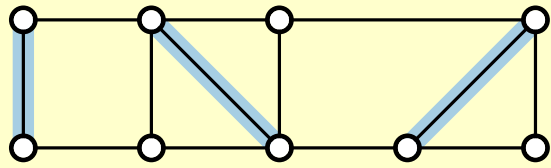
Def.

Sei $G = (V, E)$ ein ungerichteter Graph.

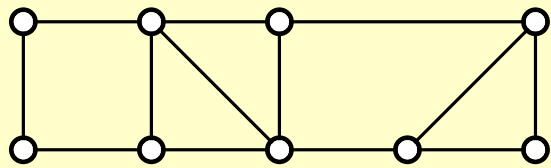
$P \subseteq E$ ist eine **Paarung** (engl. *matching*),
wenn je zwei Kanten in P keinen gleichen
Endpunkt haben.



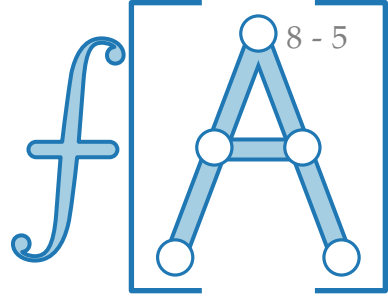
Falls für jede Kante $e \notin P$ gilt,
dass $P \cup \{e\}$ keine Paarung ist,
so ist P **nicht erweiterbar** (engl. *maximal*).



Falls für alle Paarungen P' in G gilt,
dass $|P'| \leq |P|$,
so ist P eine **größte Paarung** (engl. *maximum*).



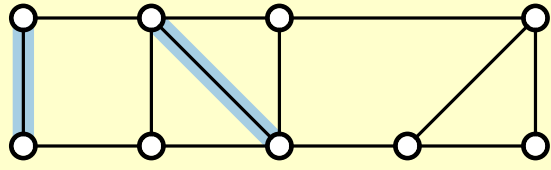
Paarungen (Matchings)



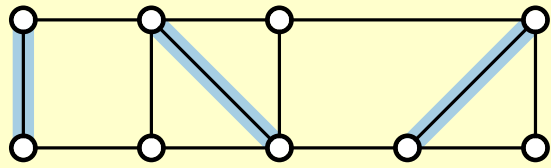
Def.

Sei $G = (V, E)$ ein ungerichteter Graph.

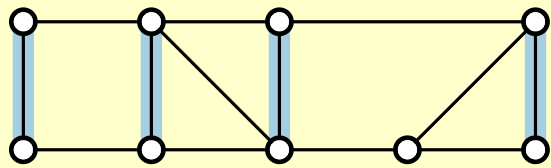
$P \subseteq E$ ist eine **Paarung** (engl. *matching*),
wenn je zwei Kanten in P keinen gleichen
Endpunkt haben.



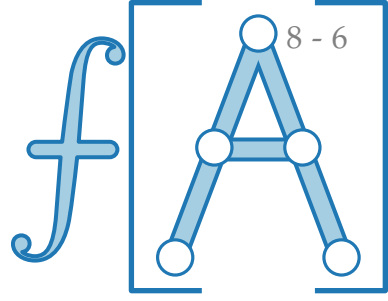
Falls für jede Kante $e \notin P$ gilt,
dass $P \cup \{e\}$ keine Paarung ist,
so ist P **nicht erweiterbar** (engl. *maximal*).



Falls für alle Paarungen P' in G gilt,
dass $|P'| \leq |P|$,
so ist P eine **größte Paarung** (engl. *maximum*).



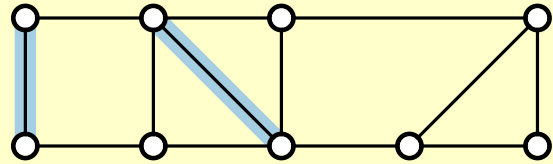
Paarungen (Matchings)



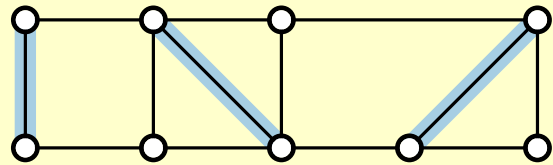
Def.

Sei $G = (V, E)$ ein ungerichteter Graph.

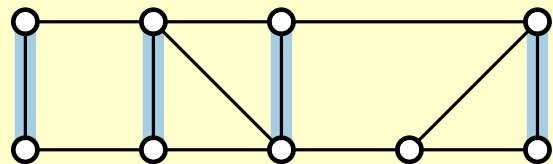
$P \subseteq E$ ist eine **Paarung** (engl. *matching*),
wenn je zwei Kanten in P keinen gleichen
Endpunkt haben.



Falls für jede Kante $e \notin P$ gilt,
dass $P \cup \{e\}$ keine Paarung ist,
so ist P **nicht erweiterbar** (engl. *maximal*).

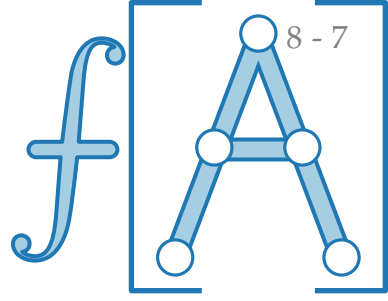


Falls für alle Paarungen P' in G gilt,
dass $|P'| \leq |P|$,
so ist P eine **größte Paarung** (engl. *maximum*).



Falls jeder Knoten in G durch P *gepaart* ist,
so ist P eine **perfekte Paarung** (engl. *perfect*).

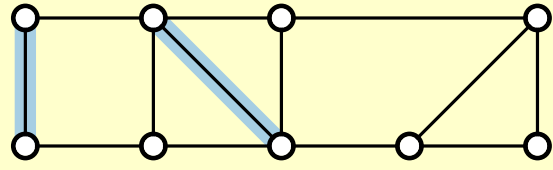
Paarungen (Matchings)



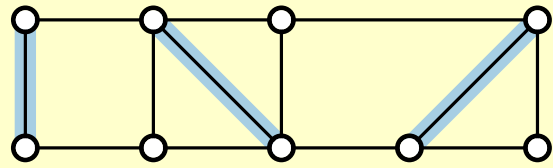
Def.

Sei $G = (V, E)$ ein ungerichteter Graph.

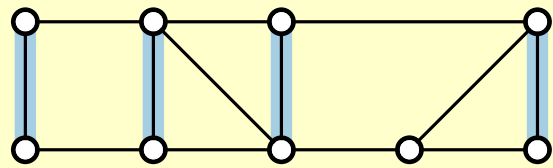
$P \subseteq E$ ist eine **Paarung** (engl. *matching*),
wenn je zwei Kanten in P keinen gleichen
Endpunkt haben.



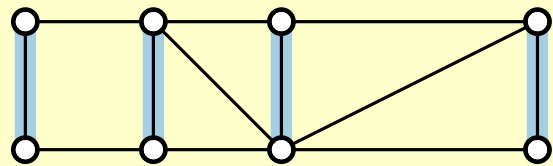
Falls für jede Kante $e \notin P$ gilt,
dass $P \cup \{e\}$ keine Paarung ist,
so ist P **nicht erweiterbar** (engl. *maximal*).



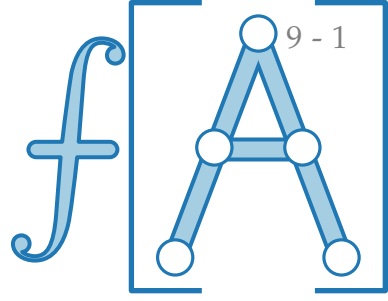
Falls für alle Paarungen P' in G gilt,
dass $|P'| \leq |P|$,
so ist P eine **größte Paarung** (engl. *maximum*).



Falls jeder Knoten in G durch P *gepaart* ist,
so ist P eine **perfekte Paarung** (engl. *perfect*).



Problemstellung

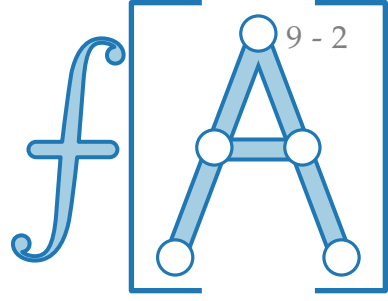


Def. $P \subseteq E$ ist eine **Paarung** (engl. *matching*), wenn je zwei Kanten in P keinen gleichen Endpunkt haben.

Problem. GRÖSSTE PAARUNG

Finde eine möglichst große Paarung P in G .

Problemstellung



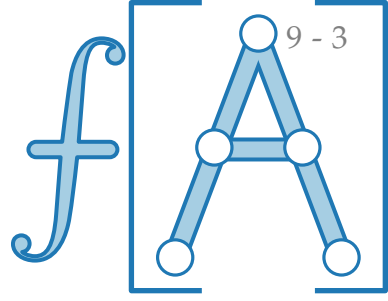
Def. $P \subseteq E$ ist eine **Paarung** (engl. *matching*), wenn je zwei Kanten in P keinen gleichen Endpunkt haben.

Problem. GRÖSSTE PAARUNG

Finde eine möglichst große Paarung P in G .

Bemerkung. Man könnte auch jeder Kante ein Gewicht zuweisen und dann die Summe der Gewichte gewählter Kanten maximieren.

Problemstellung



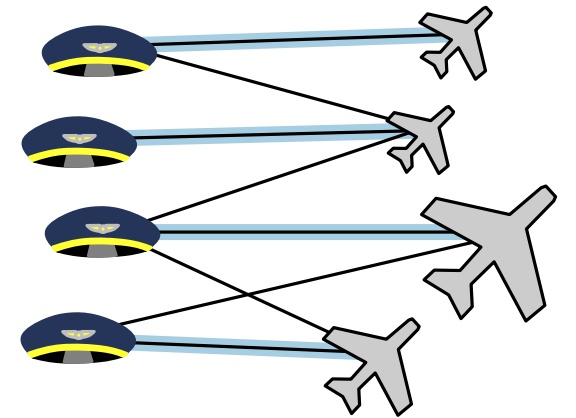
Def. $P \subseteq E$ ist eine **Paarung** (engl. *matching*), wenn je zwei Kanten in P keinen gleichen Endpunkt haben.

Problem. GRÖSSTE PAARUNG

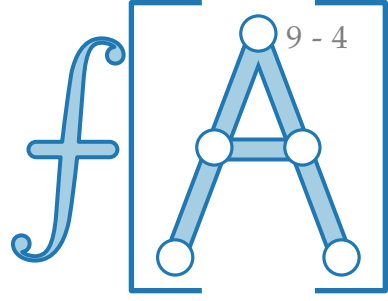
Finde eine möglichst große Paarung P in G .

Bemerkung.

Man könnte auch jeder Kante ein Gewicht zuweisen und dann die Summe der Gewichte gewählter Kanten maximieren.



Problemstellung



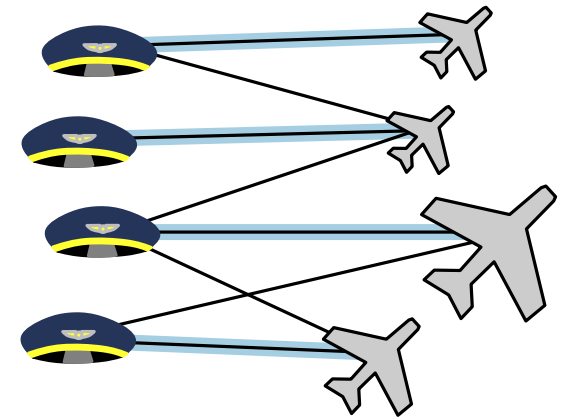
Def. $P \subseteq E$ ist eine **Paarung** (engl. *matching*), wenn je zwei Kanten in P keinen gleichen Endpunkt haben.

Problem. GRÖSSTE PAARUNG

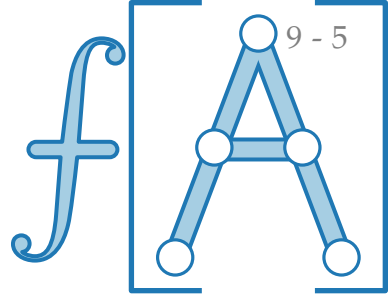
Finde eine möglichst große Paarung P in G .

Bemerkung. Man könnte auch jeder Kante ein Gewicht zuweisen und dann die Summe der Gewichte gewählter Kanten maximieren.

Einschränkung. Wir nehmen im Folgenden an, dass G **bipartit** ist.



Problemstellung



Def. $P \subseteq E$ ist eine **Paarung** (engl. *matching*), wenn je zwei Kanten in P keinen gleichen Endpunkt haben.

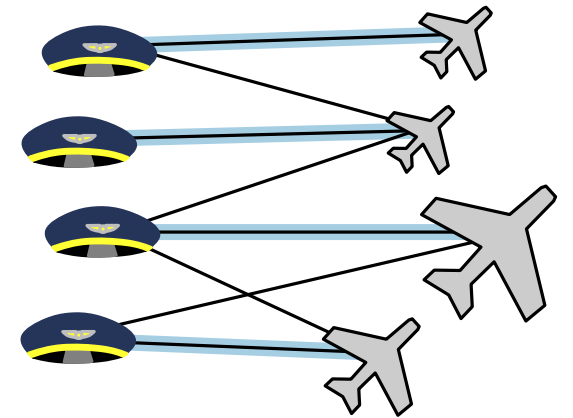
Problem. GRÖSSTE PAARUNG

Finde eine möglichst große Paarung P in G .

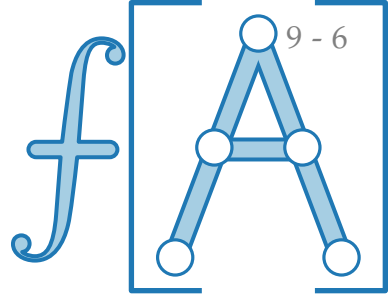
Bemerkung. Man könnte auch jeder Kante ein Gewicht zuweisen und dann die Summe der Gewichte gewählter Kanten maximieren.

Einschränkung. Wir nehmen im Folgenden an, dass G **bipartit** ist.

Def. Ein Graph $G = (\underbrace{\text{Knoten}}_{\text{links}} \cup \underbrace{\text{Knoten}}_{\text{rechts}}, E)$ ist **bipartit**, wenn jede Kante in E genau einen Knoten aus $\underbrace{\text{Knoten}}_{\text{links}}$ mit genau einem Knoten aus $\underbrace{\text{Knoten}}_{\text{rechts}}$ verbindet.



Problemstellung



Def. $P \subseteq E$ ist eine **Paarung** (engl. *matching*), wenn je zwei Kanten in P keinen gleichen Endpunkt haben.

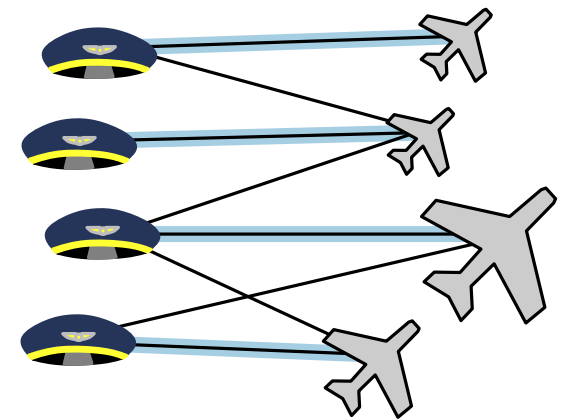
Problem. GRÖSSTE PAARUNG

Finde eine möglichst große Paarung P in G .

Bemerkung. Man könnte auch jeder Kante ein Gewicht zuweisen und dann die Summe der Gewichte gewählter Kanten maximieren.

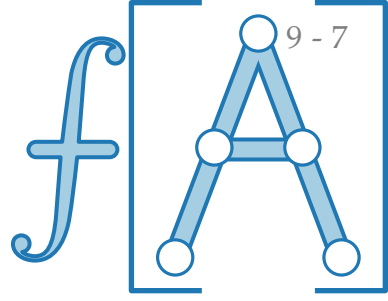
Einschränkung. Wir nehmen im Folgenden an, dass G **bipartit** ist.

Def. Ein Graph $G = (\underbrace{\text{Knoten}}_{\text{links}} \cup \underbrace{\text{Knoten}}_{\text{rechts}}, E)$ ist **bipartit**, wenn jede Kante in E genau einen Knoten aus $\underbrace{\text{Knoten}}_{\text{links}}$ mit genau einem Knoten aus $\underbrace{\text{Knoten}}_{\text{rechts}}$ verbindet.



$\cup$ = disjunkte Vereinigung

Problemstellung



Def. $P \subseteq E$ ist eine **Paarung** (engl. *matching*), wenn je zwei Kanten in P keinen gleichen Endpunkt haben.

Problem. GRÖSSTE PAARUNG

Finde eine möglichst große Paarung P in G .

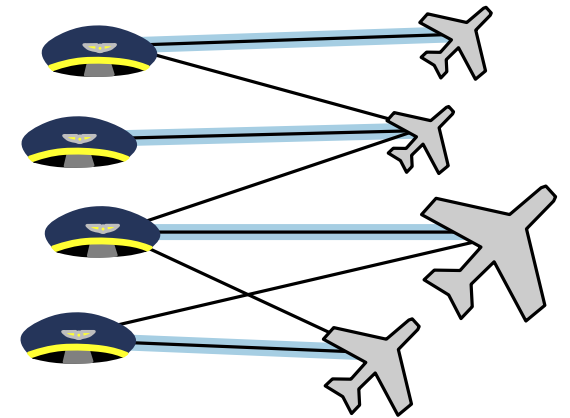
Bemerkung. Man könnte auch jeder Kante ein Gewicht zuweisen und dann die Summe der Gewichte gewählter Kanten maximieren.

Einschränkung. Wir nehmen im Folgenden an, dass G **bipartit** ist.

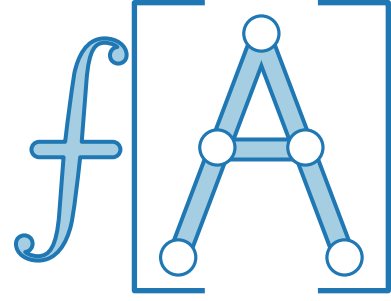
Def. Ein Graph $G = (\underbrace{\dots}_{\text{links}} \cup \underbrace{\dots}_{\text{rechts}}, E)$ ist **bipartit**, wenn jede Kante in E genau einen Knoten aus $\underbrace{\dots}_{\text{links}}$ mit genau einem Knoten aus $\underbrace{\dots}_{\text{rechts}}$ verbindet.

Problem. GRÖSSTE BIPARTITE PAARUNG

Finde eine möglichst große Paarung P in einem bipartiten Graphen G .



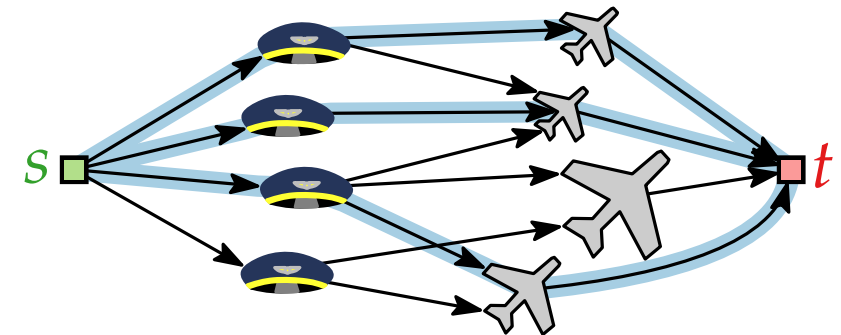
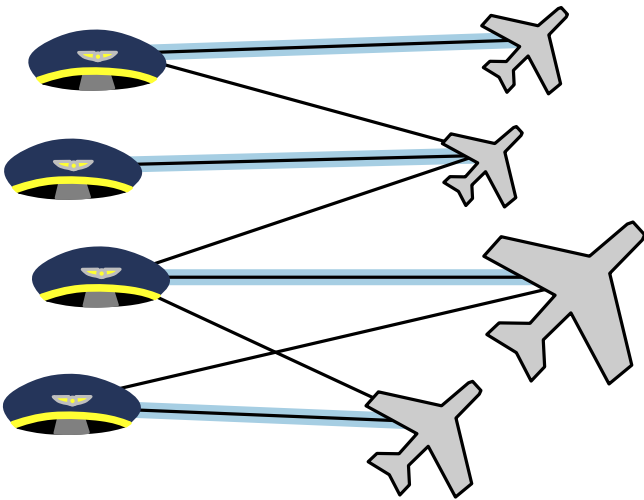
$\cup$ = disjunkte Vereinigung



Fortgeschrittene Algorithmen

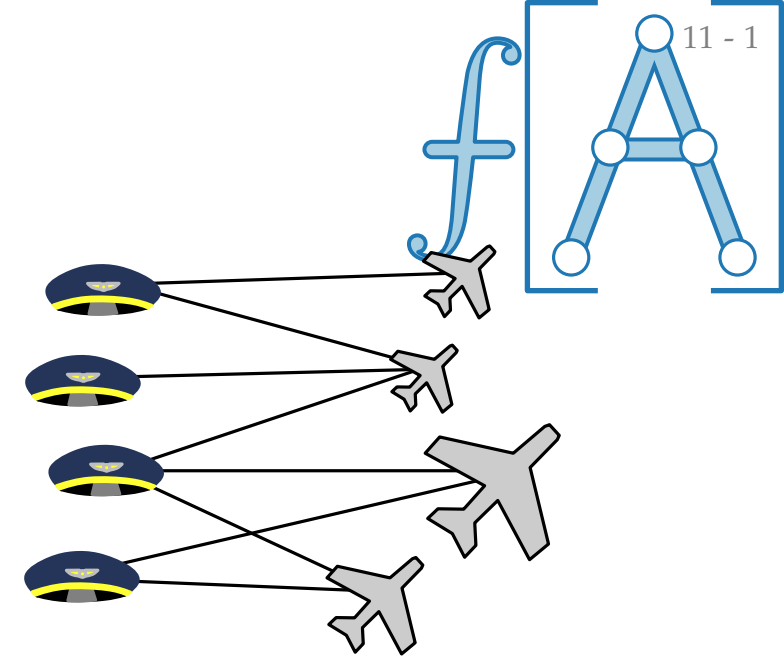
9. Vorlesung Graphalgorithmen III: Paarungen / Matchings

Teil III:
Lösung mittels Flüssen



Lösungsansatz mittels Flussberechnung

Sei $G = (V = \text{Pilot} \cup \text{Flugzeug}, E)$ ein bipartiter Graph.

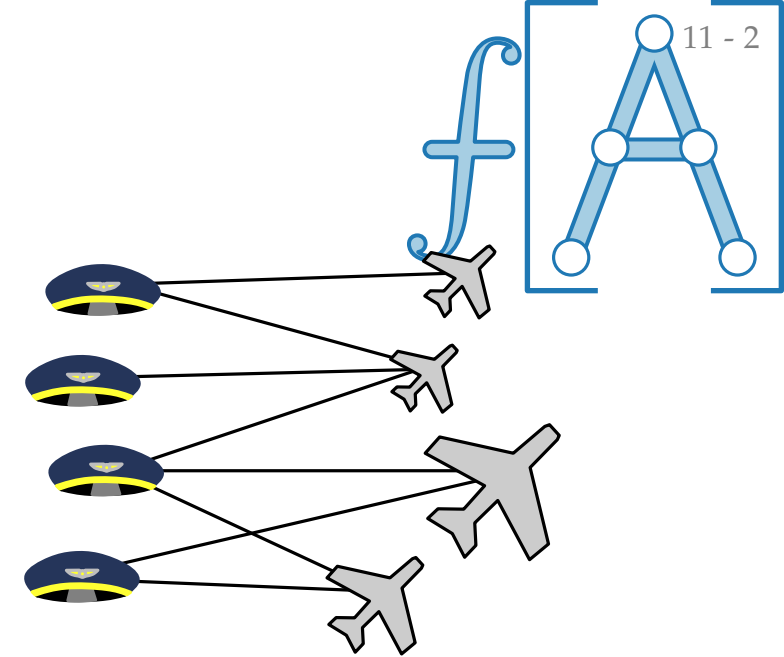


Lösungsansatz mittels Flussberechnung

Sei $G = (V = \text{Pilot} \cup \text{Flugzeug}, E)$ ein bipartiter Graph.

Das **zugehörige Flussnetzwerk**

$(G' = (V', E'); s; t; c)$ ist wie folgt definiert.



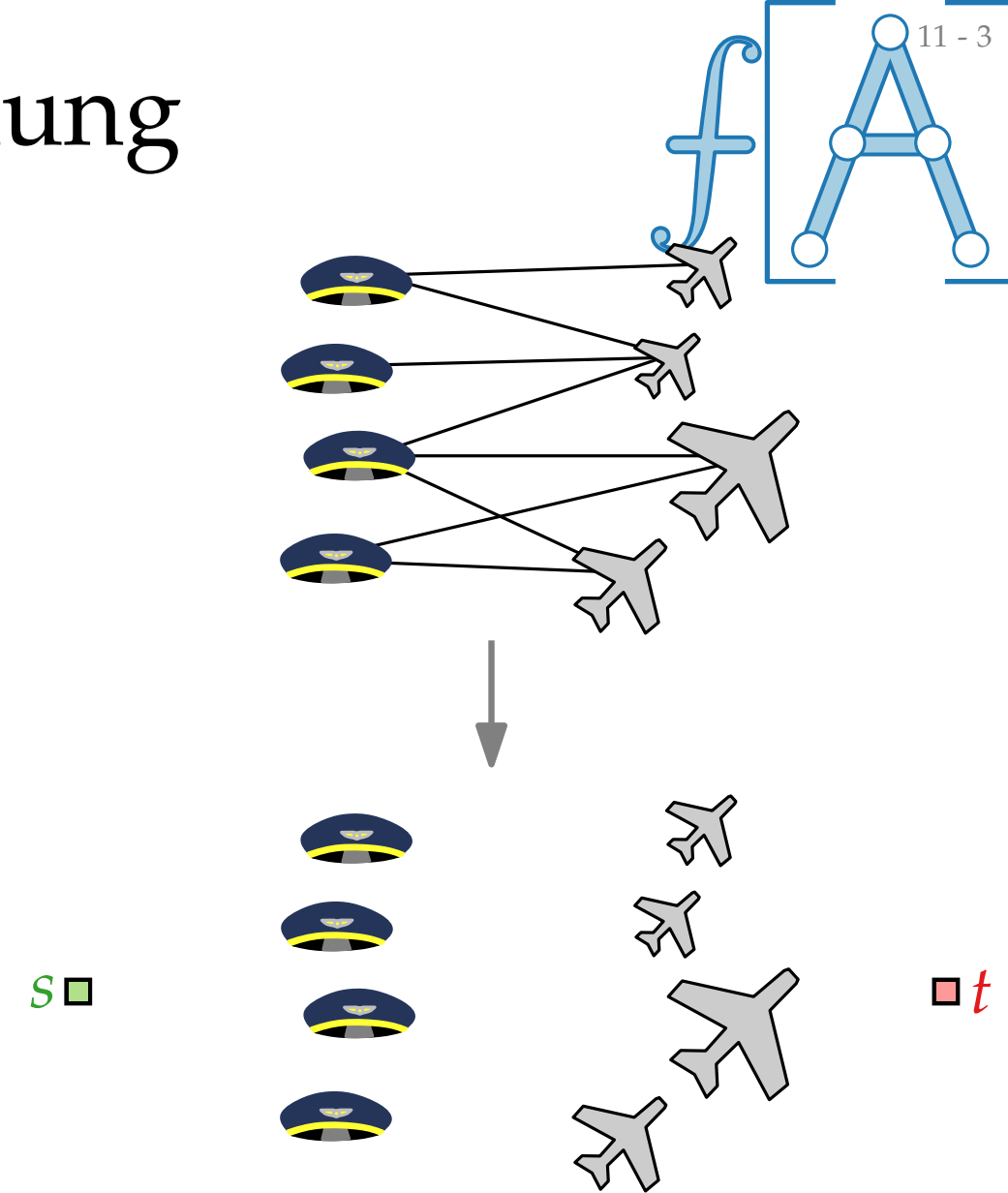
Lösungsansatz mittels Flussberechnung

Sei $G = (V = \text{Pilot} \cup \text{Flugzeug}, E)$ ein bipartiter Graph.

Das **zugehörige Flussnetzwerk**

$(G' = (V', E'); s; t; c)$ ist wie folgt definiert.

■ $V' = V \cup \{s, t\}$



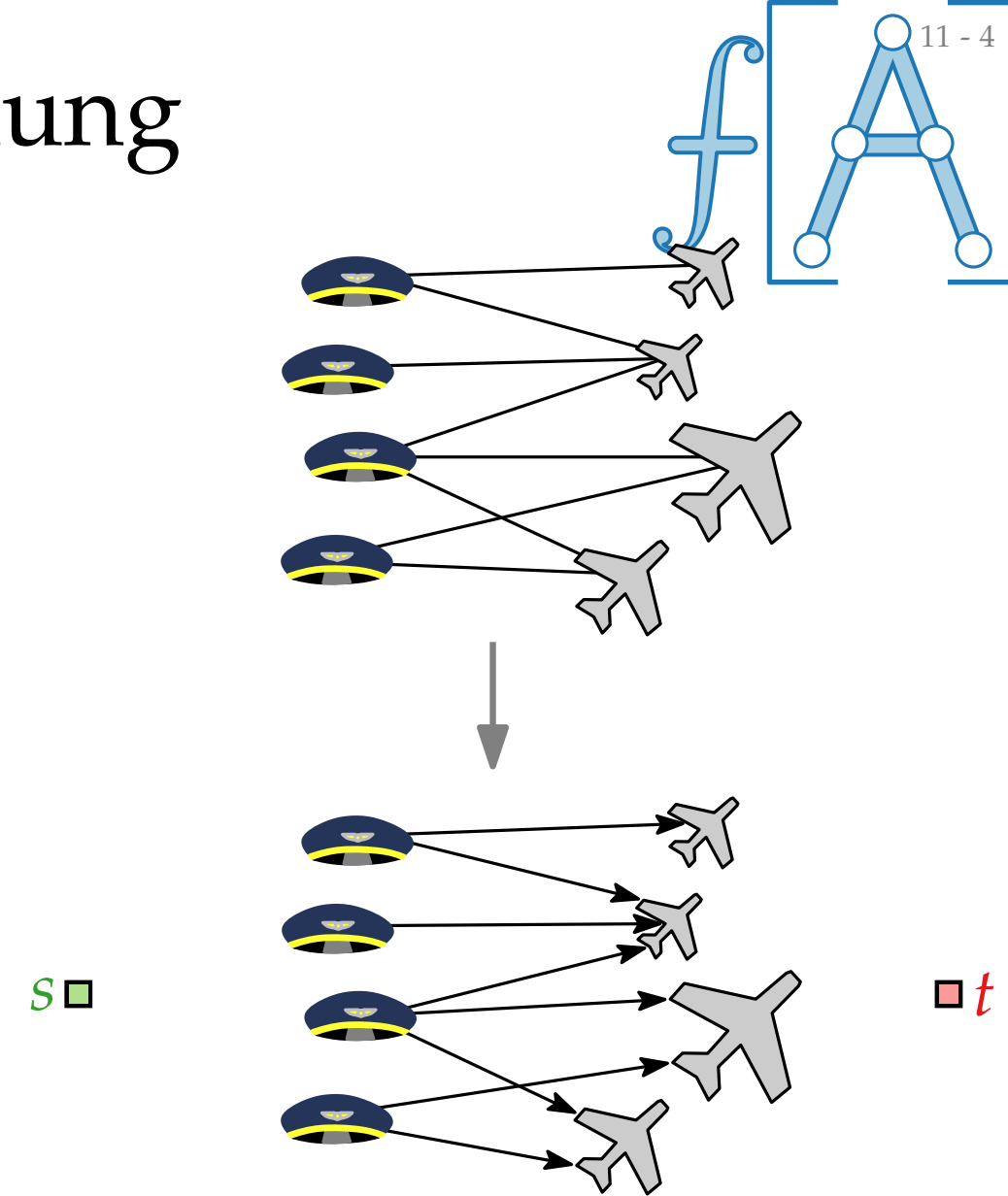
Lösungsansatz mittels Flussberechnung

Sei $G = (V = \text{🚂} \cup \text{✈️}, E)$ ein bipartiter Graph.

Das **zugehörige Flussnetzwerk**

$(G' = (V', E'); s; t; c)$ ist wie folgt definiert.

- $V' = V \cup \{s, t\}$
- $(\text{🚂}, \text{✈️}) \in E'$ für jede Kante $\{\text{🚂}, \text{✈️}\} \in E$



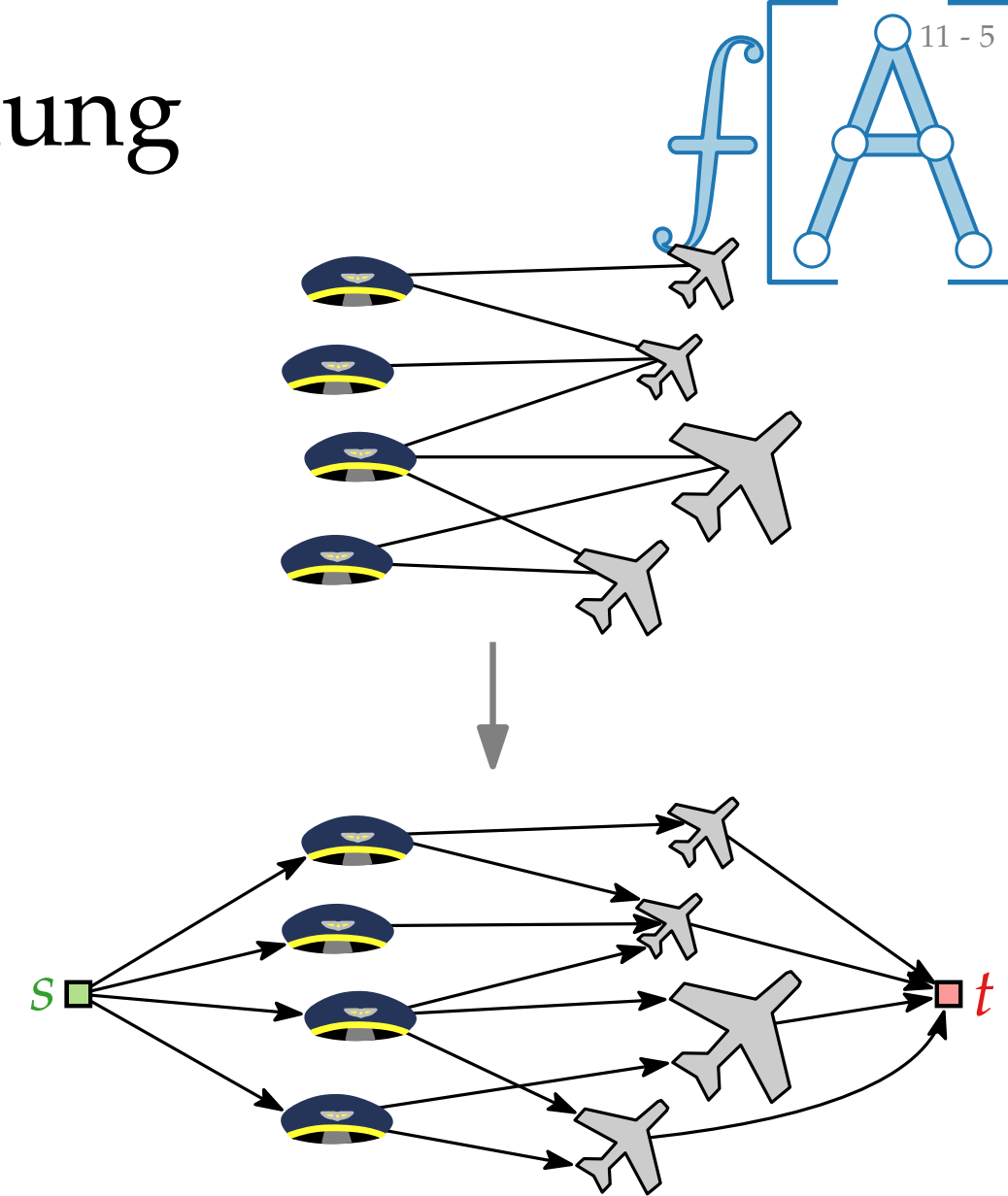
Lösungsansatz mittels Flussberechnung

Sei $G = (V = \text{🚂} \cup \text{✈️}, E)$ ein bipartiter Graph.

Das **zugehörige Flussnetzwerk**

$(G' = (V', E'); s; t; c)$ ist wie folgt definiert.

- $V' = V \cup \{s, t\}$
- $(\text{🚂}, \text{✈️}) \in E'$ für jede Kante $\{\text{🚂}, \text{✈️}\} \in E$
- $(s, \text{🚂}) \in E'$ für jeden Knoten 🚂 und $(\text{✈️}, t) \in E'$ für jeden Knoten ✈️



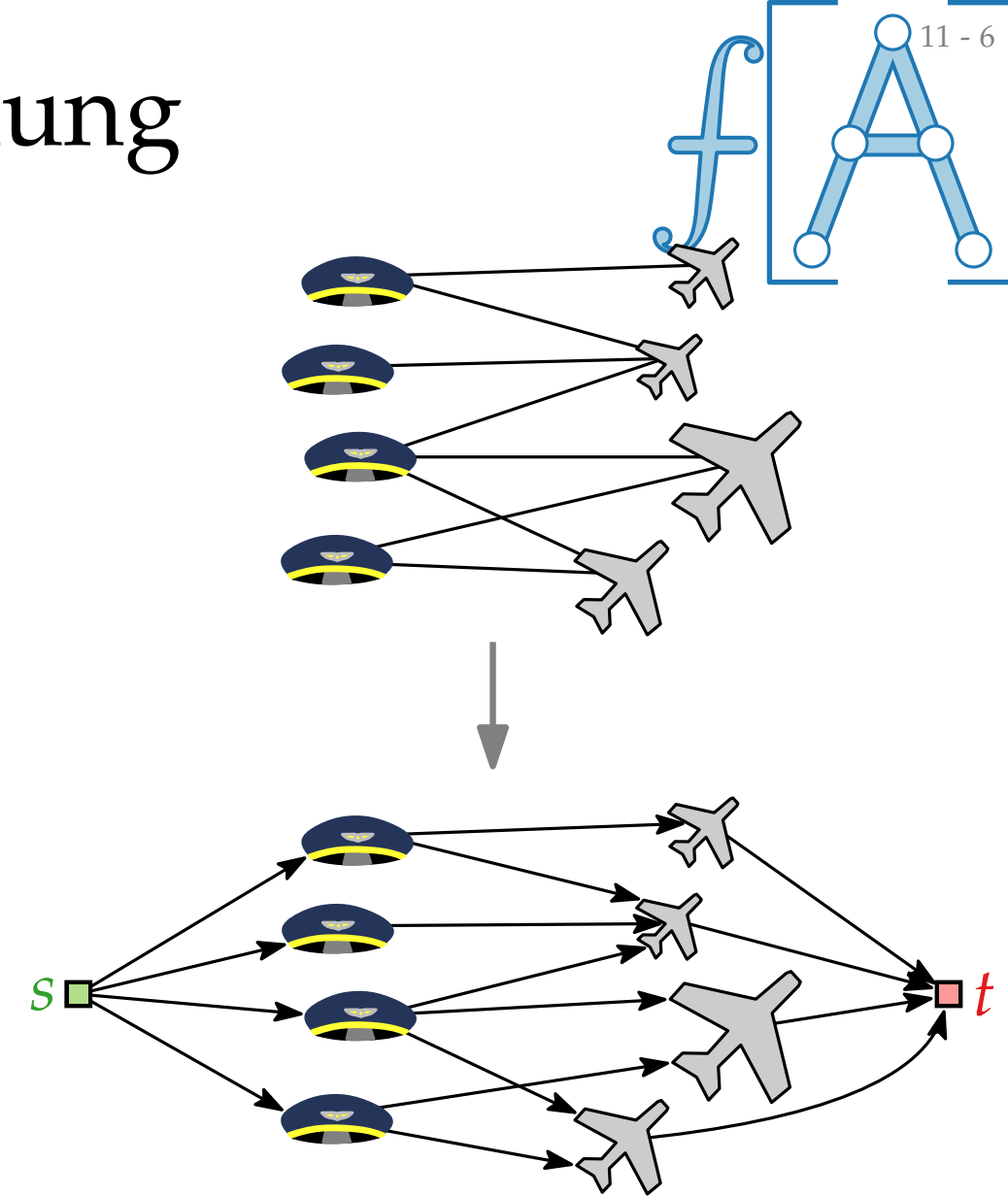
Lösungsansatz mittels Flussberechnung

Sei $G = (V = \text{🚂} \cup \text{✈️}, E)$ ein bipartiter Graph.

Das **zugehörige Flussnetzwerk**

$(G' = (V', E'); s; t; c)$ ist wie folgt definiert.

- $V' = V \cup \{s, t\}$
- $(\text{🚂}, \text{✈️}) \in E'$ für jede Kante $\{\text{🚂}, \text{✈️}\} \in E$
- $(s, \text{🚂}) \in E'$ für jeden Knoten 🚂 und $(\text{✈️}, t) \in E'$ für jeden Knoten ✈️
- $c(e) = 1$ für alle Kanten $e \in E'$.



Lösungsansatz mittels Flussberechnung

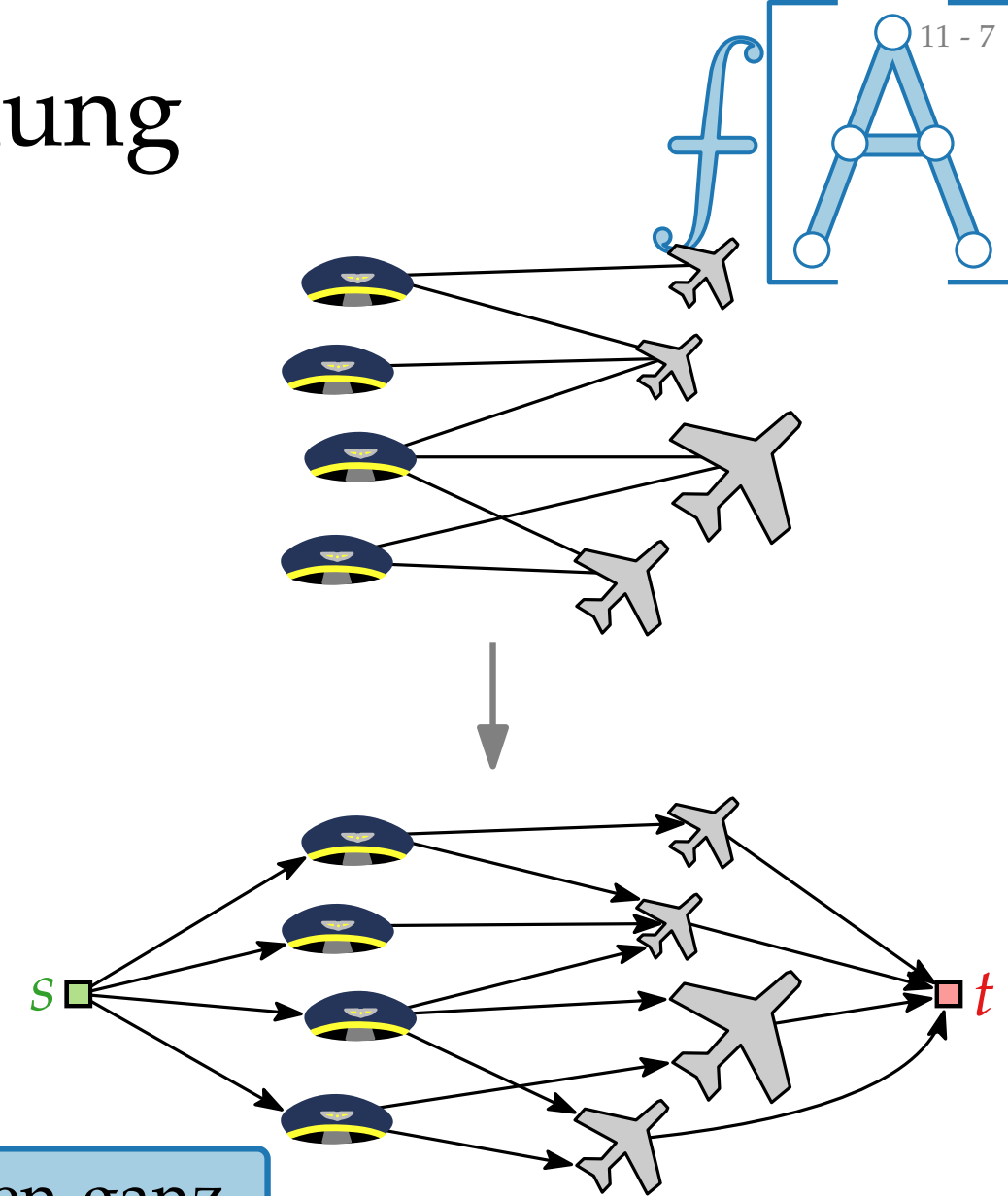
Sei $G = (V = \text{🛩️} \cup \text{✈️}, E)$ ein bipartiter Graph.

Das **zugehörige Flussnetzwerk**

$(G' = (V', E'); s; t; c)$ ist wie folgt definiert.

- $V' = V \cup \{s, t\}$
- $(\text{🛩️}, \text{✈️}) \in E'$ für jede Kante $\{\text{🛩️}, \text{✈️}\} \in E$
- $(s, \text{🛩️}) \in E'$ für jeden Knoten 🛩️ und $(\text{✈️}, t) \in E'$ für jeden Knoten ✈️
- $c(e) = 1$ für alle Kanten $e \in E'$.

Lemma. 1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.



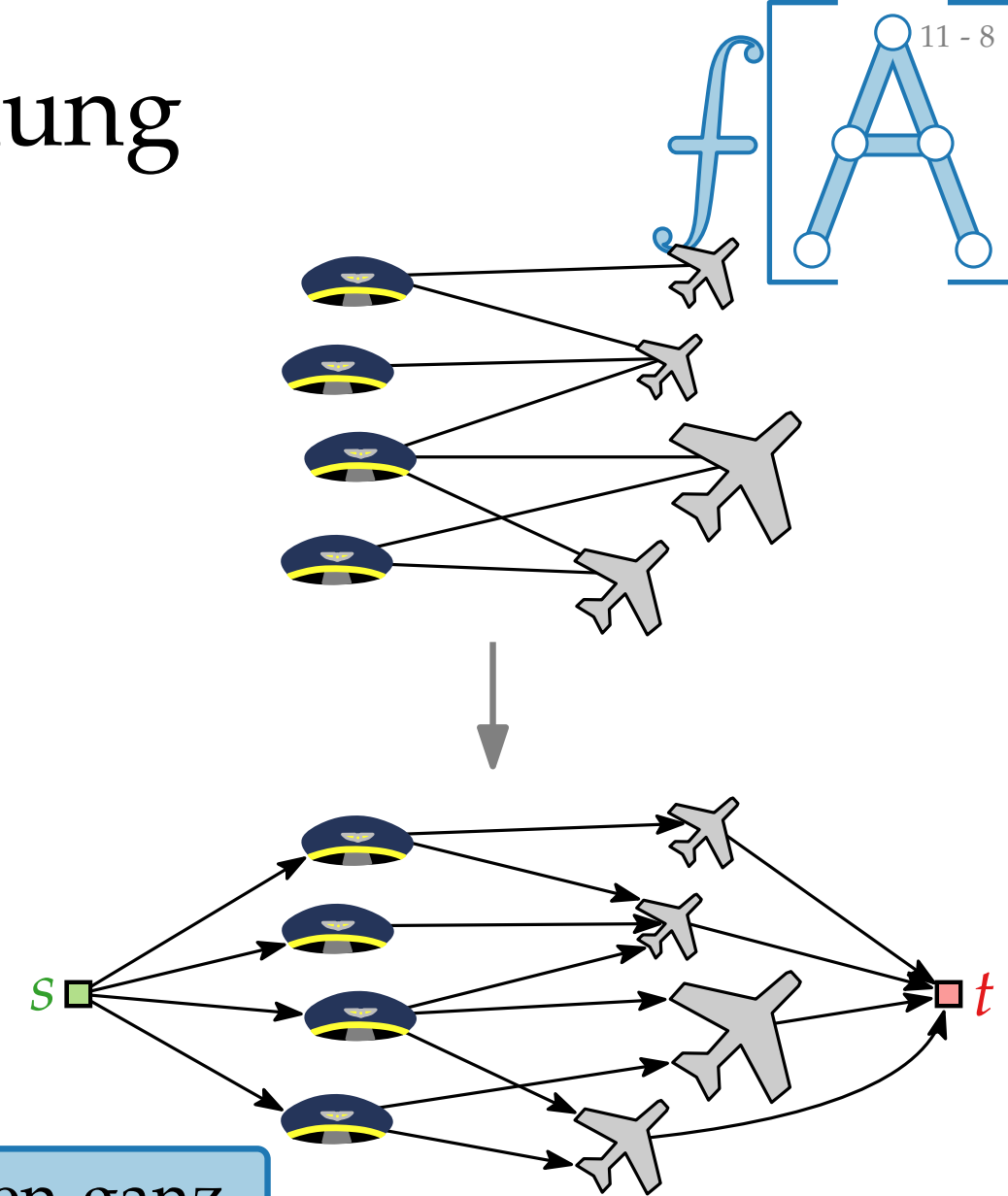
Lösungsansatz mittels Flussberechnung

Sei $G = (V = \text{🚗} \cup \text{✈️}, E)$ ein bipartiter Graph.

Das **zugehörige Flussnetzwerk**

$(G' = (V', E'); s; t; c)$ ist wie folgt definiert.

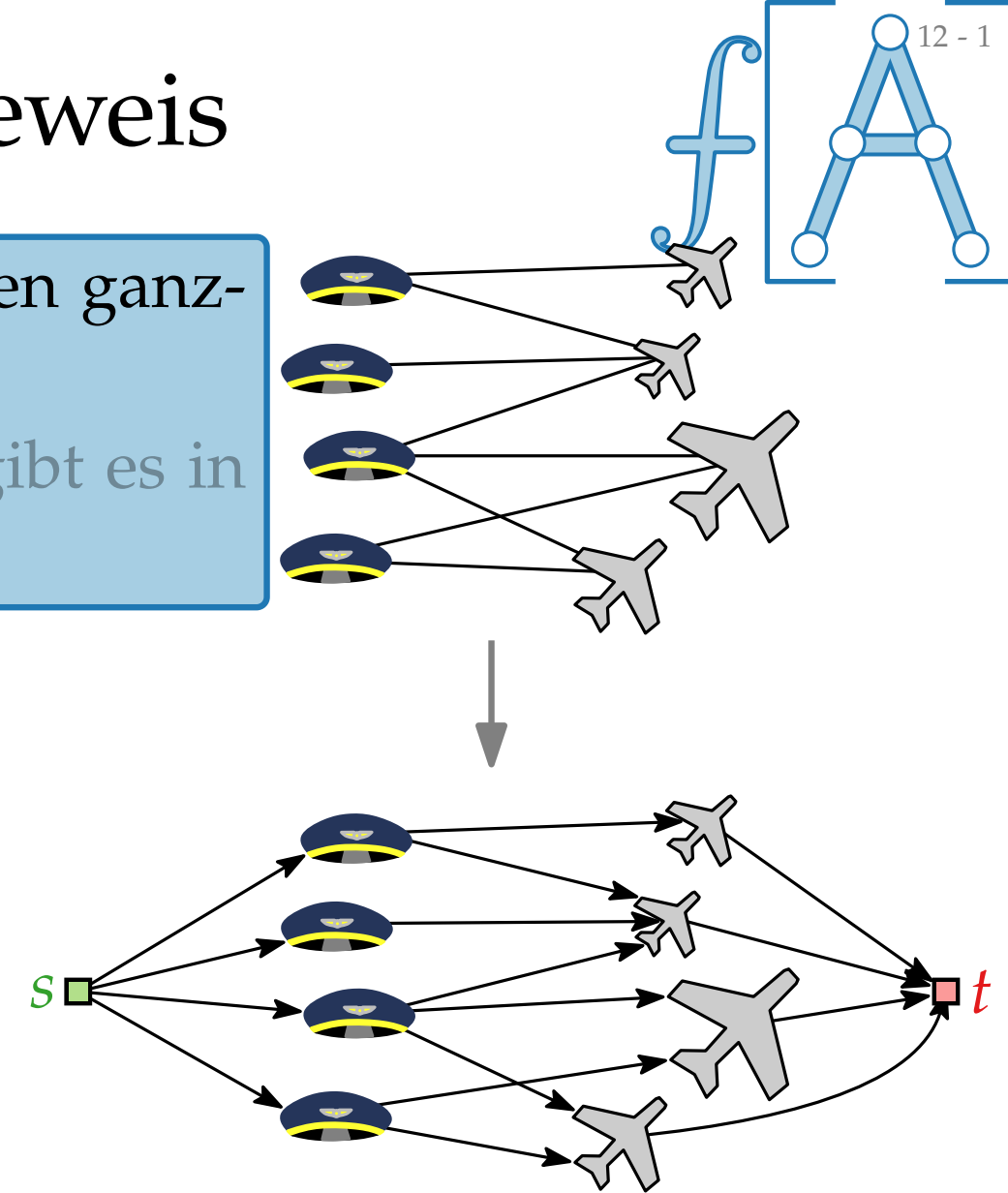
- $V' = V \cup \{s, t\}$
- $(\text{🚗}, \text{✈️}) \in E'$ für jede Kante $\{\text{🚗}, \text{✈️}\} \in E$
- $(s, \text{🚗}) \in E'$ für jeden Knoten 🚗 und $(\text{✈️}, t) \in E'$ für jeden Knoten ✈️
- $c(e) = 1$ für alle Kanten $e \in E'$.



- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

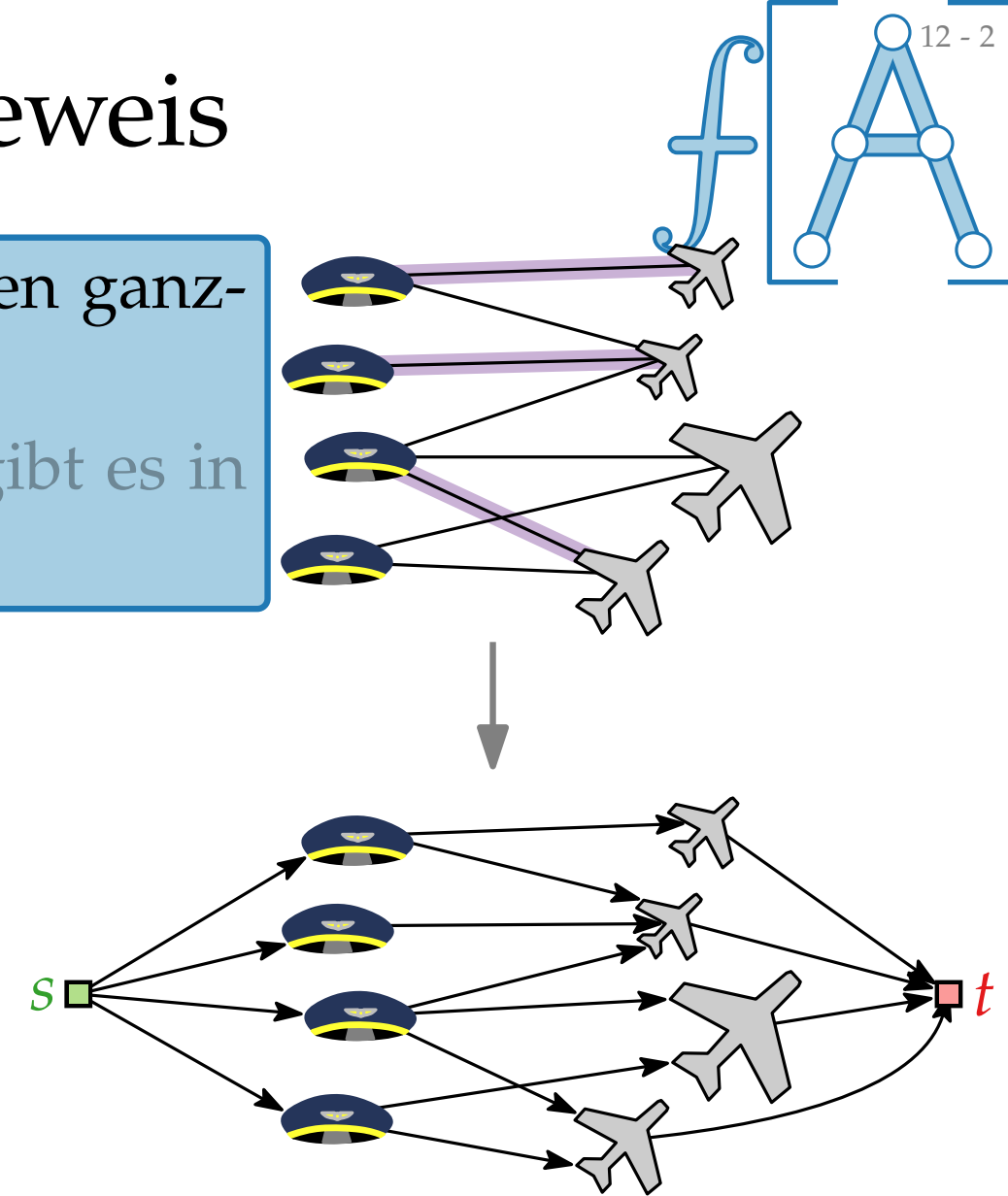
Äquivalenz des Flussnetzwerks – Beweis

- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.



Äquivalenz des Flussnetzwerks – Beweis

- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

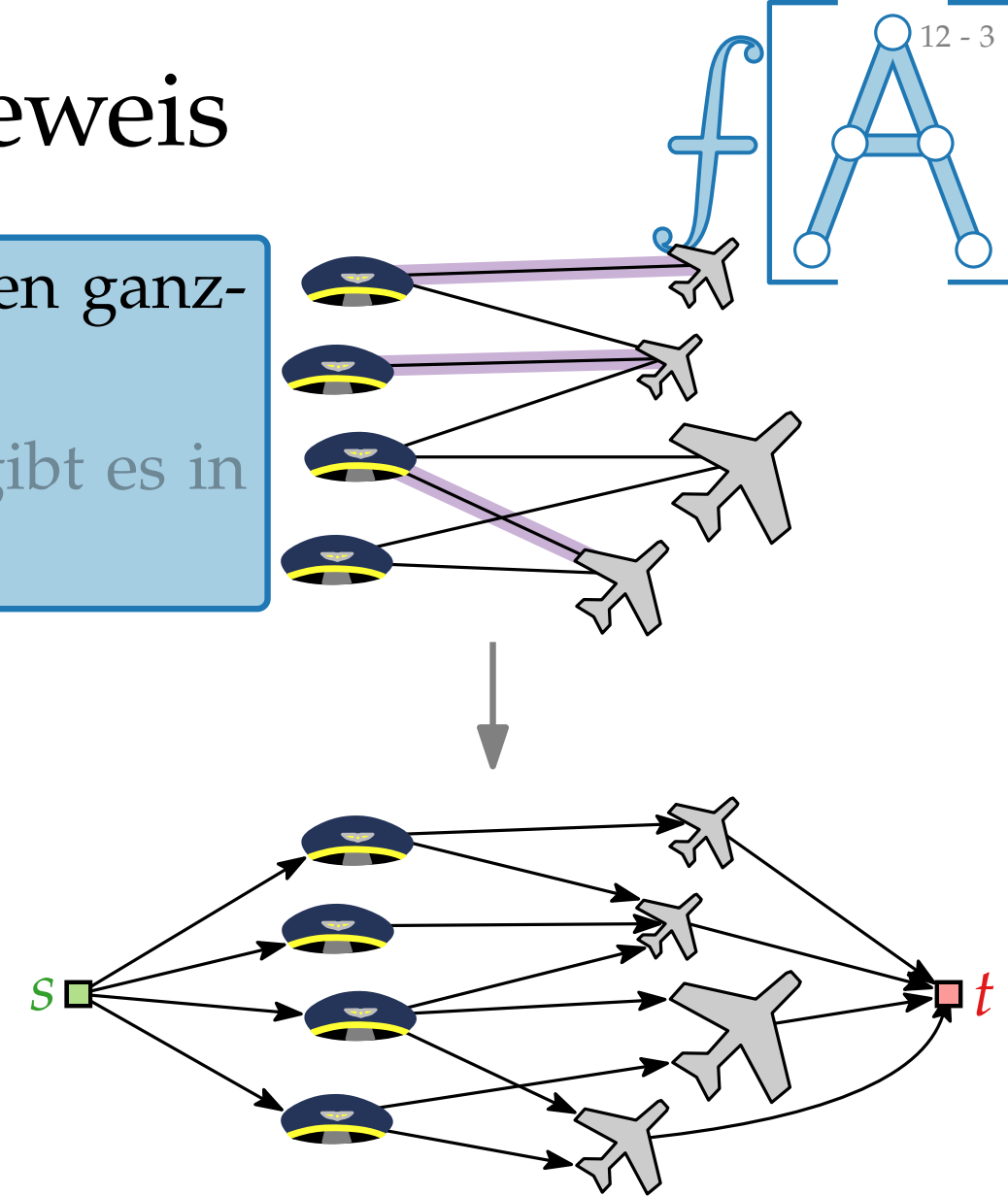


Äquivalenz des Flussnetzwerks – Beweis

- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis. Definiere f wie folgt:

- Für $\{ \text{🚗}, \text{✈️} \} \in P$ setze



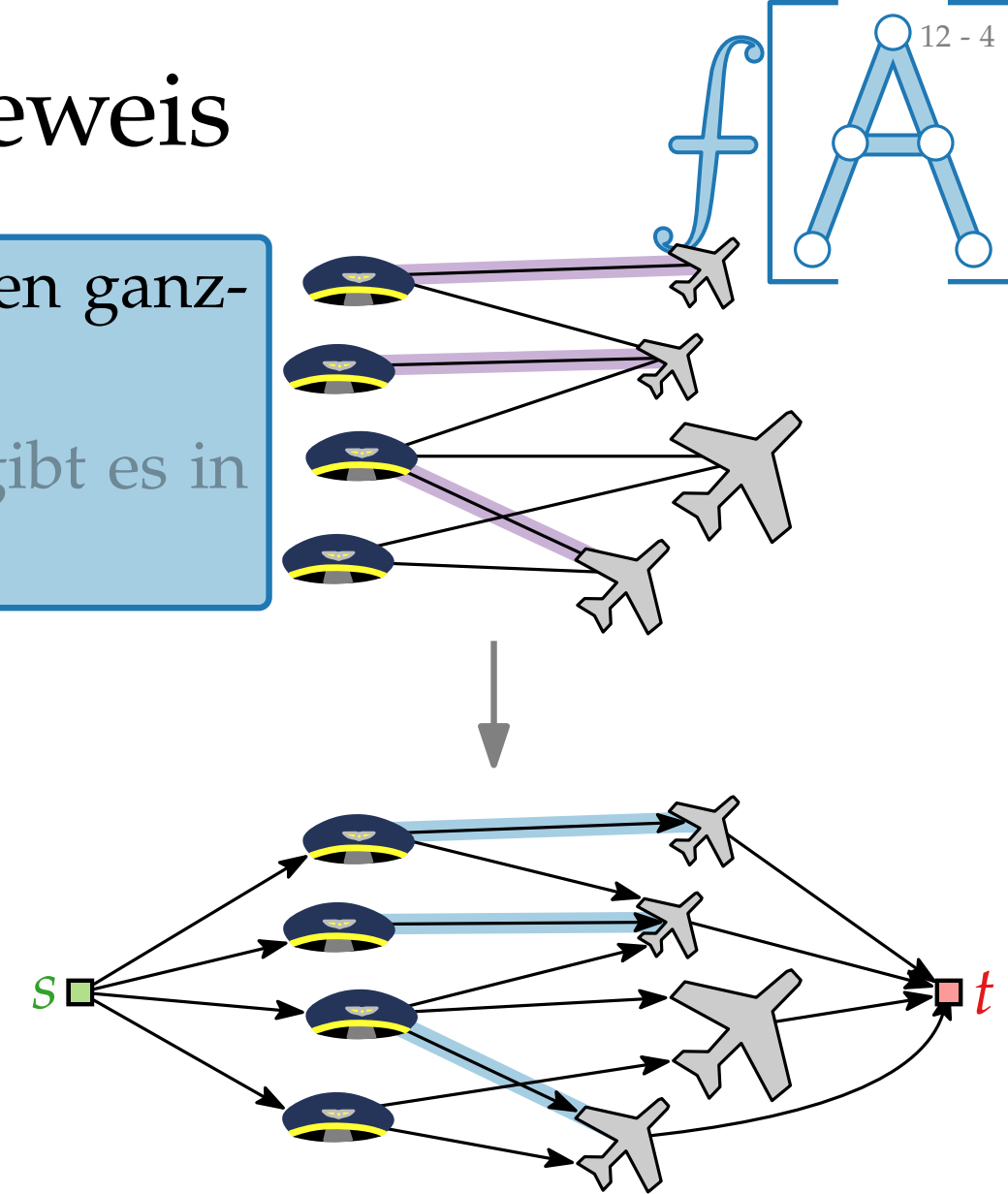
Äquivalenz des Flussnetzwerks – Beweis

Lemma. 1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.

2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis. Definiere f wie folgt:

■ Für $\{ \text{🚗}, \text{✈️} \} \in P$ setze
 $f(\text{🚗}, \text{✈️}) = 1$



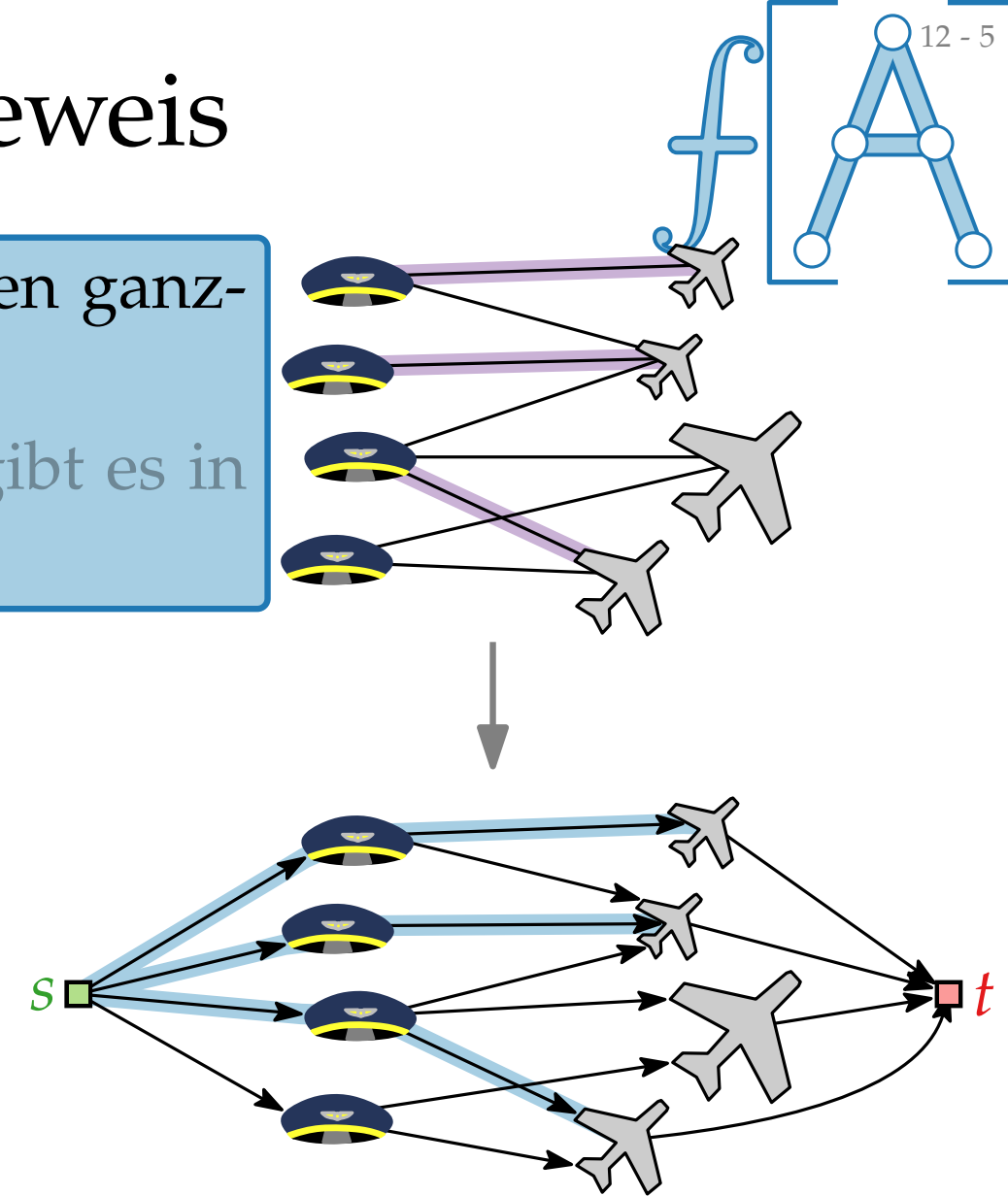
Äquivalenz des Flussnetzwerks – Beweis

Lemma. 1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.

2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis. Definiere f wie folgt:

- Für $\{ \text{🚗}, \text{✈️} \} \in P$ setze
 $f(s, \text{🚗}) = f(\text{🚗}, \text{✈️}) = 1$



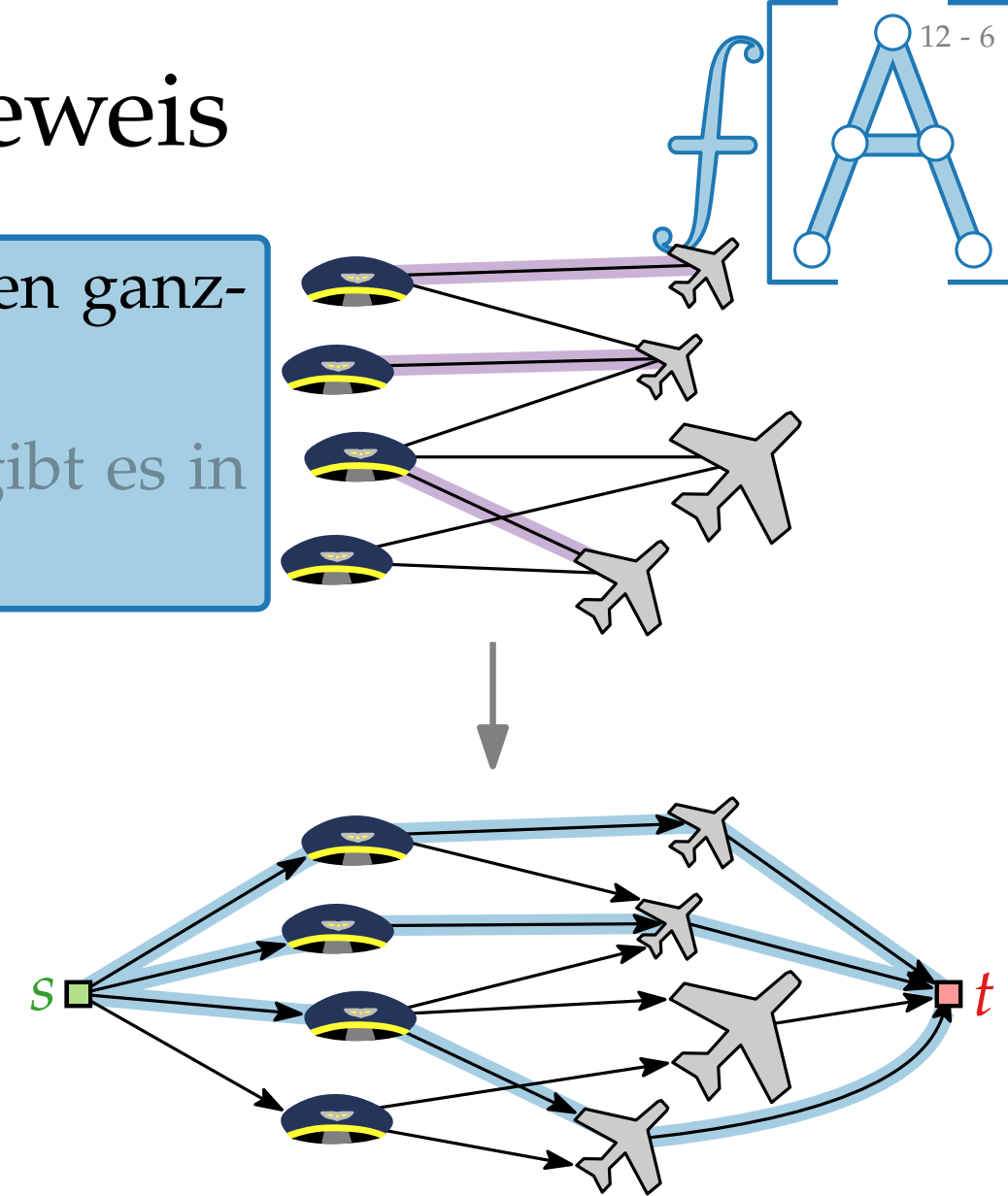
Äquivalenz des Flussnetzwerks – Beweis

Lemma. 1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.

2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis. Definiere f wie folgt:

- Für $\{ \text{🚗}, \text{✈️} \} \in P$ setze
 $f(s, \text{🚗}) = f(\text{🚗}, \text{✈️}) = f(\text{✈️}, t) = 1$.



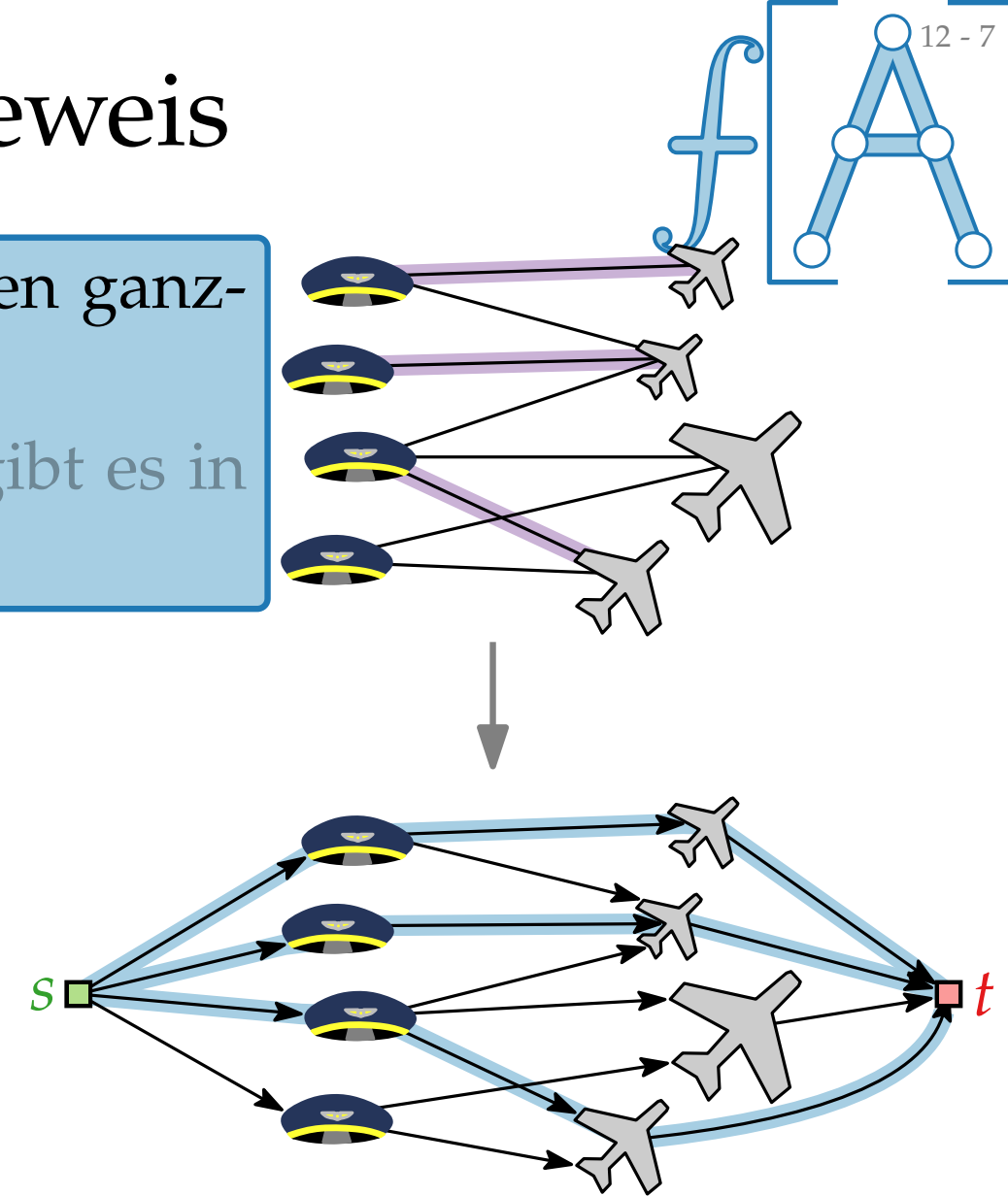
Äquivalenz des Flussnetzwerks – Beweis

Lemma. 1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.

2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis. Definiere f wie folgt:

- Für $\{ \text{👤}, \text{✈️} \} \in P$ setze
 $f(s, \text{👤}) = f(\text{👤}, \text{✈️}) = f(\text{✈️}, t) = 1$.
- Für alle anderen Kanten $e \in E'$ setze $f(e) = 0$.



Äquivalenz des Flussnetzwerks – Beweis

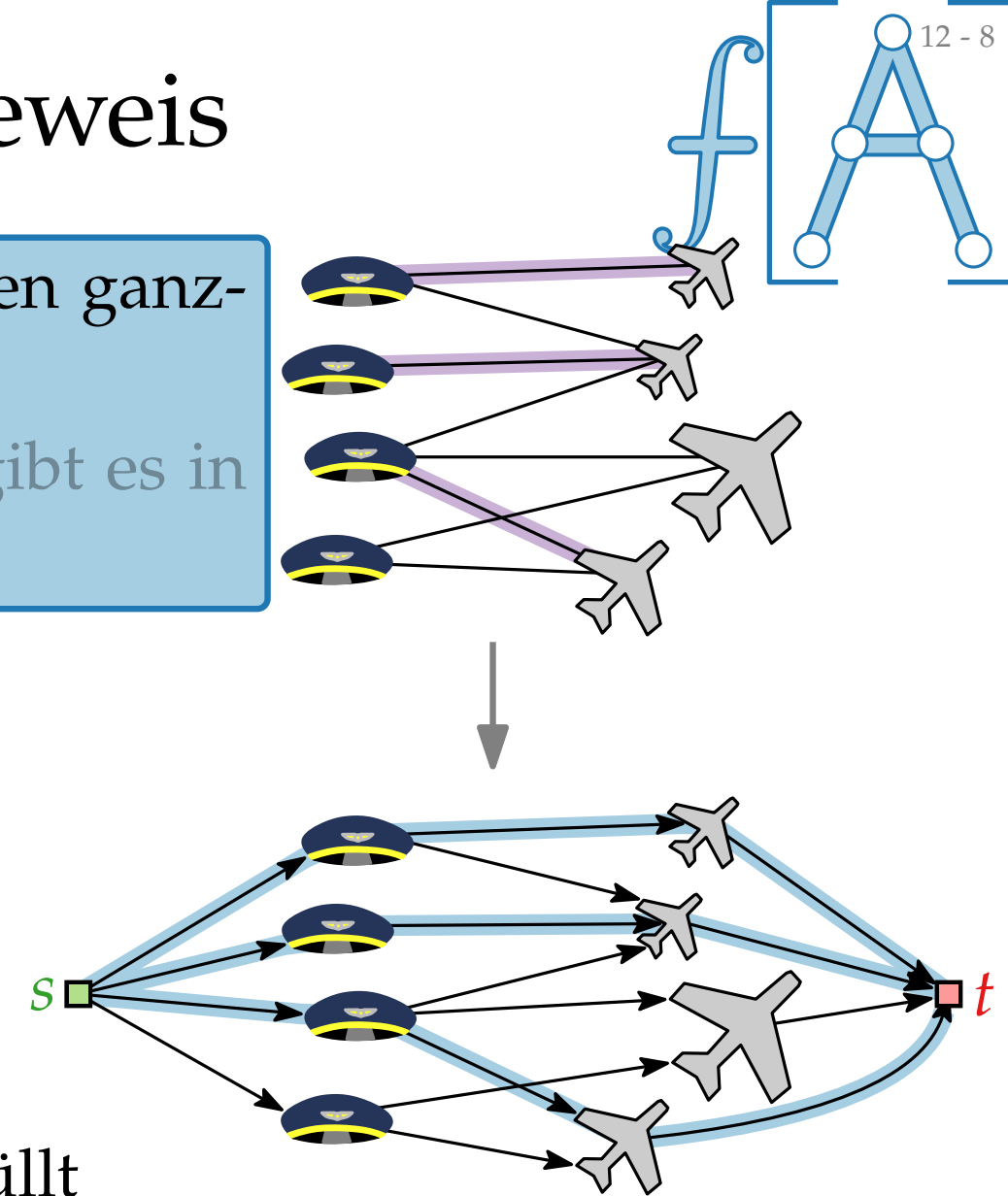
- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis. Definiere f wie folgt:

- Für $\{ \text{🚗}, \text{✈️} \} \in P$ setze
 $f(s, \text{🚗}) = f(\text{🚗}, \text{✈️}) = f(\text{✈️}, t) = 1$.
- Für alle anderen Kanten $e \in E'$ setze $f(e) = 0$.

Die Abbildung f tut das richtige, denn:

- Kapazitäts- und Flusserhaltungsbedingung sind erfüllt
 $\Rightarrow f$ ist ein Fluss
- f ist ganzzahlig
- $|f| = |P|$



Äquivalenz des Flussnetzwerks – Beweis

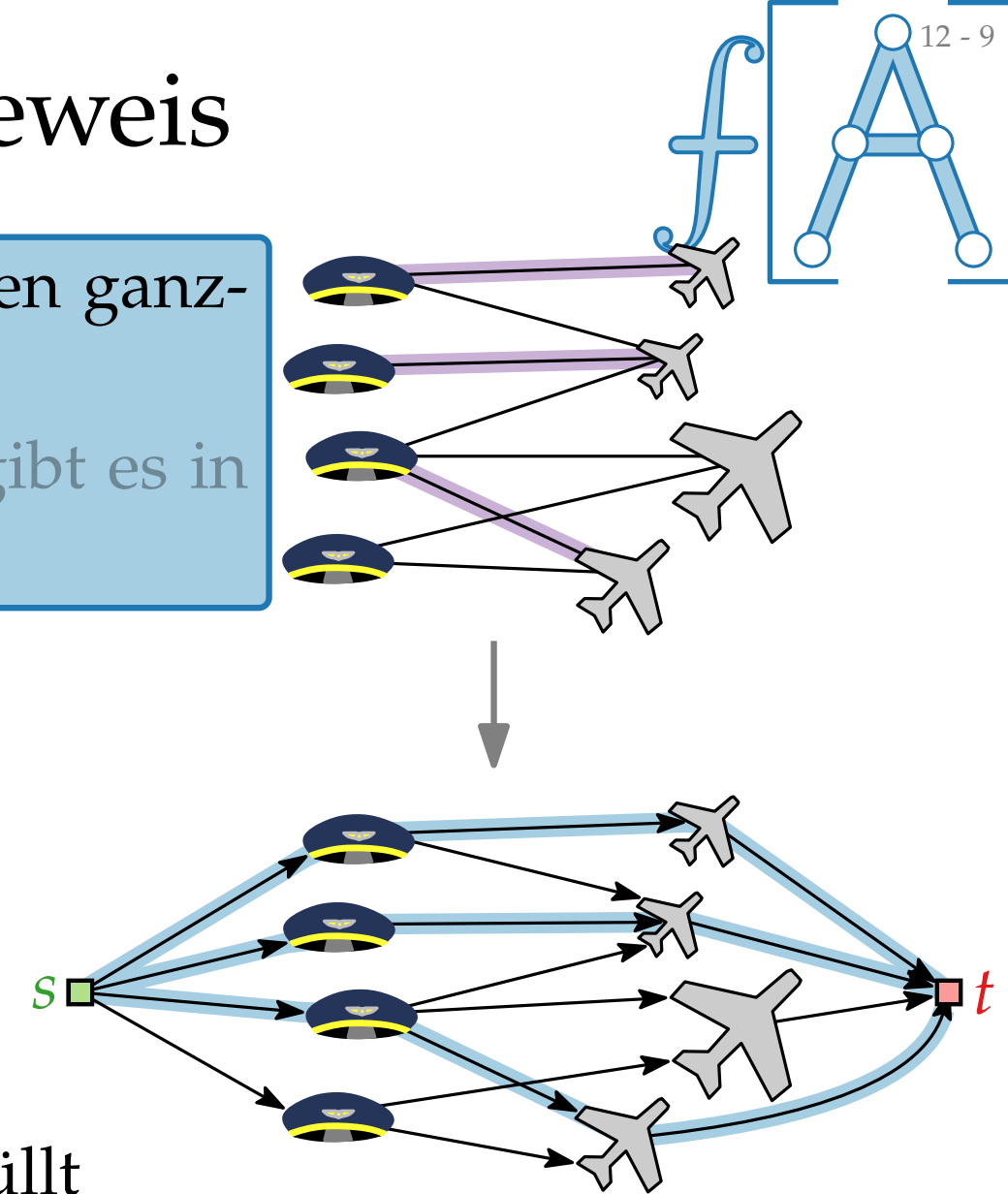
- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis. Definiere f wie folgt:

- Für $\{\text{🚗}, \text{✈️}\} \in P$ setze
 $f(s, \text{🚗}) = f(\text{🚗}, \text{✈️}) = f(\text{✈️}, t) = 1$.
- Für alle anderen Kanten $e \in E'$ setze $f(e) = 0$.

Die Abbildung f tut das richtige, denn:

- Kapazitäts- und Flusserhaltungsbedingung sind erfüllt
 $\Rightarrow f$ ist ein Fluss ✓
- f ist ganzzahlig
- $|f| = |P|$



Äquivalenz des Flussnetzwerks – Beweis

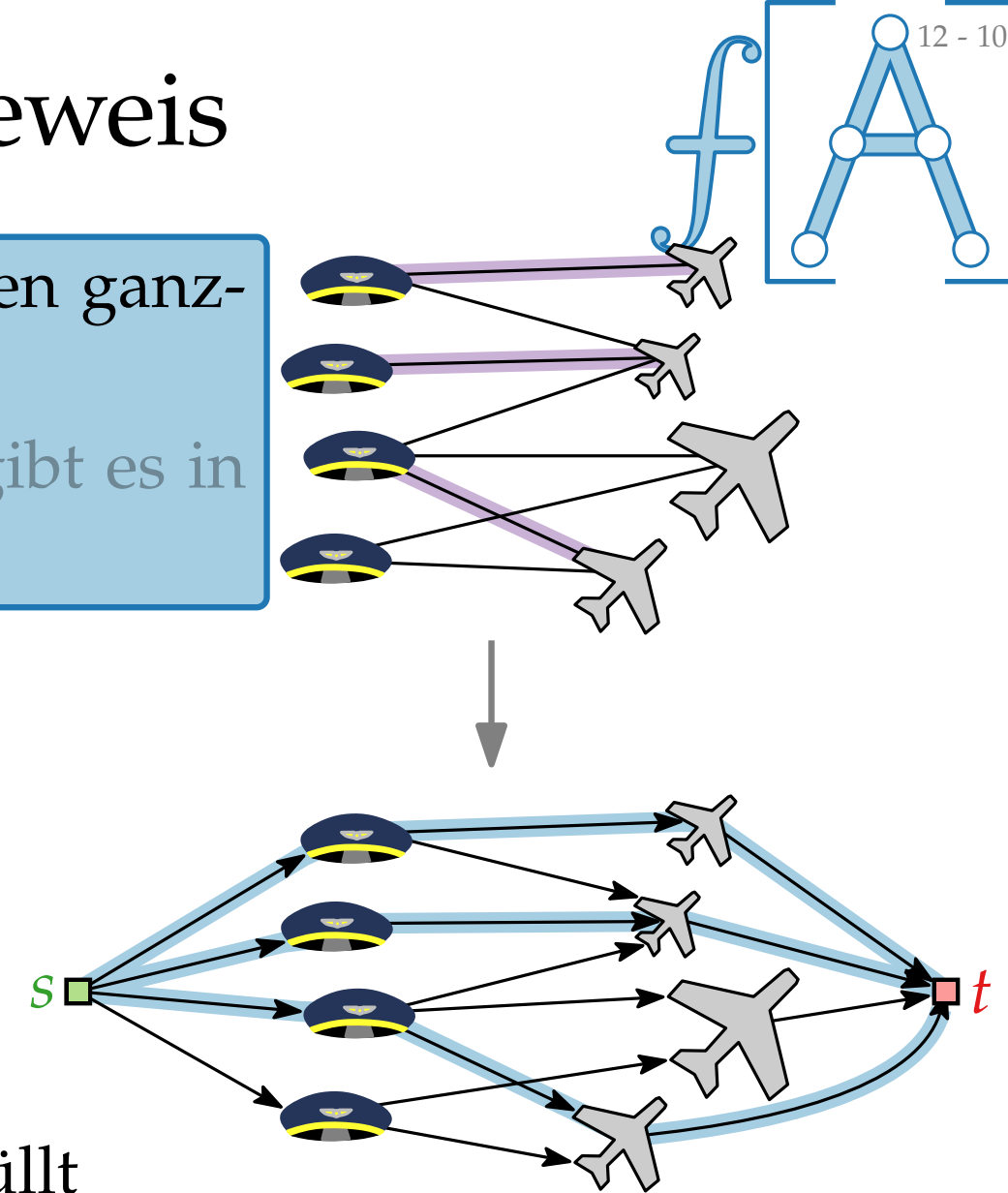
- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis. Definiere f wie folgt:

- Für $\{\text{🚗}, \text{✈️}\} \in P$ setze
 $f(s, \text{🚗}) = f(\text{🚗}, \text{✈️}) = f(\text{✈️}, t) = 1$.
- Für alle anderen Kanten $e \in E'$ setze $f(e) = 0$.

Die Abbildung f tut das richtige, denn:

- Kapazitäts- und Flusserhaltungsbedingung sind erfüllt
 $\Rightarrow f$ ist ein Fluss ✓
- f ist ganzzahlig ✓
- $|f| = |P|$



Äquivalenz des Flussnetzwerks – Beweis

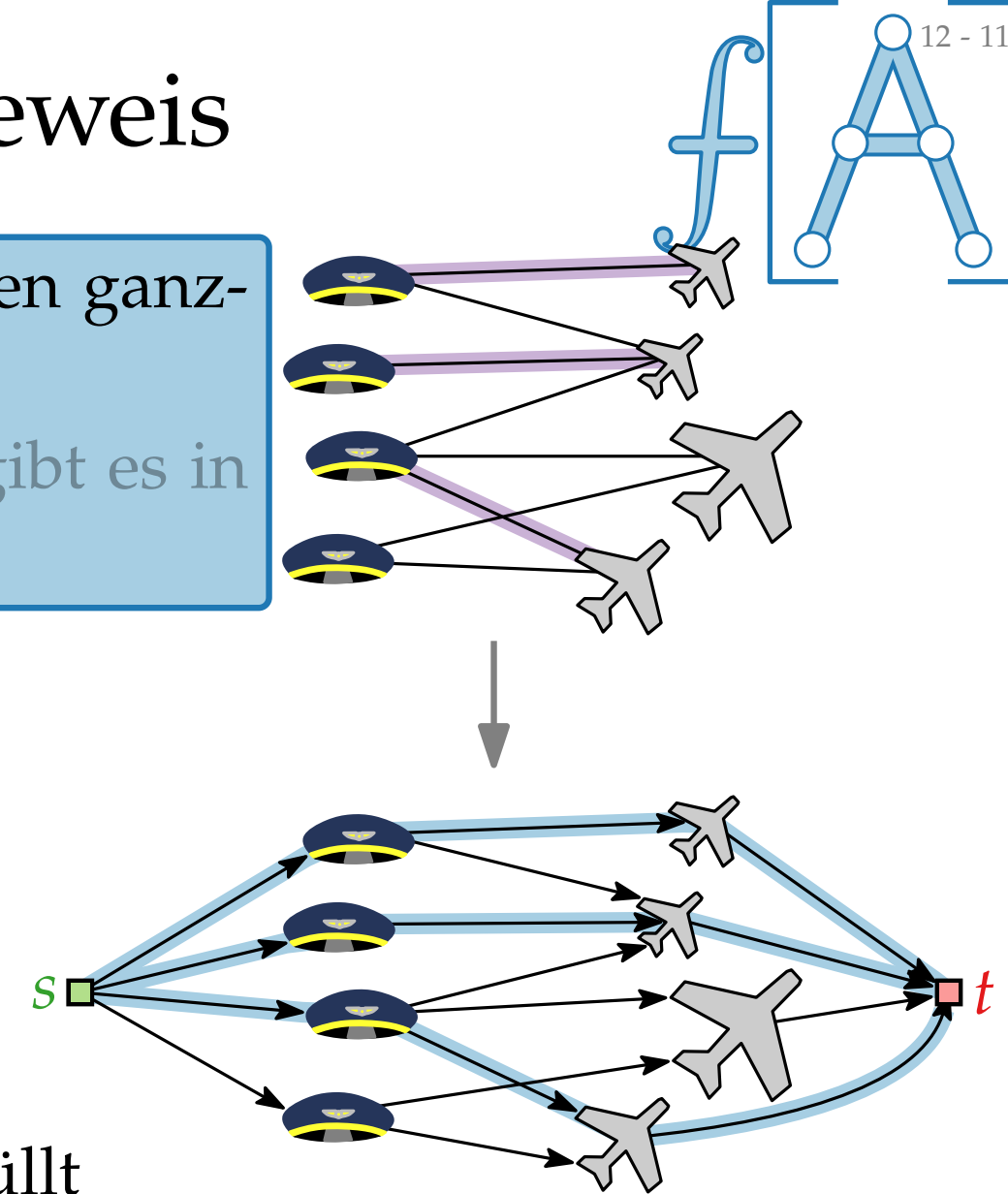
- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis. Definiere f wie folgt:

- Für $\{\text{🛩️}, \text{✈️}\} \in P$ setze
 $f(s, \text{🛩️}) = f(\text{🛩️}, \text{✈️}) = f(\text{✈️}, t) = 1$.
- Für alle anderen Kanten $e \in E'$ setze $f(e) = 0$.

Die Abbildung f tut das richtige, denn:

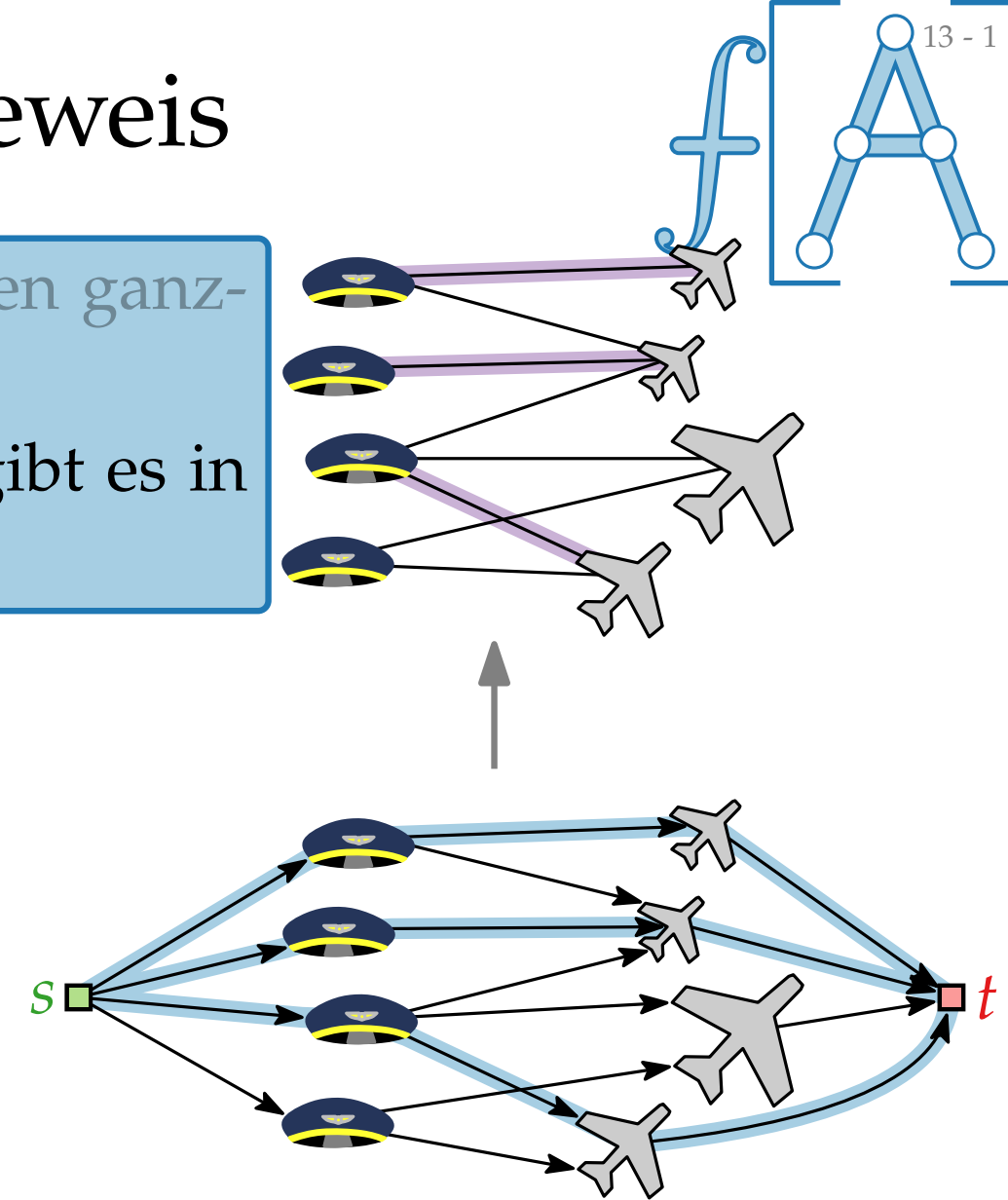
- Kapazitäts- und Flusserhaltungsbedingung sind erfüllt
 $\Rightarrow f$ ist ein Fluss ✓
- f ist ganzzahlig ✓
- $|f| = |P|$ ✓



Äquivalenz des Flussnetzwerks – Beweis

- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

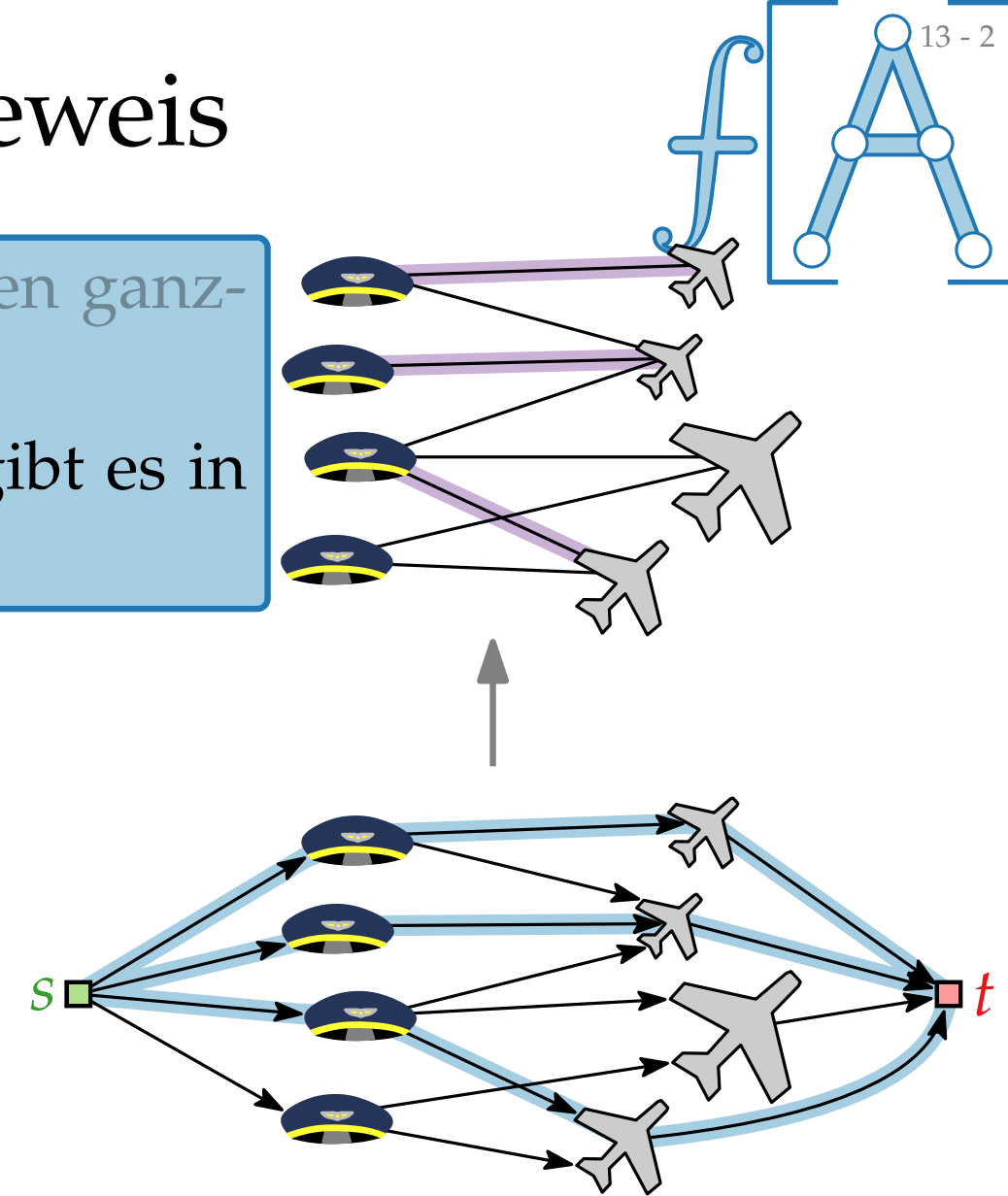


Äquivalenz des Flussnetzwerks – Beweis

- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

■ $c(e) = 1$ für alle Kanten $e \in E'$



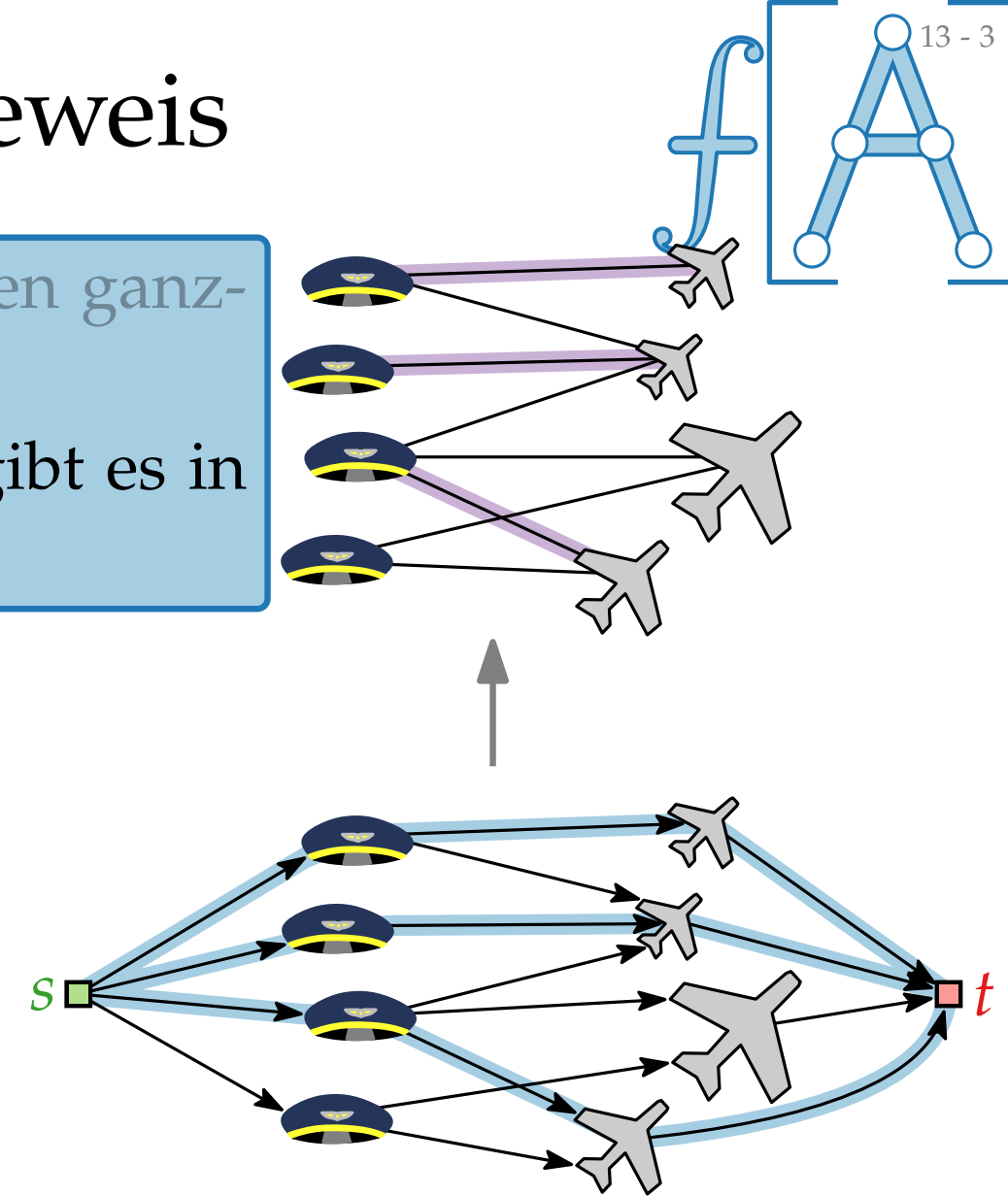
Äquivalenz des Flussnetzwerks – Beweis

Lemma.

1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

- $c(e) = 1$ für alle Kanten $e \in E'$
- Ganzzahligkeit von $f \Rightarrow f(e) = 0$ oder $f(e) = 1$



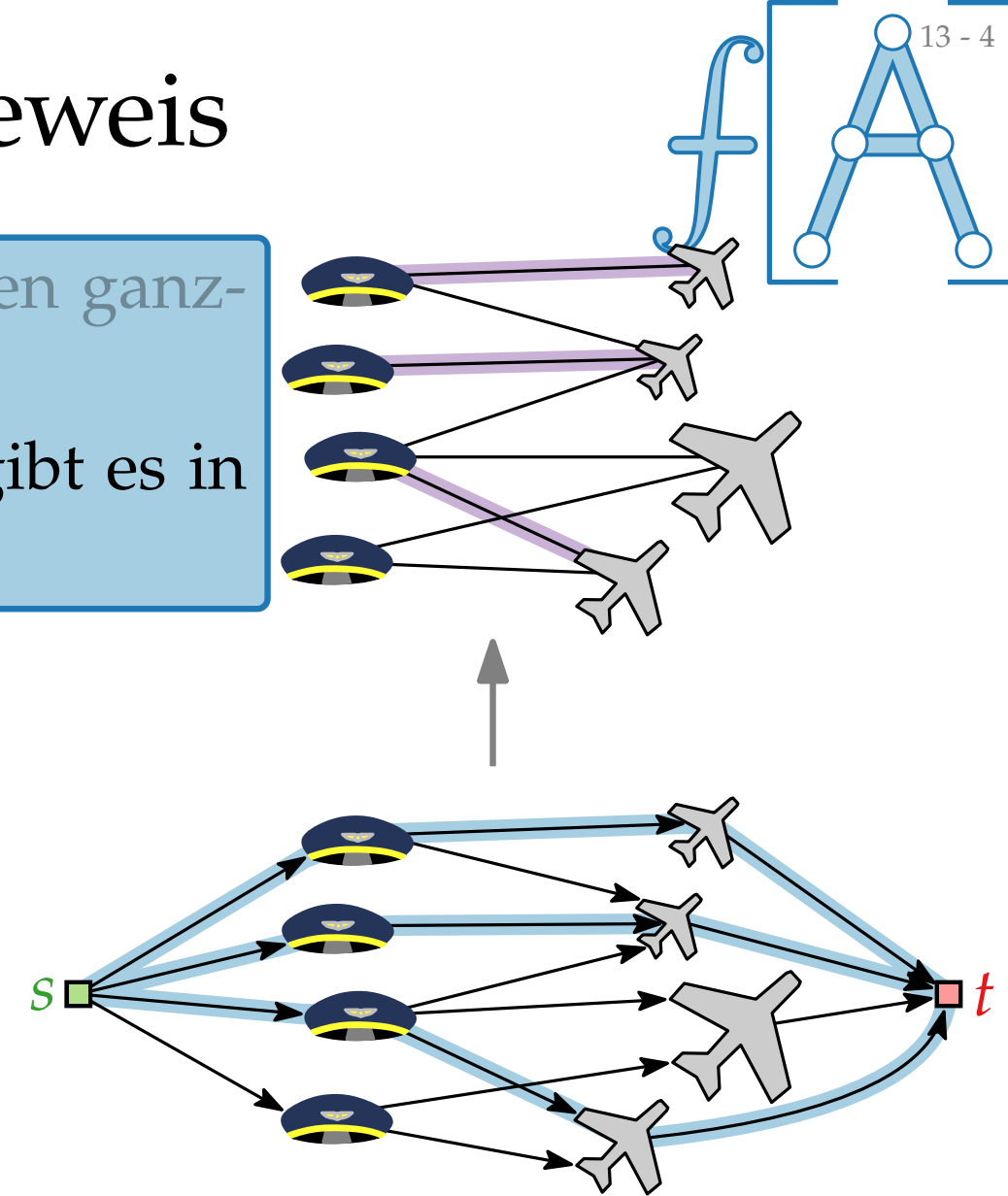
Äquivalenz des Flussnetzwerks – Beweis

Lemma.

1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

- $c(e) = 1$ für alle Kanten $e \in E'$
- Ganzzahligkeit von $f \Rightarrow f(e) = 0$ oder $f(e) = 1$
- Sei $(s, \text{🛩️})$ eine Kante mit $f(s, \text{🛩️}) = 1$.



Äquivalenz des Flussnetzwerks – Beweis

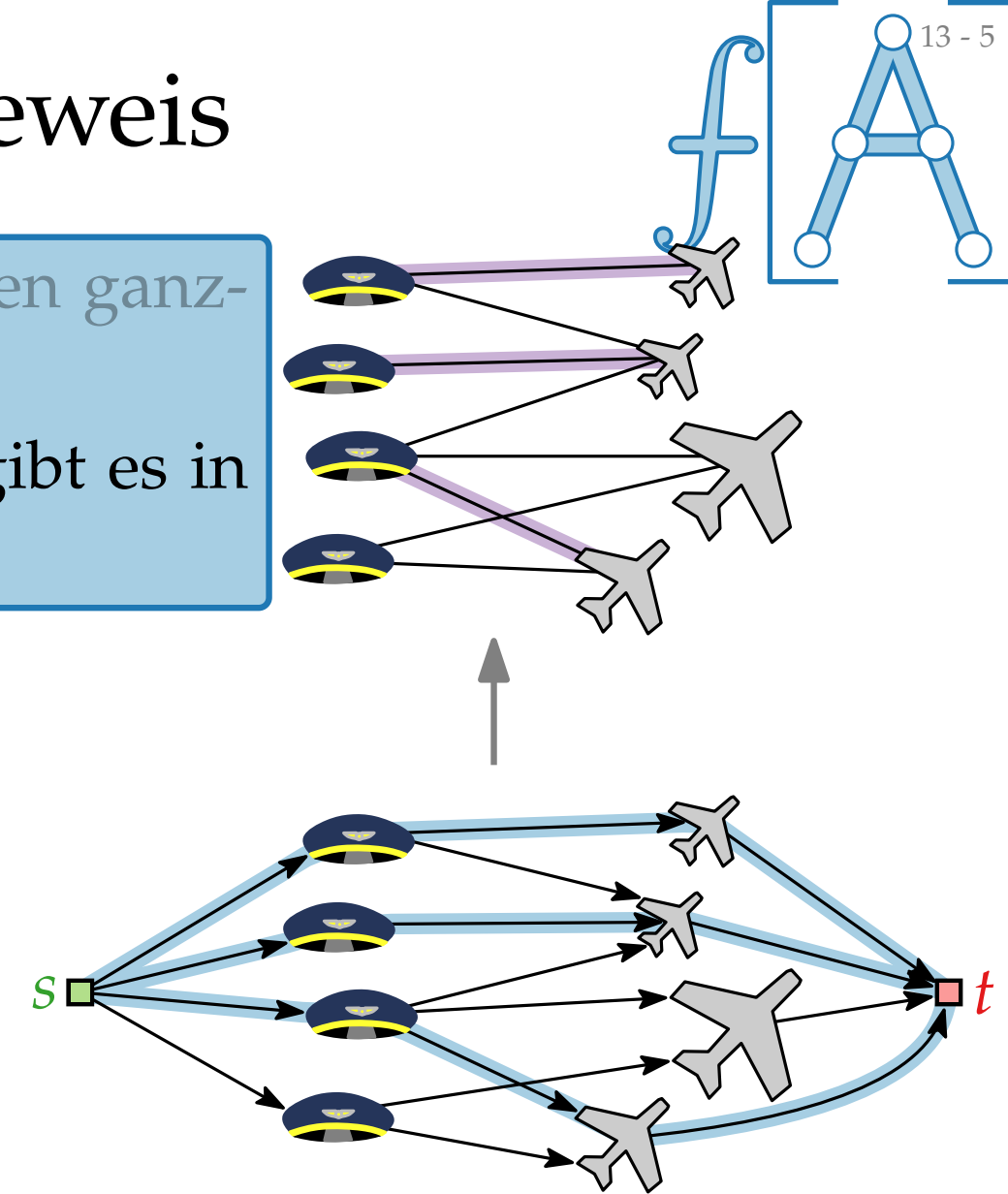
- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

- $c(e) = 1$ für alle Kanten $e \in E'$
- Ganzzahligkeit von $f \Rightarrow f(e) = 0$ oder $f(e) = 1$
- Sei $(s, \text{👤})$ eine Kante mit $f(s, \text{👤}) = 1$.

$\Rightarrow$ 👤 hat einen Nachbarn ✈ mit $f(\text{👤}, \text{✈}) = 1$,

Flusserhaltung



Äquivalenz des Flussnetzwerks – Beweis

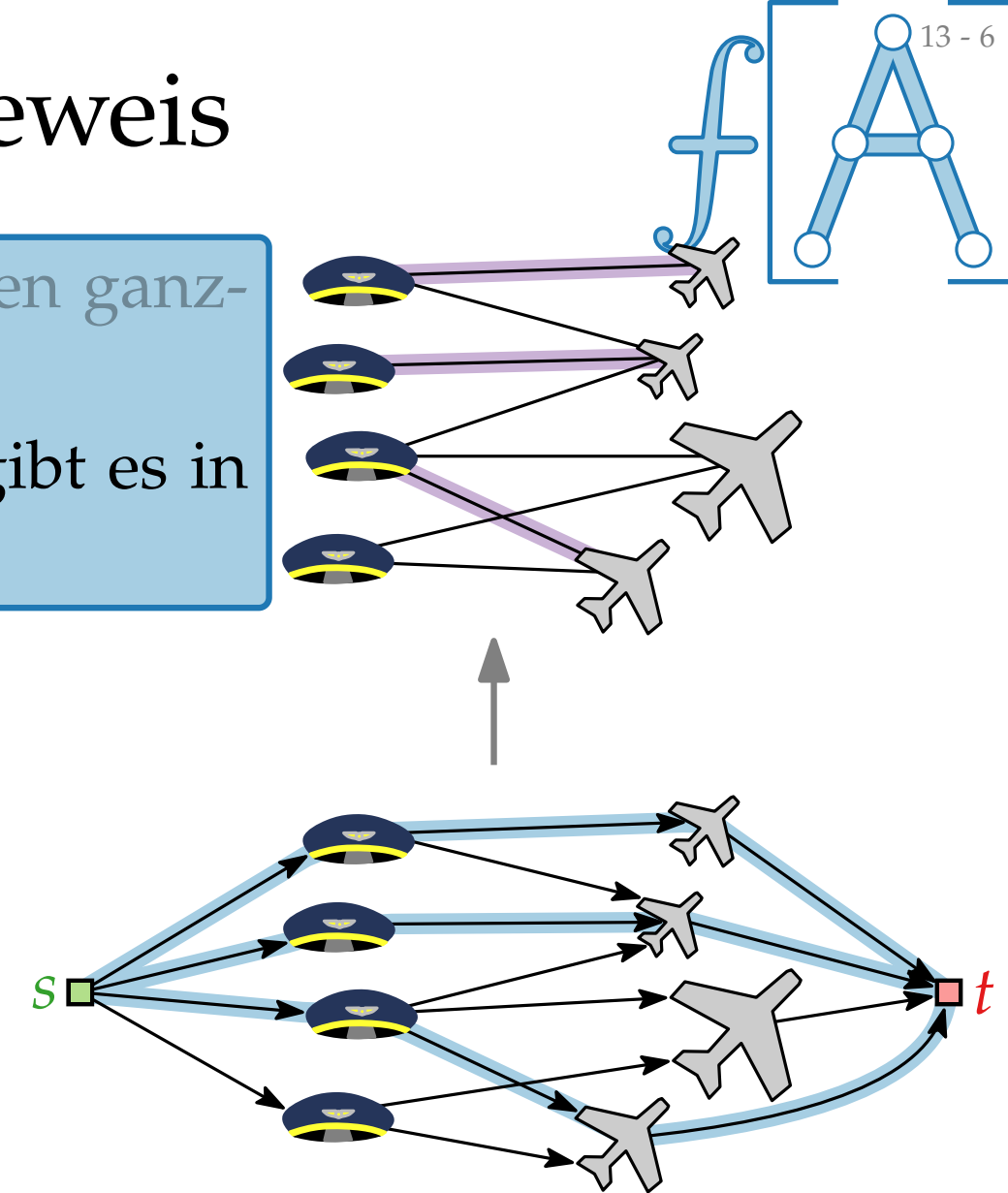
- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

- $c(e) = 1$ für alle Kanten $e \in E'$
- Ganzzahligkeit von $f \Rightarrow f(e) = 0$ oder $f(e) = 1$
- Sei $(s, \text{👤})$ eine Kante mit $f(s, \text{👤}) = 1$.

$\Rightarrow$ 👤 hat einen Nachbarn ✈ mit $f(\text{👤}, \text{✈}) = 1$,
für alle anderen Nachbarn ✈ gilt $f(\text{👤}, \text{✈}) = 0$

Flusser-
haltung



Äquivalenz des Flussnetzwerks – Beweis

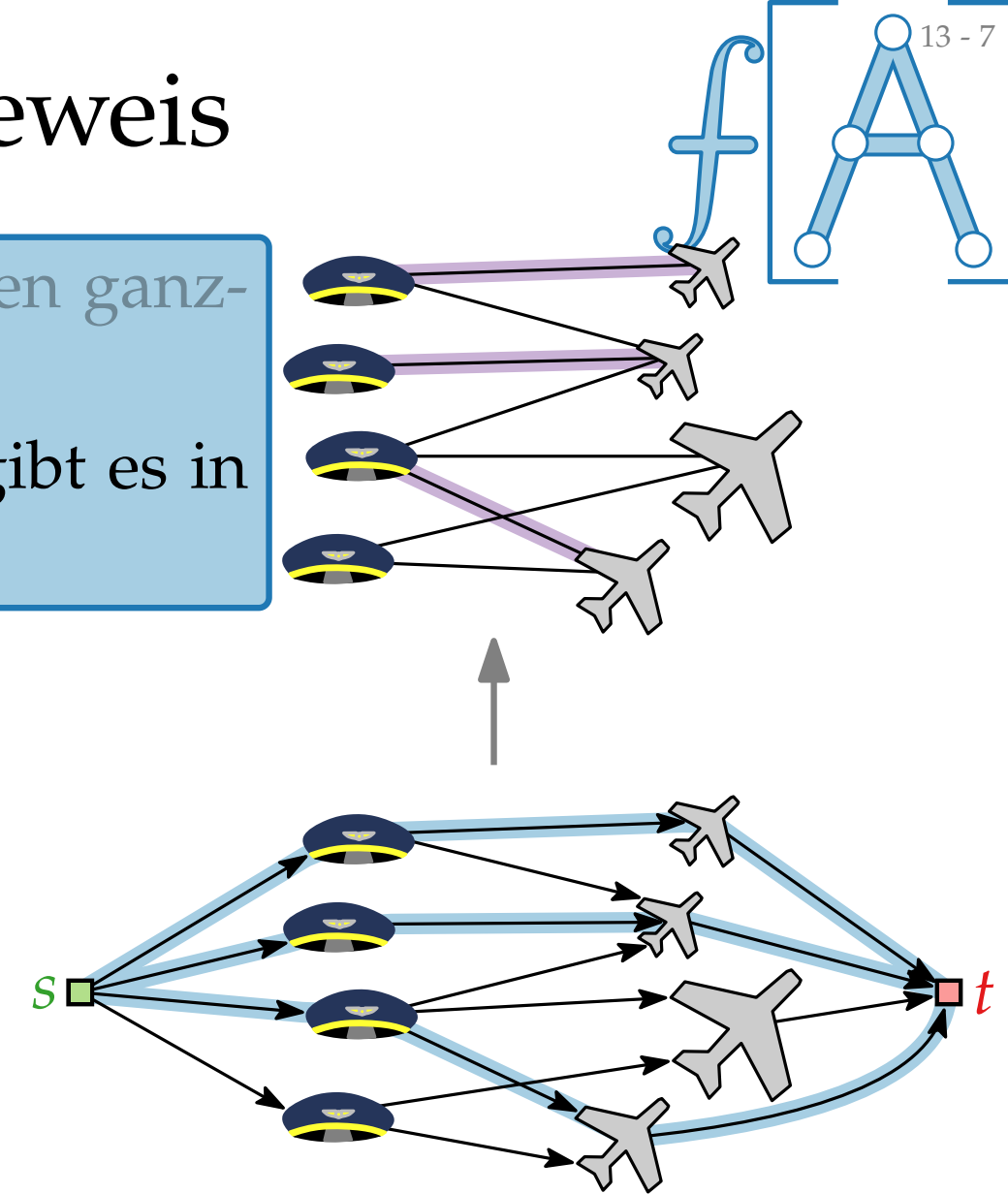
- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

- $c(e) = 1$ für alle Kanten $e \in E'$
- Ganzzahligkeit von $f \Rightarrow f(e) = 0$ oder $f(e) = 1$
- Sei $(s, \text{🛩️})$ eine Kante mit $f(s, \text{🛩️}) = 1$.

$\Rightarrow$ 🛩️ hat einen Nachbarn ✈️ mit $f(\text{🛩️}, \text{✈️}) = 1$,
für alle anderen Nachbarn ✈️ gilt $f(\text{🛩️}, \text{✈️}) = 0$
 $\Rightarrow f(\text{✈️}, t) = 1$

Flusserhaltung



Äquivalenz des Flussnetzwerks – Beweis

Lemma. 1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.

2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

■ $c(e) = 1$ für alle Kanten $e \in E'$

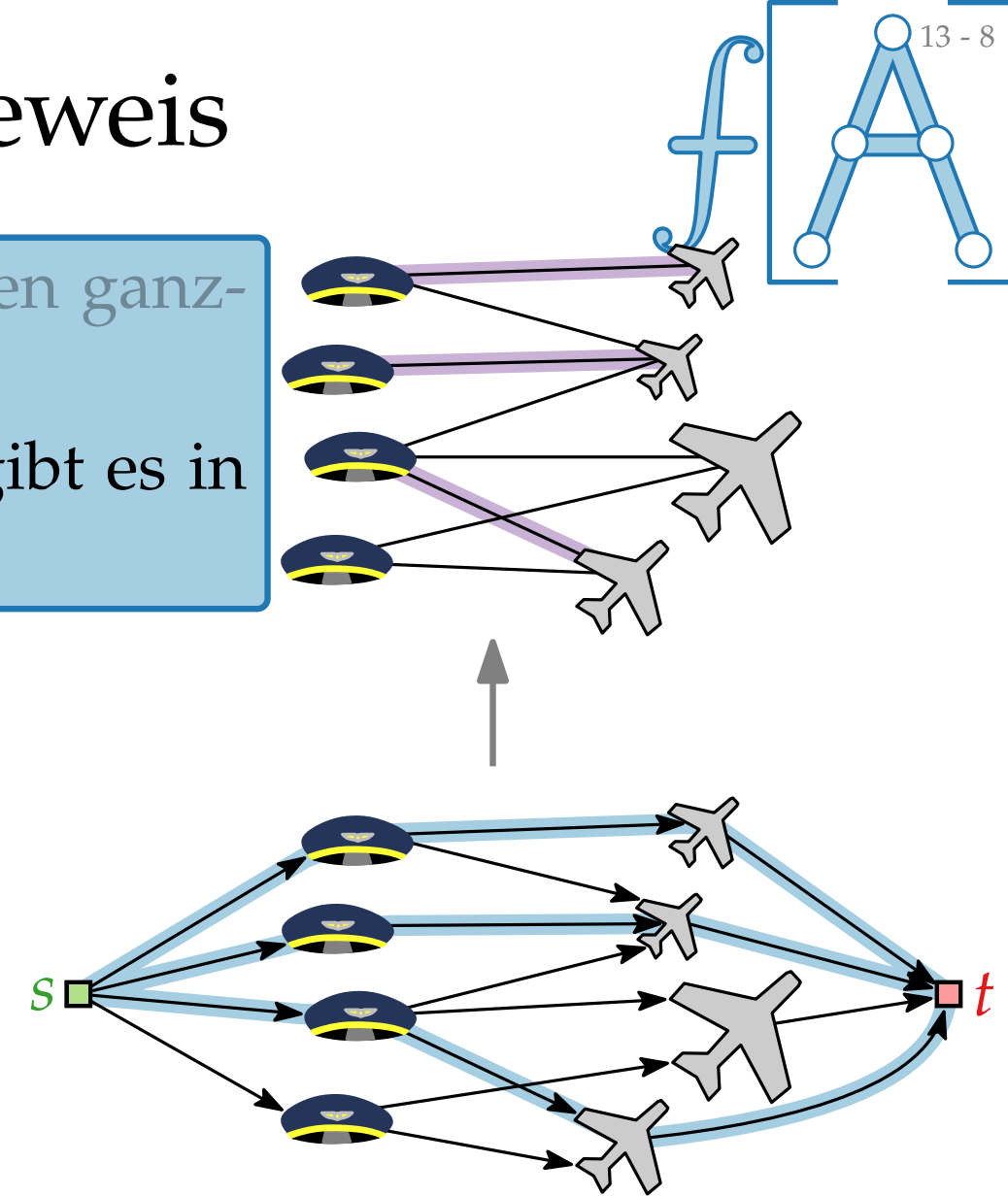
■ Ganzzahligkeit von $f \Rightarrow f(e) = 0$ oder $f(e) = 1$

■ Sei $(s, \text{🚂})$ eine Kante mit $f(s, \text{🚂}) = 1$.

$\Rightarrow$ 🚂 hat einen Nachbarn ✈ mit $f(\text{🚂}, \text{✈}) = 1$,
für alle anderen Nachbarn ✈ gilt $f(\text{🚂}, \text{✈}) = 0$

$\Rightarrow f(\text{✈}, t) = 1$

■ Jeder Knoten 🚂/✈ hat nur eine ein-/ausgehende Kante



Flusserhaltung

Äquivalenz des Flussnetzwerks – Beweis

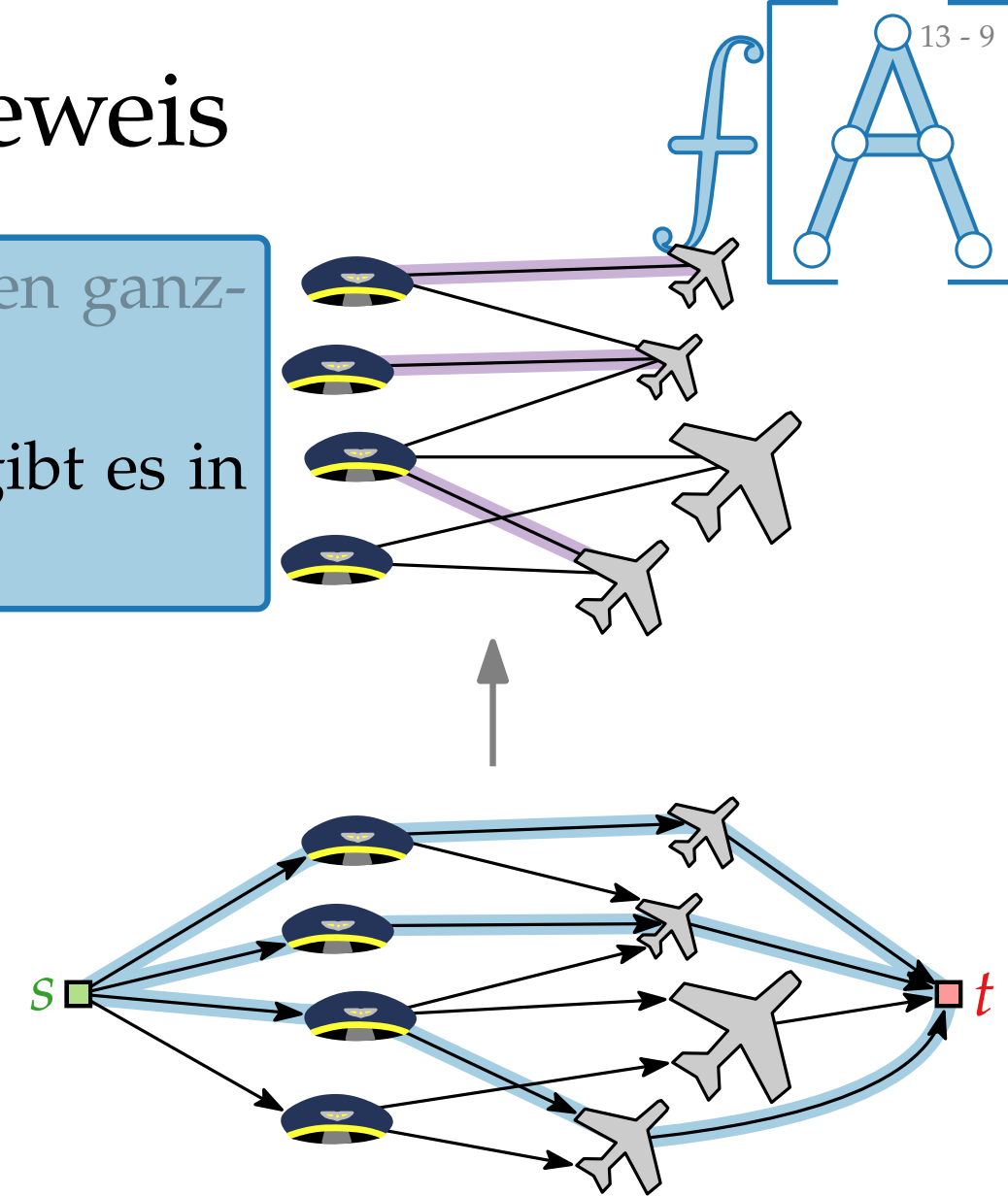
- Lemma.**
1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.
 2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

- $c(e) = 1$ für alle Kanten $e \in E'$
- Ganzzahligkeit von $f \Rightarrow f(e) = 0$ oder $f(e) = 1$
- Sei $(s, \text{🚂})$ eine Kante mit $f(s, \text{🚂}) = 1$.

$\Rightarrow$ 🚂 hat einen Nachbarn ✈ mit $f(\text{🚂}, \text{✈}) = 1$,
für alle anderen Nachbarn ✈ gilt $f(\text{🚂}, \text{✈}) = 0$
 $\Rightarrow f(\text{✈}, t) = 1$

- Jeder Knoten 🚂/✈ hat nur eine ein-/ausgehende Kante
 $\Rightarrow$ Kanten $(\text{🚂}, \text{✈})$ mit Fluss 1 in G' bilden Paarung P in G mit $|P| = |f|$.



Äquivalenz des Flussnetzwerks – Beweis

Lemma. 1. Sei P eine Paarung in G . Dann gibt es einen ganzzahligen Fluss f in G' mit Wert $|f| = |P|$.

2. Sei f ein ganzzahliger Fluss in G' . Dann gibt es in G eine Paarung P mit $|P| = |f|$.

Beweis.

■ $c(e) = 1$ für alle Kanten $e \in E'$

■ Ganzzahligkeit von $f \Rightarrow f(e) = 0$ oder $f(e) = 1$

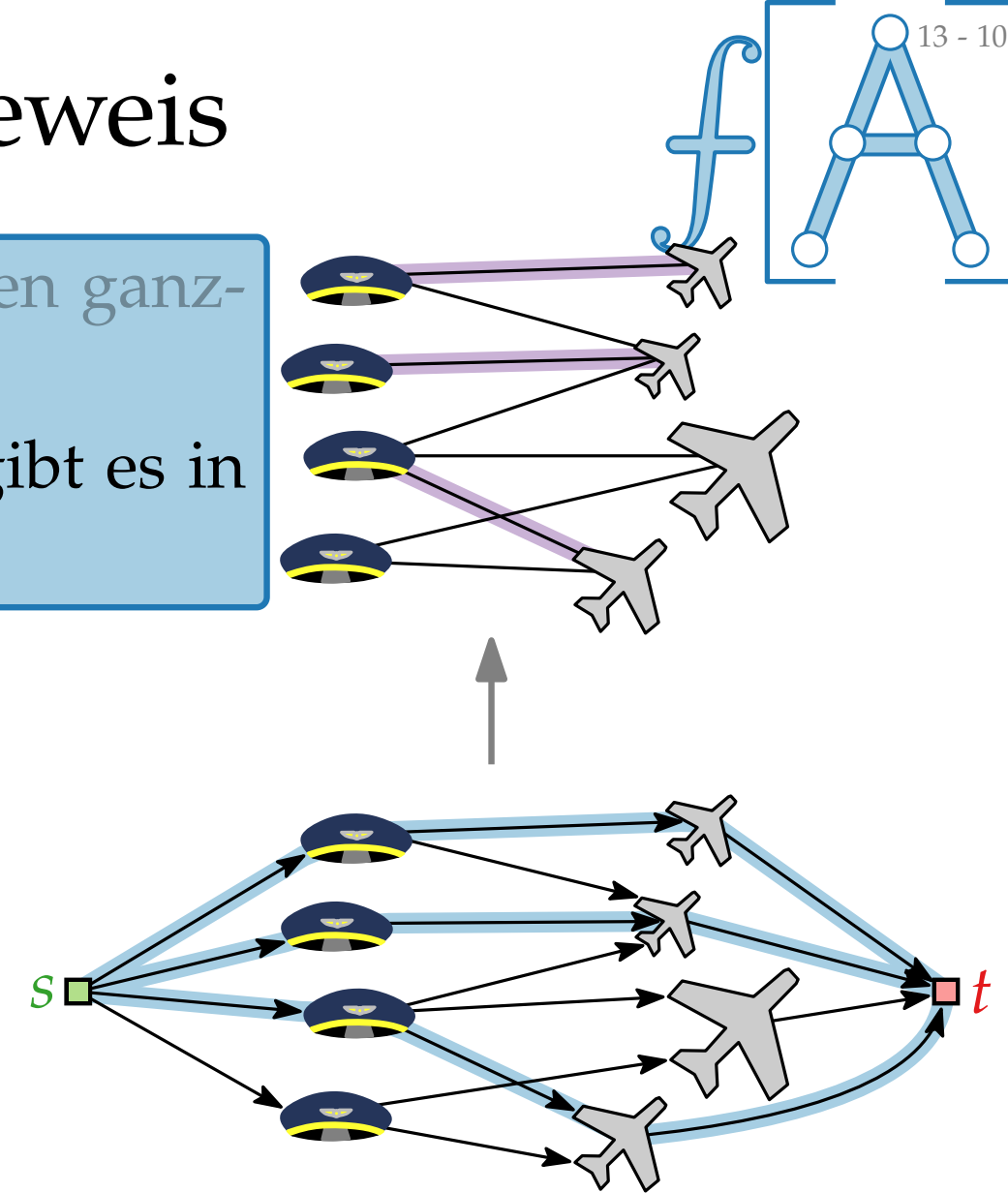
■ Sei $(s, \text{🚂})$ eine Kante mit $f(s, \text{🚂}) = 1$.

$\Rightarrow$ 🚂 hat einen Nachbarn ✈ mit $f(\text{🚂}, \text{✈}) = 1$,
für alle anderen Nachbarn ✈ gilt $f(\text{🚂}, \text{✈}) = 0$

$\Rightarrow f(\text{✈}, t) = 1$

■ Jeder Knoten 🚂/✈ hat nur eine ein-/ausgehende Kante

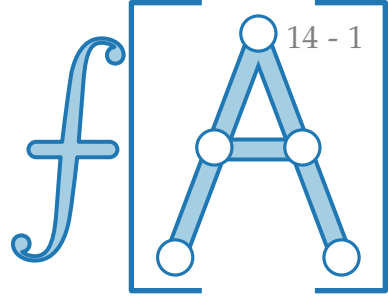
$\Rightarrow$ Kanten $(\text{🚂}, \text{✈})$ mit Fluss 1 in G' bilden Paarung P in G mit $|P| = |f|$. $\square$



Flusserhaltung

Der Algorithmus

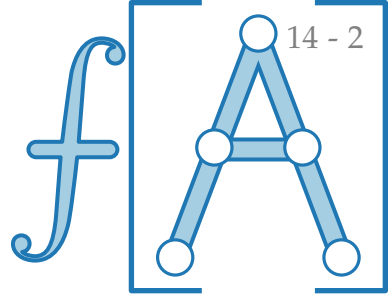
Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.



Der Algorithmus

Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

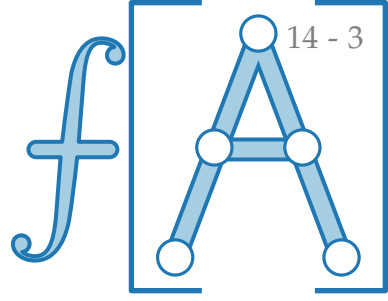


Der Algorithmus

Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\mathbb{B} \cup \mathbb{A}, E)$)



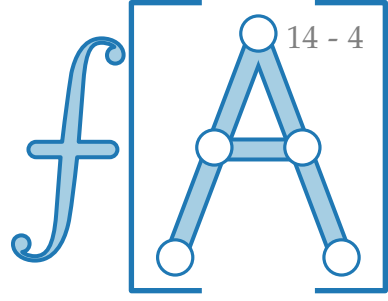
Der Algorithmus

Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{♂} \cup \text{♀}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G



Der Algorithmus

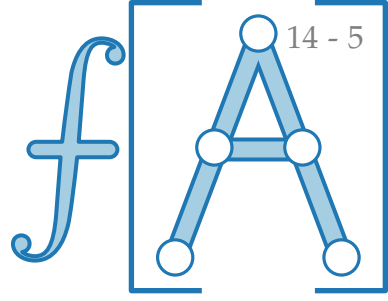
Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

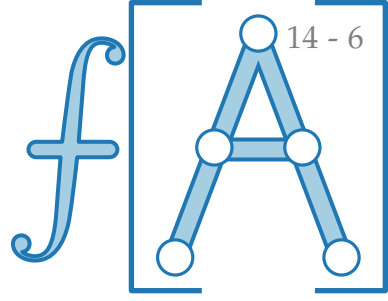
GRÖSSTEBIPARTITEPAARUNG($G = (\text{♂} \cup \text{♀}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss



Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{👤} \cup \text{👩}, E)$)

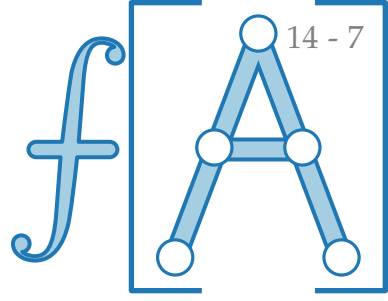
$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

└

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{♂} \cup \text{♀}, E)$)

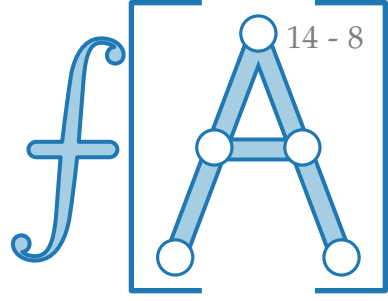
$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{👤} \cup \text{✈️}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

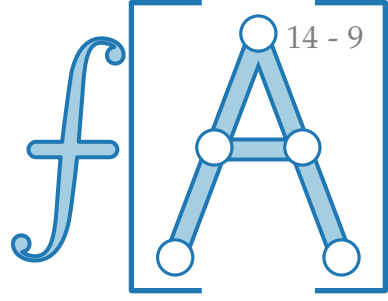
$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{👤}, \text{✈️}\}$ mit $f(\text{👤}, \text{✈️}) = 1$

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{👤} \cup \text{✈️}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

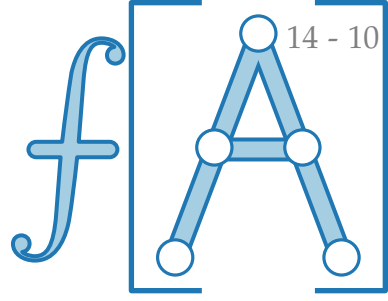
while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{👤}, \text{✈️}\}$ mit $f(\text{👤}, \text{✈️}) = 1$

$\mathcal{O}(m)$

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{👤} \cup \text{✈️}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

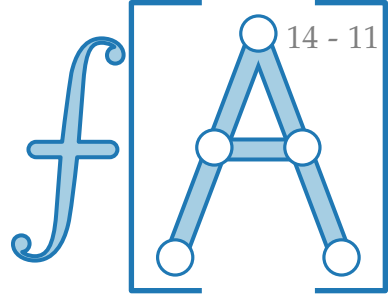
$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{👤}, \text{✈️}\}$ mit $f(\text{👤}, \text{✈️}) = 1$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{👤} \cup \text{✈️}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

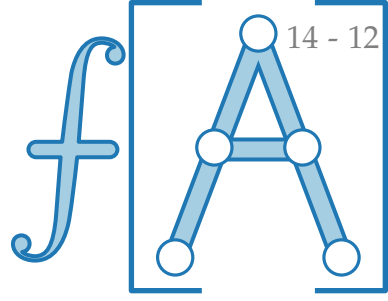
$P \leftarrow$ alle Kanten $\{\text{👤}, \text{✈️}\}$ mit $f(\text{👤}, \text{✈️}) = 1$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{👤} \cup \text{✈️}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{👤}, \text{✈️}\}$ mit $f(\text{👤}, \text{✈️}) = 1$

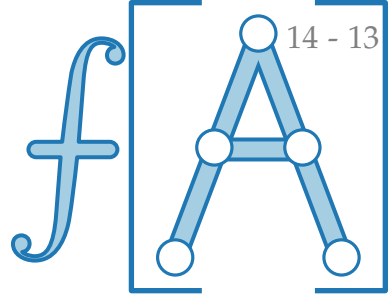
$\mathcal{O}(m)$

$\mathcal{O}(m)$

$\mathcal{O}(nm)$

$\mathcal{O}(m)$

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{👤} \cup \text{✈️}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{👤}, \text{✈️}\}$ mit $f(\text{👤}, \text{✈️}) = 1$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

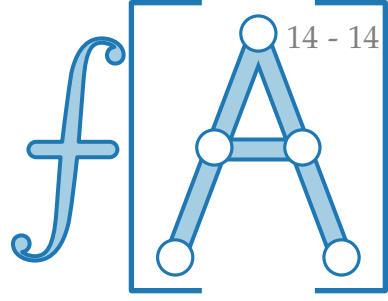
$\mathcal{O}(nm)$

$\mathcal{O}(m)$

Es gilt

$|f| = |P| \leq n/2$,
da jede Kante in P
zwei Knoten in V
besetzt.

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{👤} \cup \text{✈️}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{👤}, \text{✈️}\}$ mit $f(\text{👤}, \text{✈️}) = 1$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

$\mathcal{O}(nm)$

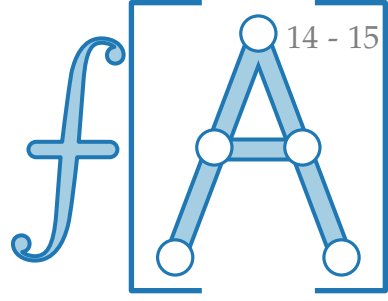
$\mathcal{O}(m)$

$\mathcal{O}(m)$

Es gilt

$|f| = |P| \leq n/2$,
da jede Kante in P
zwei Knoten in V
besetzt.

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{🚶} \cup \text{✈️}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{🚶}, \text{✈️}\}$ mit $f(\text{🚶}, \text{✈️}) = 1$

$\mathcal{O}(nm)$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

$\mathcal{O}(nm)$

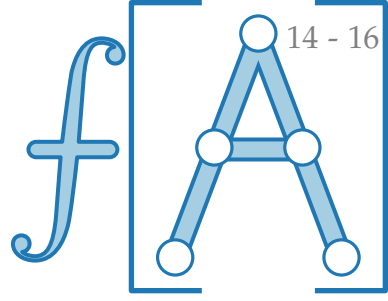
$\mathcal{O}(m)$

$\mathcal{O}(m)$

Es gilt

$|f| = |P| \leq n/2$,
da jede Kante in P
zwei Knoten in V
besetzt.

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{☺} \cup \text{✈}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{☺}, \text{✈}\}$ mit $f(\text{☺}, \text{✈}) = 1$

$\mathcal{O}(nm)$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

$\mathcal{O}(nm)$

$\mathcal{O}(m)$

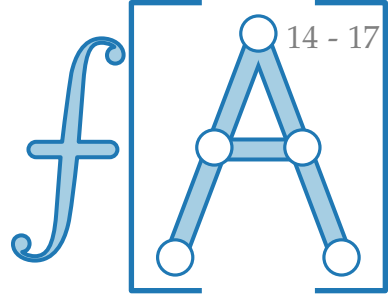
$\mathcal{O}(m)$

Es gilt

$|f| = |P| \leq n/2$,
da jede Kante in P
zwei Knoten in V
besetzt.

Satz. Sei G ein bipartiter Graph. Der Algorithmus GRÖSSTEBIPARTITEPAARUNG berechnet eine größte Paarung in G in $\mathcal{O}(nm)$ Zeit.

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{👤} \cup \text{✈️}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{👤}, \text{✈️}\}$ mit $f(\text{👤}, \text{✈️}) = 1$

$\mathcal{O}(nm)$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

$\mathcal{O}(nm)$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

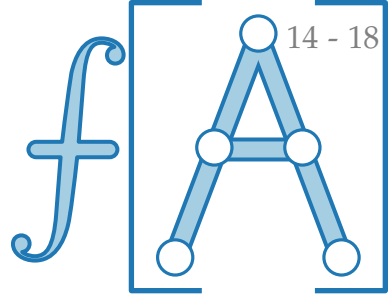
Es gilt

$|f| = |P| \leq n/2$,
da jede Kante in P
zwei Knoten in V
besetzt.

Satz. Sei G ein bipartiter Graph. Der Algorithmus GRÖSSTEBIPARTITEPAARUNG berechnet eine größte Paarung in G in $\mathcal{O}(nm)$ Zeit.

$\mathcal{O}(\sqrt{nm})$
mit DINIC
Algorithmus

Der Algorithmus



Der FORDFULKERSON Algorithmus liefert bei ganzzahligen Kapazitäten einen ganzzahligen maximalen Fluss.

⇒ Er kann benutzt werden um GRÖSSTEBIPARTITEPAARUNG zu lösen.

GRÖSSTEBIPARTITEPAARUNG($G = (\text{☞} \cup \text{✈}, E)$)

$(G', c, s, t) \leftarrow$ zugehöriges Flussnetzwerk von G

$f \leftarrow$ 0-Fluss

while Residualnetzwerk G_f von f enthält s - t -Weg **do**

$f \leftarrow f$ erhöht um s - t -Weg

$P \leftarrow$ alle Kanten $\{\text{☞}, \text{✈}\}$ mit $f(\text{☞}, \text{✈}) = 1$

$\mathcal{O}(nm)$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

$\mathcal{O}(nm)$

$\mathcal{O}(m)$

$\mathcal{O}(m)$

Es gilt

$|f| = |P| \leq n/2$,
da jede Kante in P
zwei Knoten in V
besetzt.

Satz. Sei G ein bipartiter Graph. Der Algorithmus GRÖSSTEBIPARTITEPAARUNG berechnet eine größte Paarung in G in $\mathcal{O}(nm)$ Zeit.

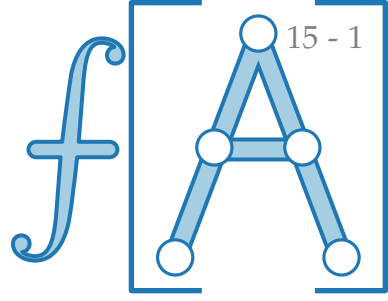
$\mathcal{O}(\sqrt{nm})$
mit DINIC
Algorithmus

Satz. Selbst in einem beliebigen Graphen $G = (V, E)$ lässt sich eine größte Paarung in $\mathcal{O}(\sqrt{nm})$ Zeit berechnen. [Micali&Vazirani, FOCS'80]

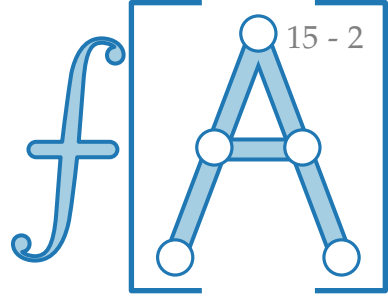
Der gewichtete Fall

Problem. GRÖSSTE GEWICHTETE PAARUNG

Finde eine möglichst **schwere** Paarung P in einem gewichteten Graphen $G = (V, E, w)$.



Der gewichtete Fall

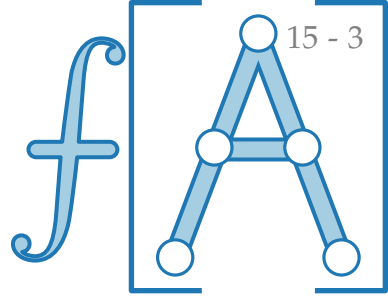


Problem. GRÖSSTE GEWICHTETE PAARUNG

Finde eine möglichst **schwere** Paarung P in einem gewichteten Graphen $G = (V, E, w)$.

Satz. Sei G ein bipartiter Graph. Der UNGARISCHE ALGORITHMUS berechnet eine größte Paarung in G in $\mathcal{O}(n^2 \log n + nm)$ Zeit.

Der gewichtete Fall

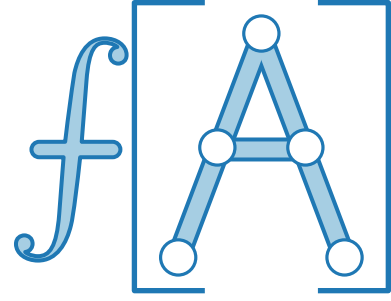


Problem. GRÖSSTE GEWICHTETE PAARUNG

Finde eine möglichst **schwere** Paarung P in einem gewichteten Graphen $G = (V, E, w)$.

Satz. Sei G ein bipartiter Graph. Der UNGARISCHE ALGORITHMUS berechnet eine größte Paarung in G in $\mathcal{O}(n^2 \log n + nm)$ Zeit.

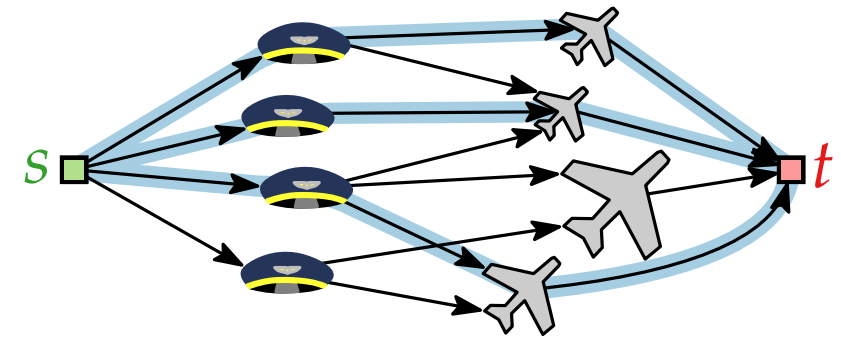
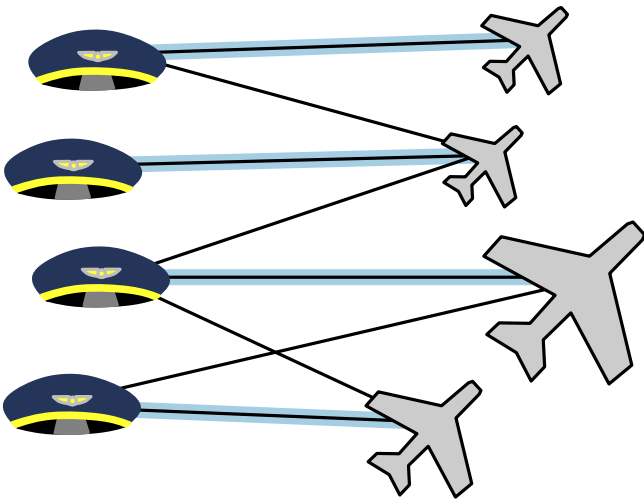
Satz. Selbst in einem beliebigen Graphen $G = (V, E)$ lässt sich eine größte Paarung in $\mathcal{O}(n^2 m)$ Zeit berechnen. [Lovász&Plummer 1986]



Fortgeschrittene Algorithmen

9. Vorlesung Graphalgorithmen III: Paarungen / Matchings

Teil IV: Der Heiratssatz

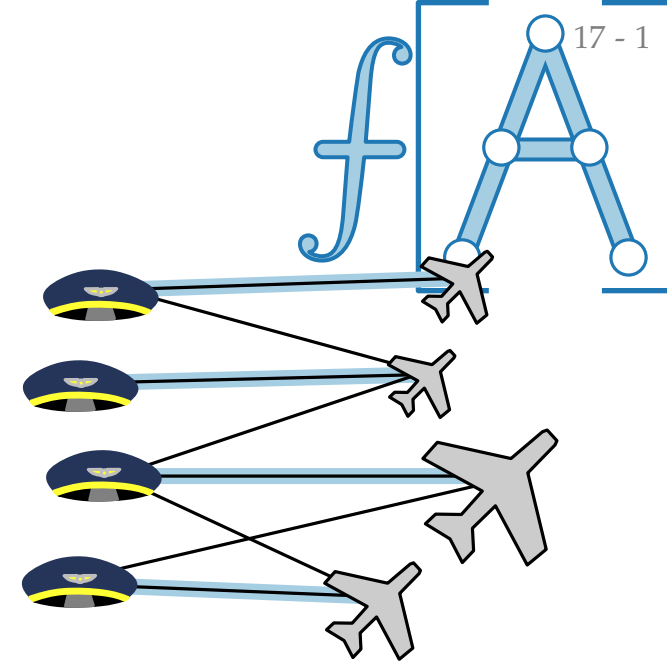


Perfekte Paarungen

Def. Falls jeder Knoten in G durch P gepaart ist, so ist M eine **perfekte Paarung** (engl. *perfect*).

Problem. PERFEKTE BIPARTITE PAARUNG

Finde eine perfekte Paarung P in einem bipartiten Graphen G .

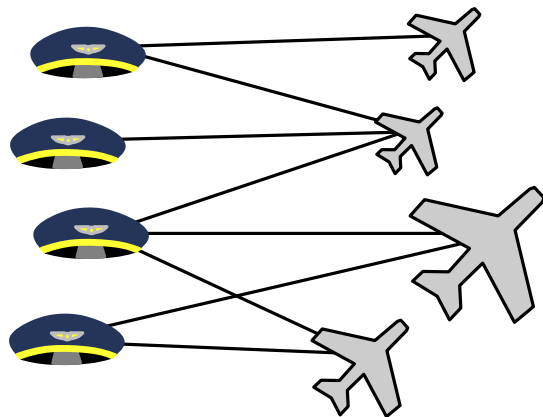
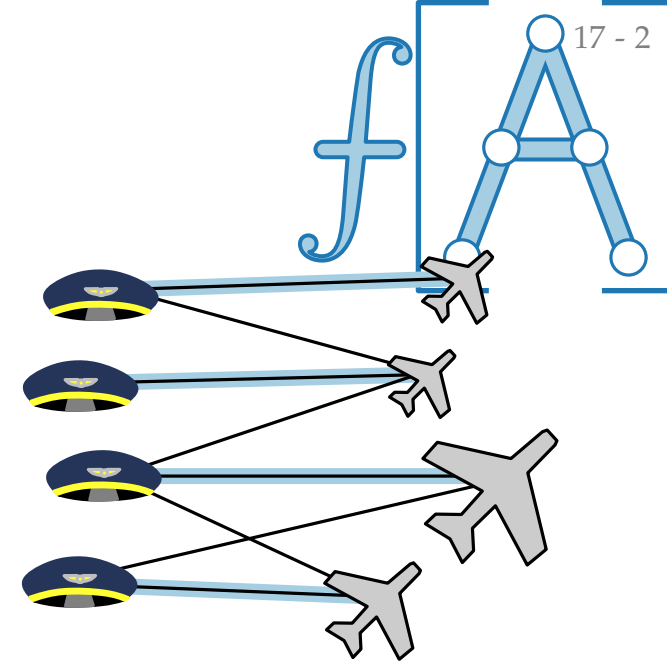


Perfekte Paarungen

Def. Falls jeder Knoten in G durch P gepaart ist, so ist M eine **perfekte Paarung** (engl. *perfect*).

Problem. PERFEKTE BIPARTITE PAARUNG

Finde eine perfekte Paarung P in einem bipartiten Graphen G .

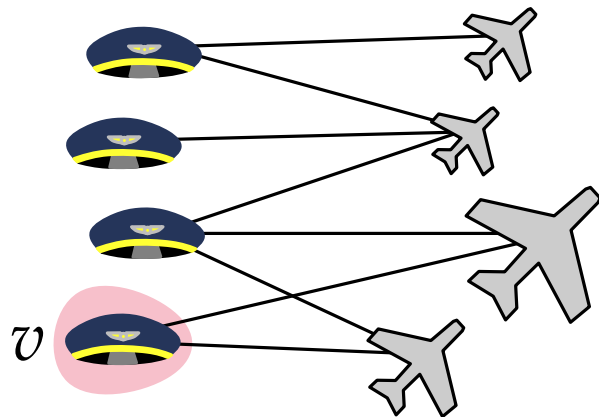
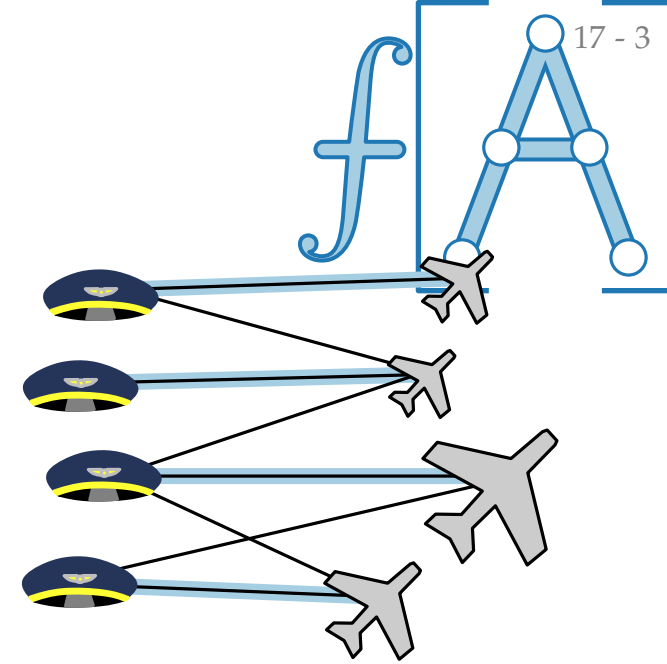


Perfekte Paarungen

Def. Falls jeder Knoten in G durch P gepaart ist, so ist M eine **perfekte Paarung** (engl. *perfect*).

Problem. PERFEKTE BIPARTITE PAARUNG

Finde eine perfekte Paarung P in einem bipartiten Graphen G .



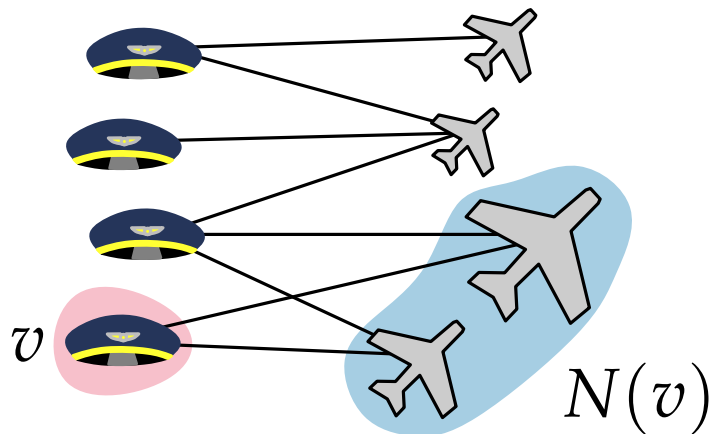
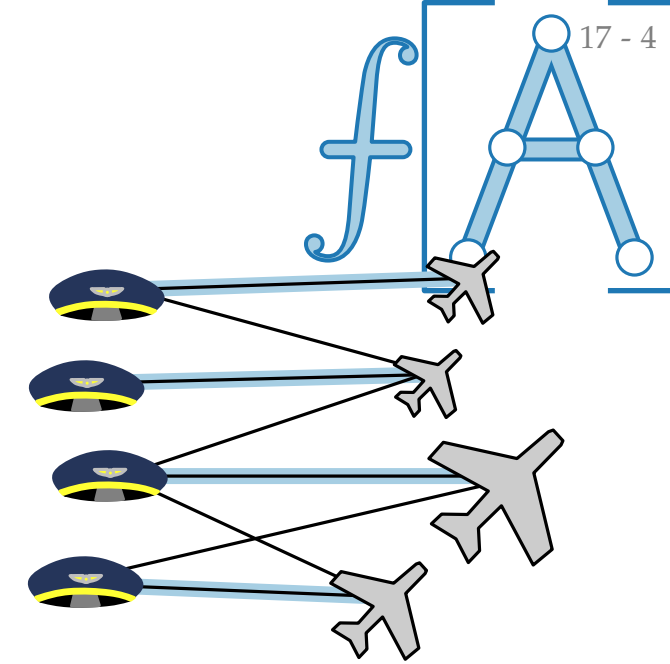
Nachbarschaft von $v \in V$ ist

Perfekte Paarungen

Def. Falls jeder Knoten in G durch P gepaart ist, so ist M eine **perfekte Paarung** (engl. *perfect*).

Problem. PERFEKTE BIPARTITE PAARUNG

Finde eine perfekte Paarung P in einem bipartiten Graphen G .



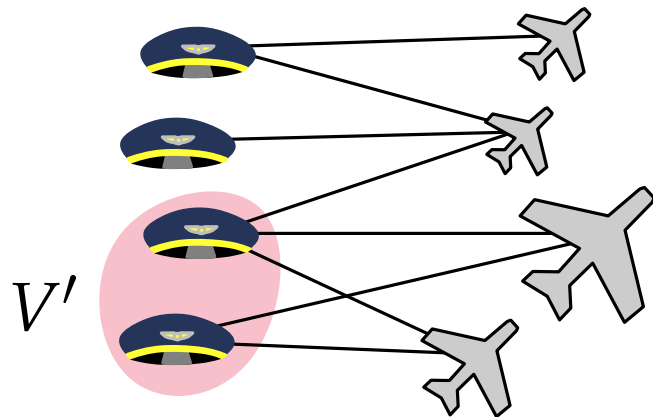
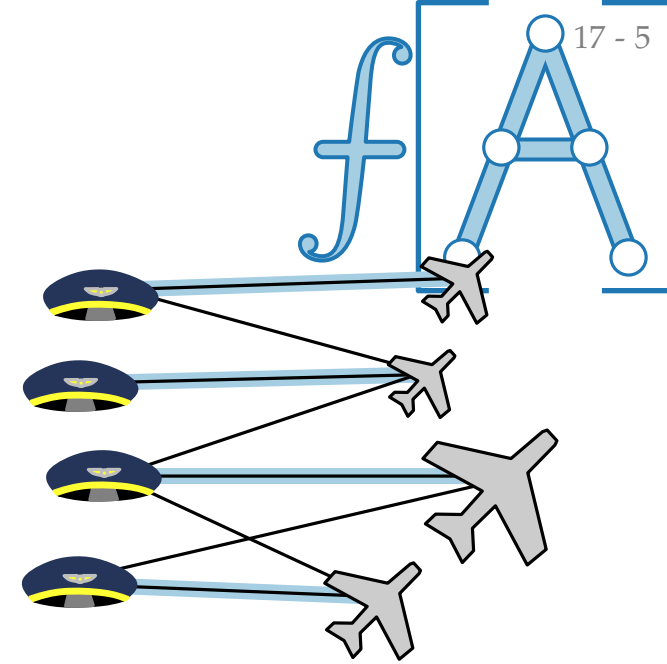
Nachbarschaft von $v \in V$ ist
$$N(v) := \{u \in V \mid uv \in E\}$$

Perfekte Paarungen

Def. Falls jeder Knoten in G durch P gepaart ist, so ist M eine **perfekte Paarung** (engl. *perfect*).

Problem. PERFEKTE BIPARTITE PAARUNG

Finde eine perfekte Paarung P in einem bipartiten Graphen G .



Nachbarschaft von $v \in V$ ist

$$N(v) := \{u \in V \mid uv \in E\}$$

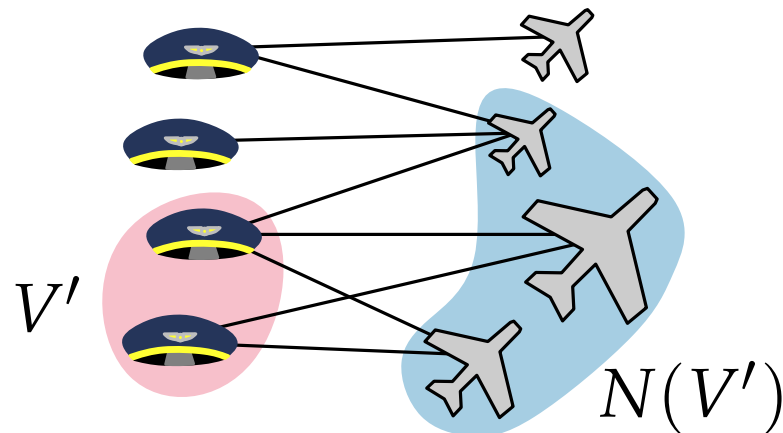
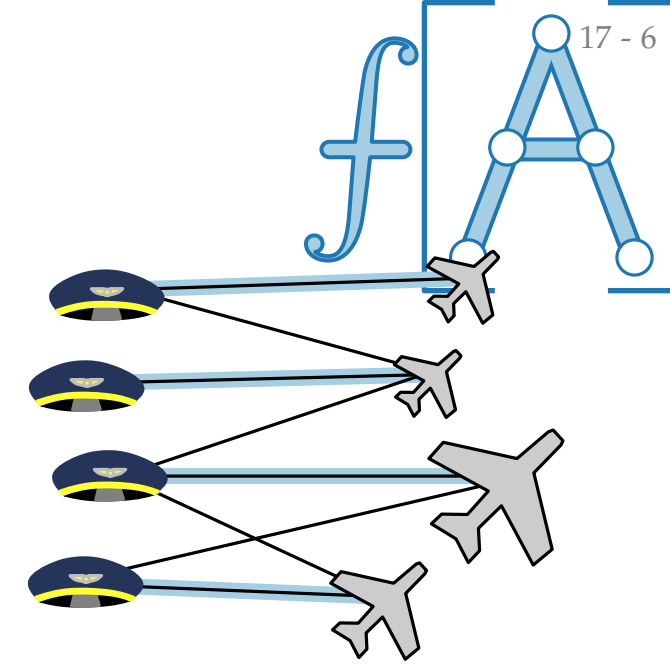
Nachbarschaft von $V' \subseteq V$ ist

Perfekte Paarungen

Def. Falls jeder Knoten in G durch P gepaart ist, so ist M eine **perfekte Paarung** (engl. *perfect*).

Problem. PERFEKTE BIPARTITE PAARUNG

Finde eine perfekte Paarung P in einem bipartiten Graphen G .



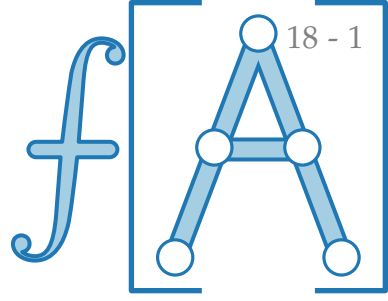
Nachbarschaft von $v \in V$ ist

$$N(v) := \{u \in V \mid uv \in E\}$$

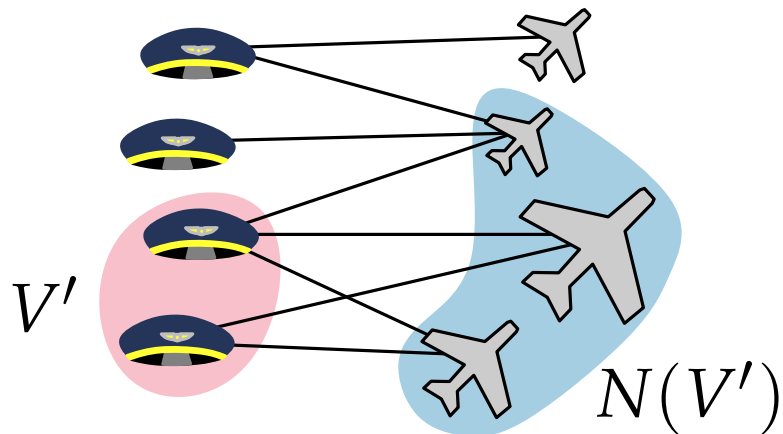
Nachbarschaft von $V' \subseteq V$ ist

$$N(V') := \bigcup_{v' \in V'} N(v')$$

Der Heiratssatz



Aufgabe. Finden Sie eine *notwendige* Bedingung für die Existenz einer perfekten Paarung in $G = (\text{♂} \cup \text{♀}, E)$!



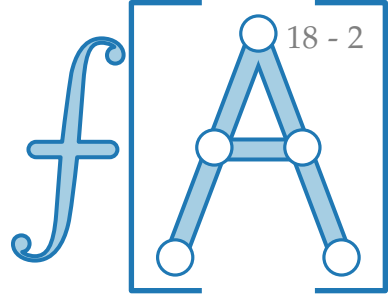
Nachbarschaft von $v \in V$ ist

$$N(v) := \{u \in V \mid uv \in E\}$$

Nachbarschaft von $V' \subseteq V$ ist

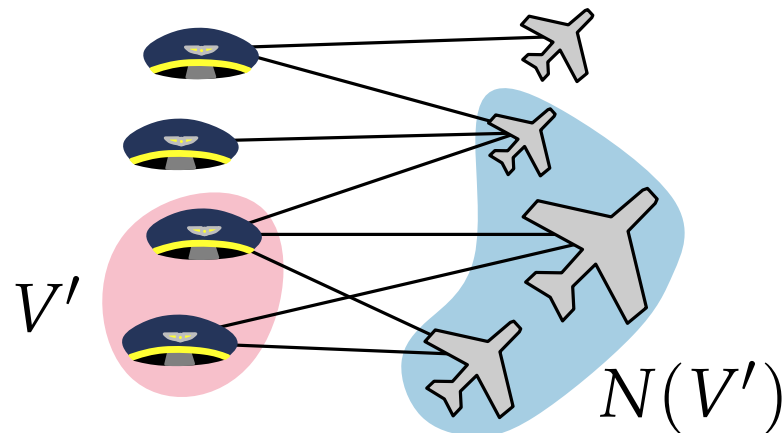
$$N(V') := \bigcup_{v' \in V'} N(v')$$

Der Heiratssatz



Aufgabe. Finden Sie eine *notwendige* Bedingung für die Existenz einer perfekten Paarung in $G = (\text{♂} \cup \text{♀}, E)$!

Lösung. Für jedes $V' \subseteq \text{♂}$ muss gelten:



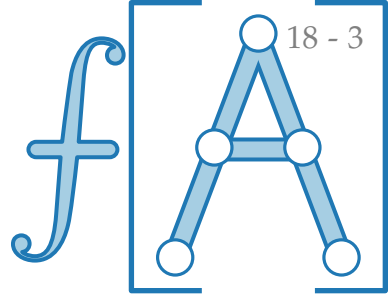
Nachbarschaft von $v \in V$ ist

$$N(v) := \{u \in V \mid uv \in E\}$$

Nachbarschaft von $V' \subseteq V$ ist

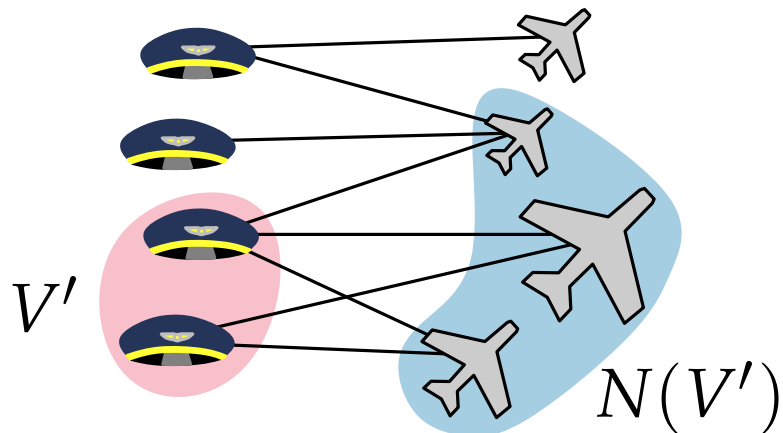
$$N(V') := \bigcup_{v' \in V'} N(v')$$

Der Heiratssatz



Aufgabe. Finden Sie eine *notwendige* Bedingung für die Existenz einer perfekten Paarung in $G = (\text{♂} \cup \text{♀}, E)$!

Lösung. Für jedes $V' \subseteq \text{♂}$ muss gelten: $|V'| \leq |N(V')|$.



Nachbarschaft von $v \in V$ ist

$$N(v) := \{u \in V \mid uv \in E\}$$

Nachbarschaft von $V' \subseteq V$ ist

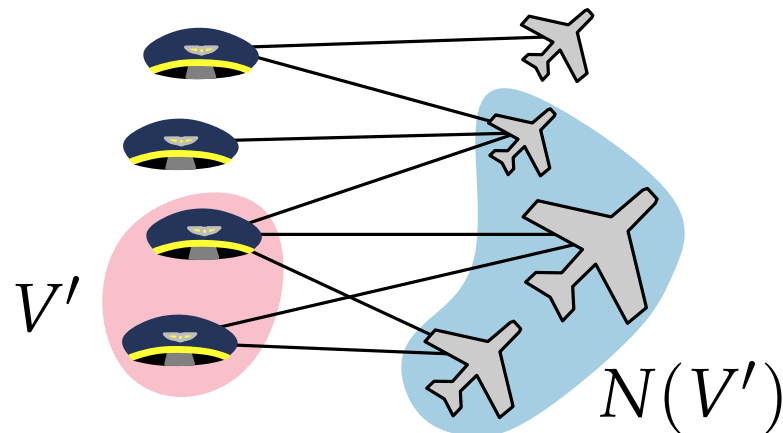
$$N(V') := \bigcup_{v' \in V'} N(v')$$

Der Heiratssatz

Aufgabe. Finden Sie eine *notwendige* Bedingung für die Existenz einer perfekten Paarung in $G = (\text{♂} \cup \text{♀}, E)$!

Lösung. Für jedes $\text{♂} \subseteq \text{♂}$ muss gelten: $|\text{♂}| \leq |N(\text{♂})|$.

Satz. [Hall 1935]

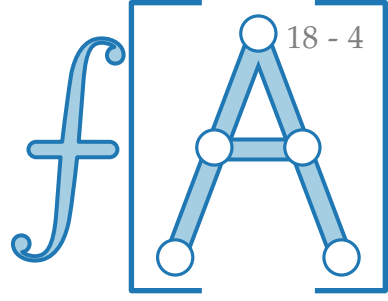


Nachbarschaft von $v \in V$ ist

$$N(v) := \{u \in V \mid uv \in E\}$$

Nachbarschaft von $V' \subseteq V$ ist

$$N(V') := \bigcup_{v' \in V'} N(v')$$



Philip Hall
*1904 London
†1982 Cambridge

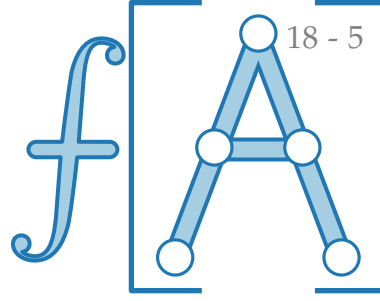


Der Heiratssatz

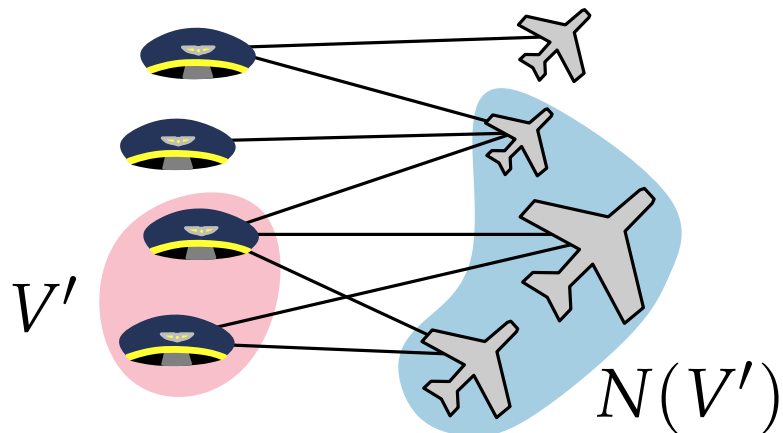
Aufgabe. Finden Sie eine *notwendige* Bedingung für die Existenz einer perfekten Paarung in $G = (\text{♂} \cup \text{♀}, E)$!

Lösung. Für jedes $\text{♂} \subseteq \text{♂}$ muss gelten: $|\text{♂}| \leq |N(\text{♂})|$.

Satz. Dies ist auch hinreichend. [Hall 1935]



Philip Hall
*1904 London
†1982 Cambridge



Nachbarschaft von $v \in V$ ist

$$N(v) := \{u \in V \mid uv \in E\}$$

Nachbarschaft von $V' \subseteq V$ ist

$$N(V') := \bigcup_{v' \in V'} N(v')$$

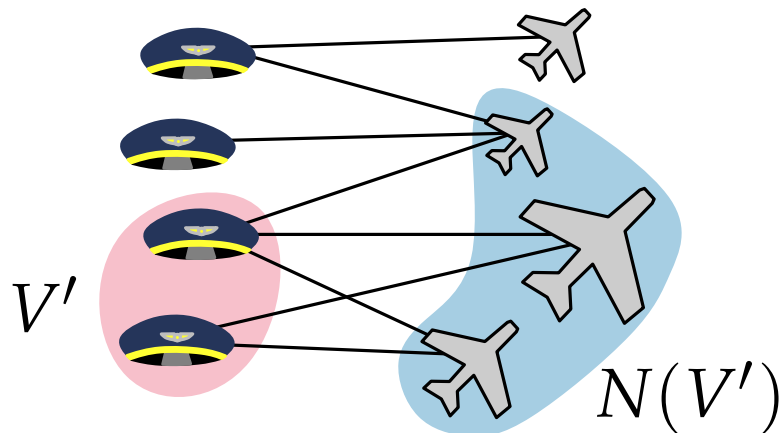
Der Heiratssatz

Aufgabe. Finden Sie eine *notwendige* Bedingung für die Existenz einer perfekten Paarung in $G = (\text{♂} \cup \text{♀}, E)$!

Lösung. Für jedes $\text{♂} \subseteq \text{♂}$ muss gelten: $|\text{♂}| \leq |N(\text{♂})|$.

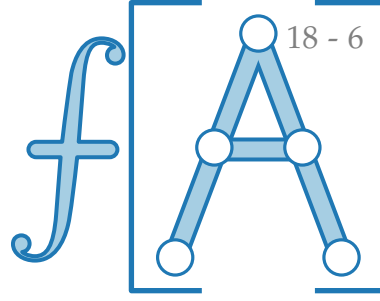
Satz. Dies ist auch hinreichend. [Hall 1935]

Das heißt: G hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\text{♂} \subseteq \text{♂}$ gilt $|\text{♂}| \leq |N(\text{♂})|$.



Nachbarschaft von $v \in V$ ist
$$N(v) := \{u \in V \mid uv \in E\}$$

Nachbarschaft von $V' \subseteq V$ ist
$$N(V') := \bigcup_{v' \in V'} N(v')$$



Philip Hall
*1904 London
†1982 Cambridge



Der Heiratssatz

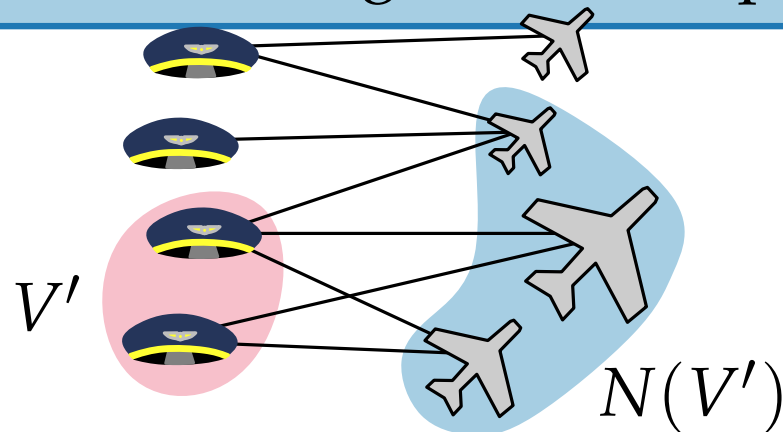
Aufgabe. Finden Sie eine *notwendige* Bedingung für die Existenz einer perfekten Paarung in $G = (\text{♂} \cup \text{♀}, E)$!

Lösung. Für jedes $\text{♂} \subseteq \text{♂}$ muss gelten: $|\text{♂}| \leq |N(\text{♂})|$.

Satz. Dies ist auch hinreichend. [Hall 1935]

Das heißt: G hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\text{♂} \subseteq \text{♂}$ gilt $|\text{♂}| \leq |N(\text{♂})|$.

Satz. Ein Graph $G = (V, E)$ hat eine perfekte Paarung $\Leftrightarrow$ für jedes $V' \subseteq V$ hat $G - V'$ höchstens $|V'|$ ungerade Komponenten. [Tutte 1950]

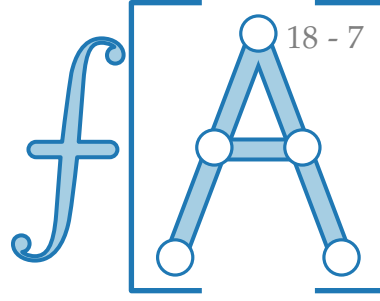


Nachbarschaft von $v \in V$ ist

$$N(v) := \{u \in V \mid uv \in E\}$$

Nachbarschaft von $V' \subseteq V$ ist

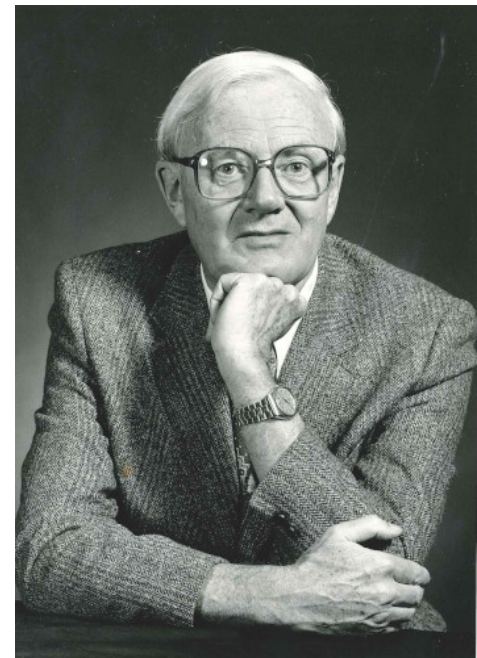
$$N(V') := \bigcup_{v' \in V'} N(v')$$



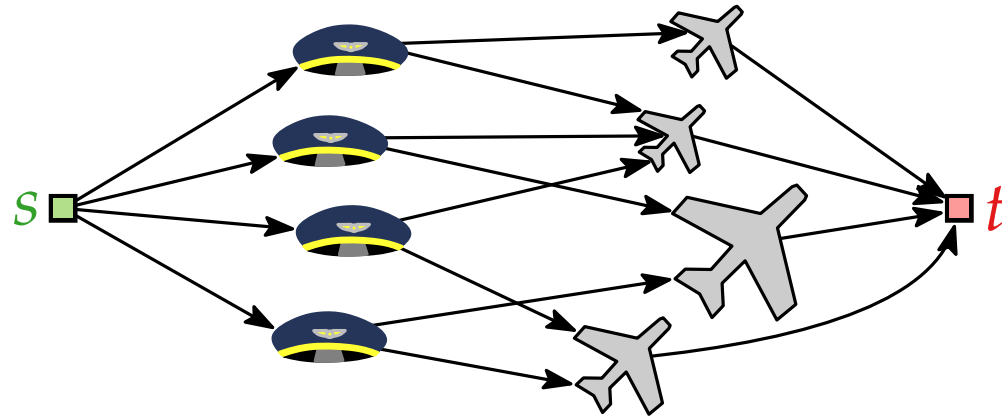
Philip Hall
 *1904 London
 †1982 Cambridge



William T. Tutte
 *1917 Newmarket, England
 †2002 Kitchener, ON, Canada



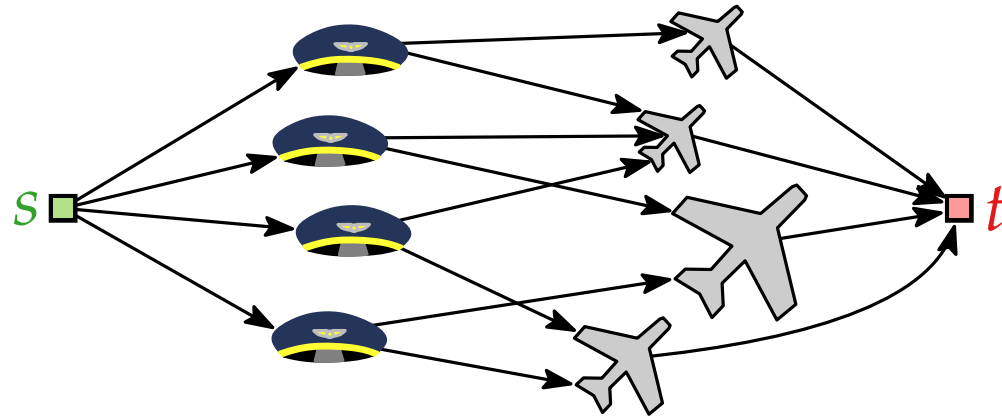
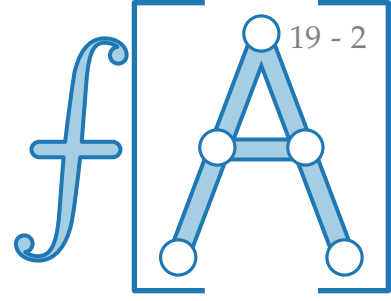
Beweis des Heiratssatzes



$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

Satz. $G = (\mathbb{M} \cup \mathbb{F}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis des Heiratssatzes



$$G \rightarrow G'$$

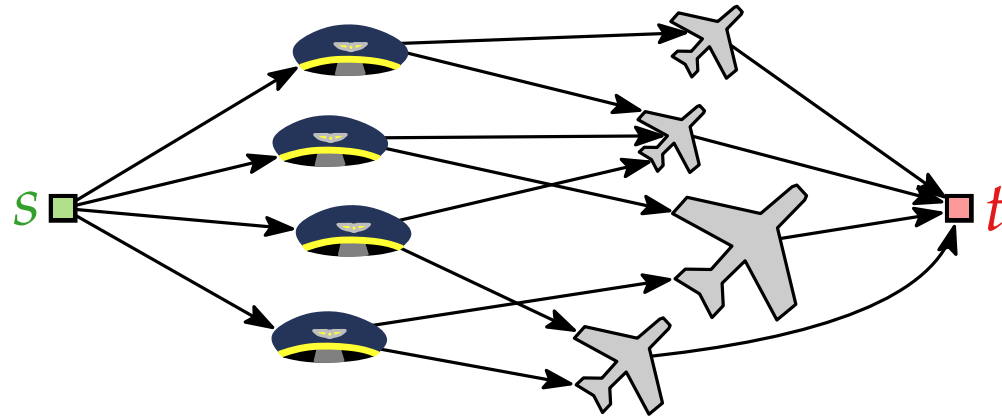
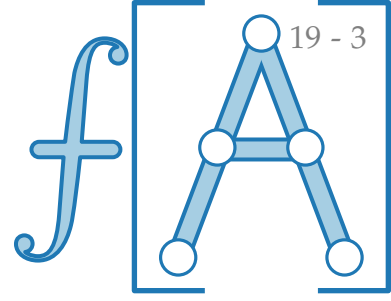
$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

Satz. $G = (\mathbb{M} \cup \mathbb{F}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis.
 „ $\Leftarrow$ “

Beweis des Heiratssatzes

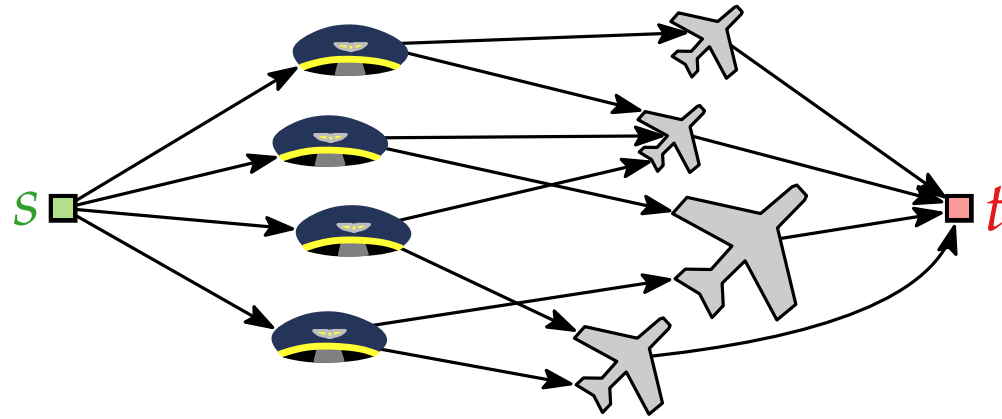
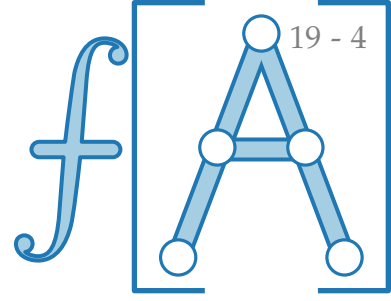


$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

Satz. $G = (\text{smiley faces} \cup \text{airplanes}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $S \subseteq \text{smiley faces}$ gilt $|S| \leq |N(S)|$.

Beweis. G hat eine perfekte Paarung
 $\Leftrightarrow G'$ hat Fluss f mit $|f| = |\text{smiley faces}| = |\text{airplanes}|$

Beweis des Heiratssatzes

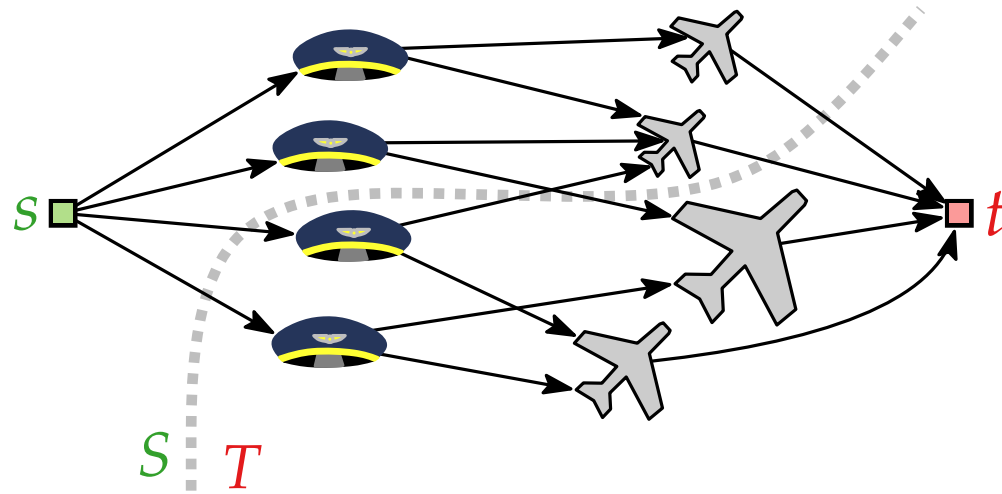
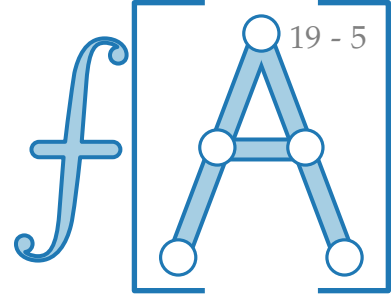


$$\begin{aligned} G &\rightarrow G' \\ V &\rightarrow V' = V \cup \{s, t\} \\ c: E' &\rightarrow \{1\} \end{aligned}$$

Satz. $G = (\text{♂} \cup \text{♀}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\text{♂} \subseteq \text{♂}$ gilt $|\text{♂}| \leq |N(\text{♂})|$.

Beweis. G hat eine perfekte Paarung
 $\Leftarrow$ G' hat Fluss f mit $|f| = |\text{♂}| = |\text{♀}|$
 $\Leftarrow$ für jeden s - t -Schnitt (S, T) in G' gilt $c(S) \geq |\text{♂}|$

Beweis des Heiratssatzes

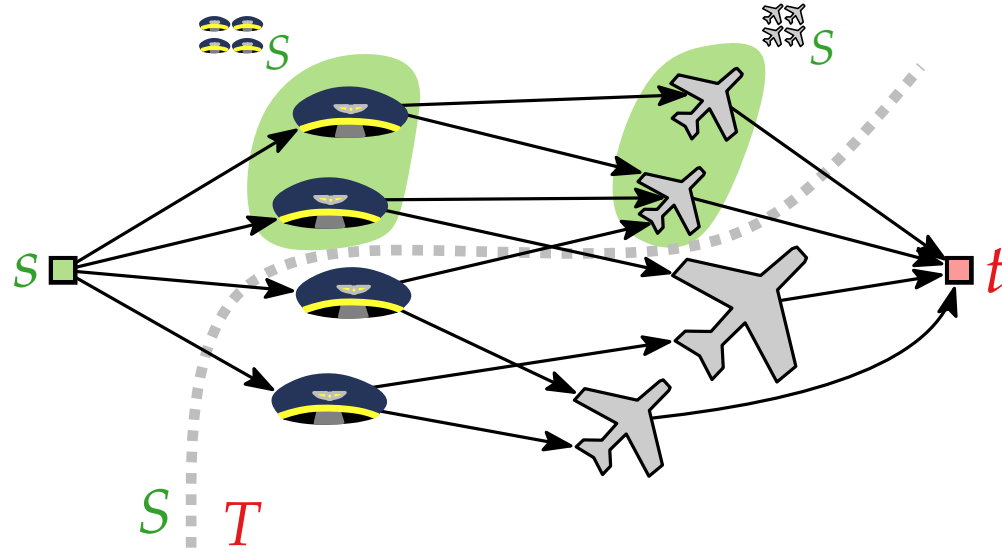
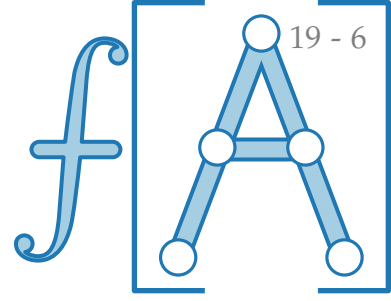


$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

Satz. $G = (\text{Personen} \cup \text{Platze}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $S \subseteq \text{Personen}$ gilt $|S| \leq |N(S)|$.

Beweis. G hat eine perfekte Paarung $\Leftrightarrow G'$ hat Fluss f mit $|f| = |\text{Personen}| = |\text{Platze}|$
 $\Leftrightarrow$ für jeden s - t -Schnitt (S, T) in G' gilt $c(S) \geq |\text{Personen}|$

Beweis des Heiratssatzes

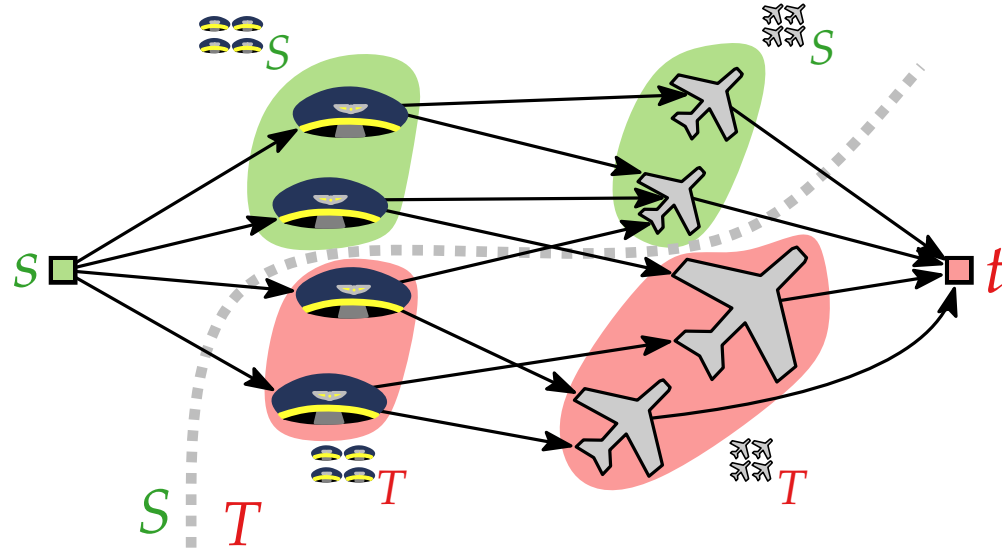
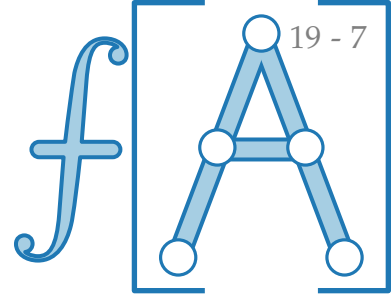


$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

Satz. $G = (\mathcal{A} \cup \mathcal{B}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\mathcal{A}' \subseteq \mathcal{A}$ gilt $|\mathcal{A}'| \leq |N(\mathcal{A}')|$.

Beweis. G hat eine perfekte Paarung
 $\Leftarrow$ G' hat Fluss f mit $|f| = |\mathcal{A}| = |\mathcal{B}|$
 $\Leftarrow$ für jeden s - t -Schnitt (S, T) in G' gilt $c(S) \geq |\mathcal{A}|$

Beweis des Heiratssatzes

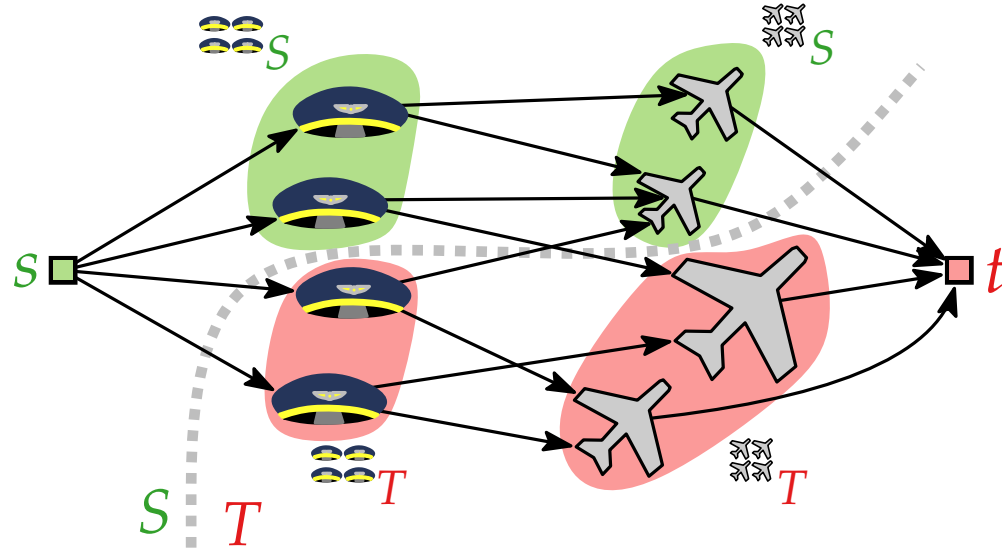
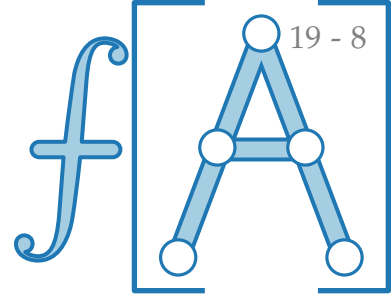


$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

Satz. $G = (\text{🛩} \cup \text{🚗}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\text{🚗} \subseteq \text{🛩}$ gilt $|\text{🚗}| \leq |N(\text{🚗})|$.

Beweis. G hat eine perfekte Paarung
 $\Leftarrow$ G' hat Fluss f mit $|f| = |\text{🛩}| = |\text{🚗}|$
 $\Leftarrow$ für jeden s - t -Schnitt (S, T) in G' gilt $c(S) \geq |\text{🛩}|$

Beweis des Heiratssatzes



$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

Satz. $G = (\text{🚗} \cup \text{✈️}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\text{🚗} \subseteq \text{🚗}$ gilt $|\text{🚗}| \leq |N(\text{🚗})|$.

Beweis.
 „ $\Leftarrow$ “

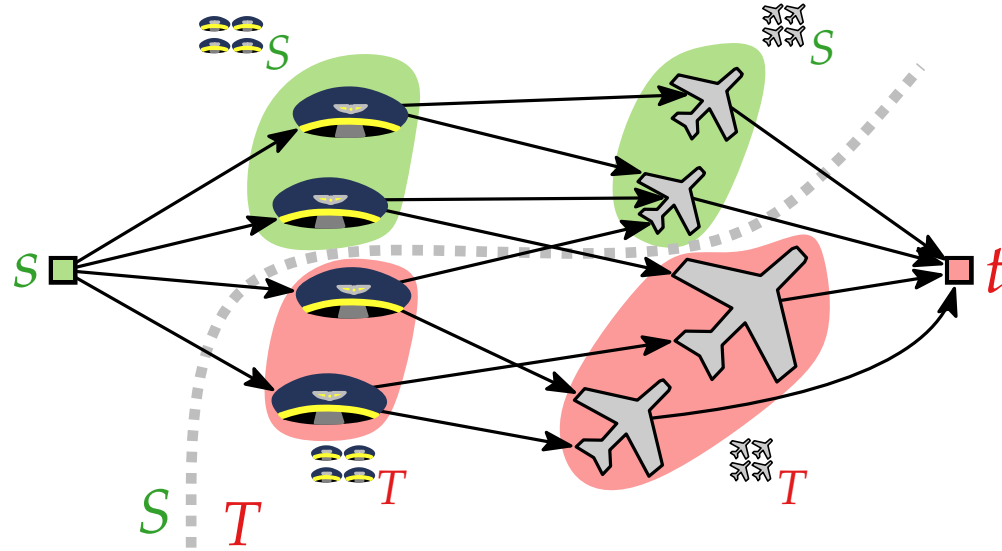
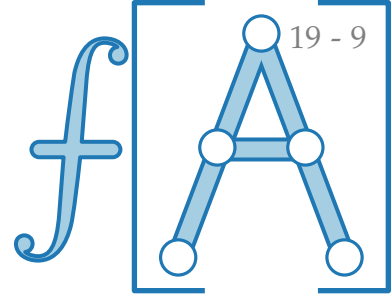
G hat eine perfekte Paarung

$\Leftarrow G'$ hat Fluss f mit $|f| = |\text{🚗}| = |\text{✈️}|$

$\Leftarrow$ für jeden s - t -Schnitt (S, T) in G' gilt $c(S) \geq |\text{🚗}|$

zu Zeigen!

Beweis des Heiratssatzes

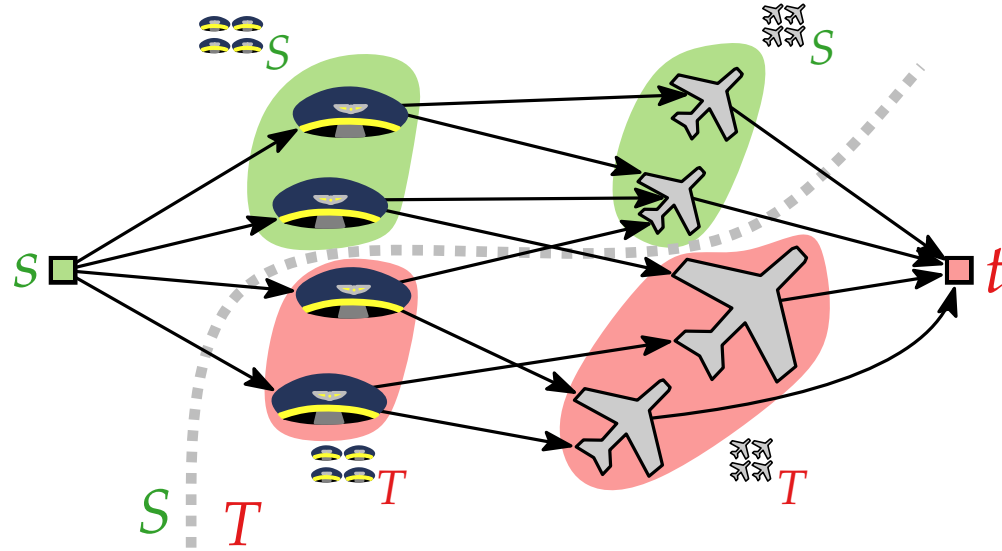
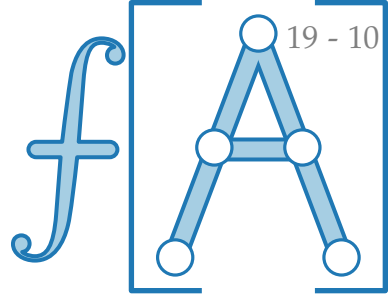


$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

Satz. $G = (\mathbb{M} \cup \mathbb{W}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathbb{M}|$
 „ $\Leftarrow$ “

Beweis des Heiratssatzes

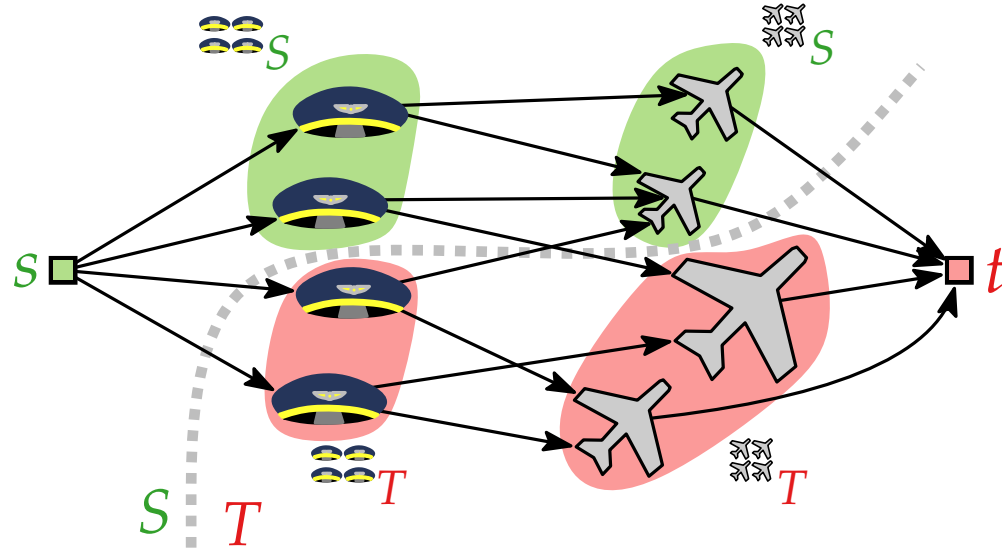


$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

Satz. $G = (\mathbb{M} \cup \mathbb{F}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathbb{M}|$
 „ $\Leftarrow$ “
 Es gilt $c(S) =$

Beweis des Heiratssatzes



$$G \rightarrow G'$$

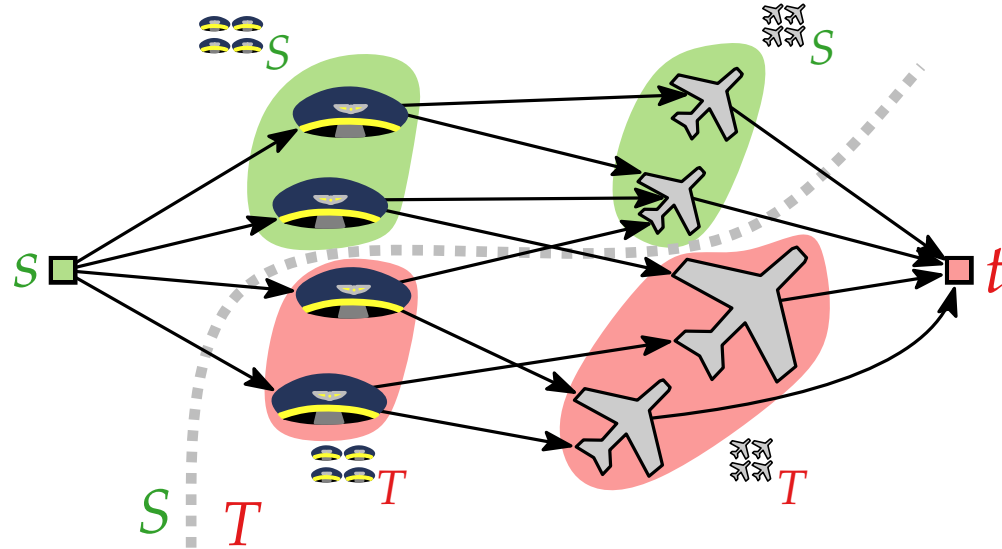
$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

Satz. $G = (\text{♂} \cup \text{♀}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\text{♂} \subseteq \text{♂}$ gilt $|\text{♂}| \leq |N(\text{♂})|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\text{♂}|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) =$

Beweis des Heiratssatzes



$$G \rightarrow G'$$

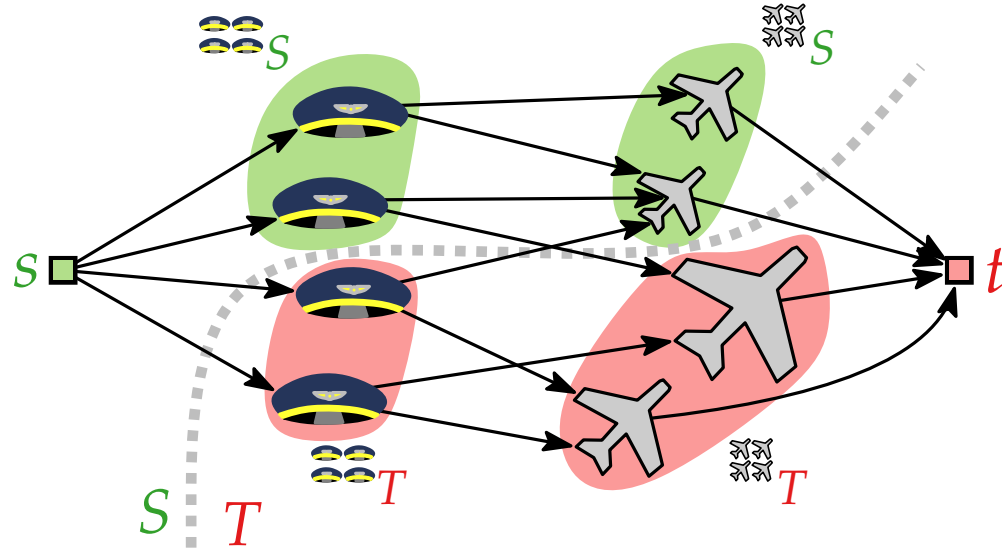
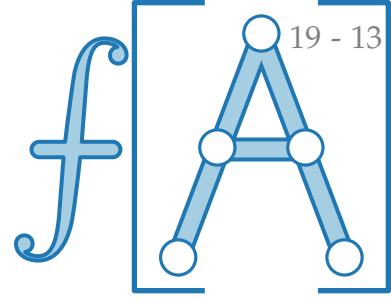
$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

Satz. $G = (\mathbb{A} \cup \mathbb{B}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathbb{A}' \subseteq \mathbb{A}$ gilt $|\mathbb{A}'| \leq |N(\mathbb{A}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathbb{A}|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e)$

Beweis des Heiratssatzes



$$G \rightarrow G'$$

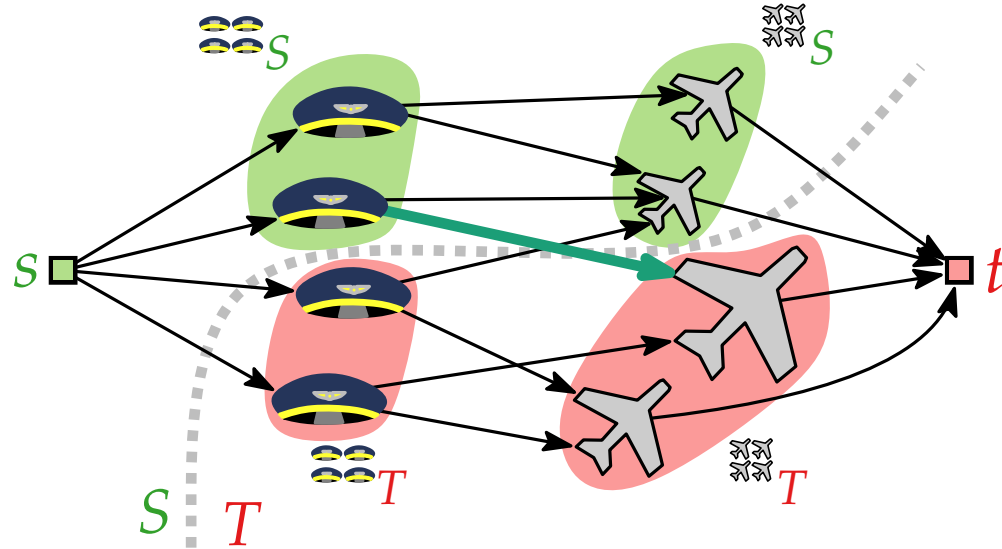
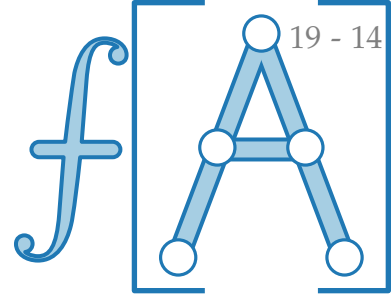
$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

Satz. $G = (\mathbb{M} \cup \mathbb{F}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathbb{M}|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)|$

Beweis des Heiratssatzes



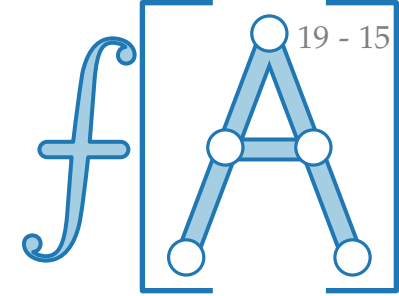
$$G \rightarrow G'$$

$$V \rightarrow V' = V \cup \{s, t\}$$

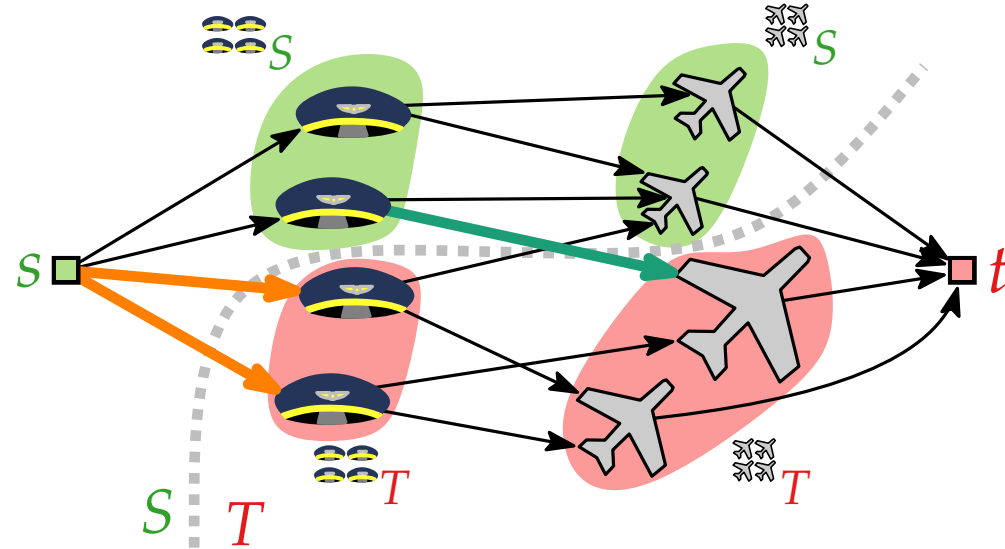
$$c: E' \rightarrow \{1\}$$

Satz. $G = (\mathbb{M} \cup \mathbb{F}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathbb{M}|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)|$
 $\geq |N_G(\mathbb{M}_S) \cap \mathbb{F}_T| +$



Beweis des Heiratssatzes



$$G \rightarrow G'$$

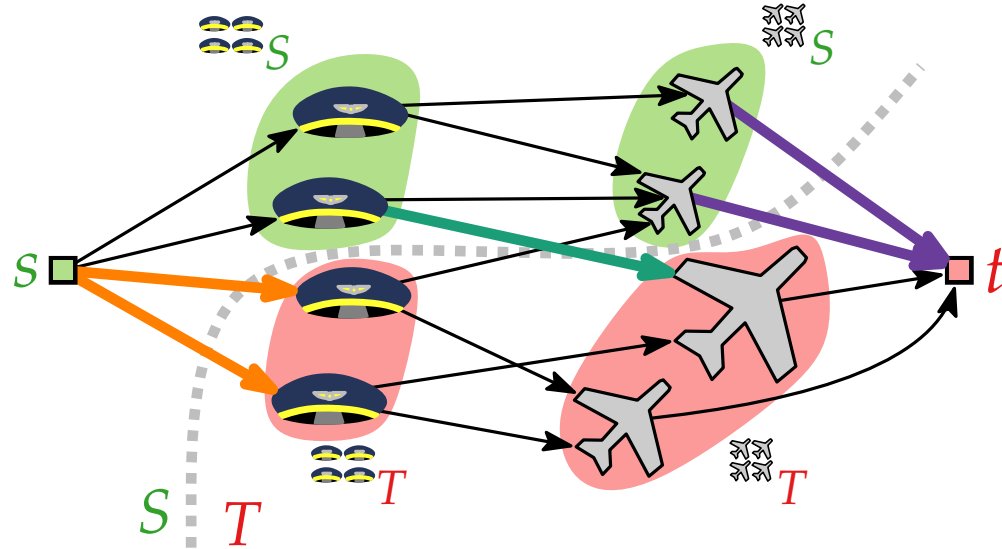
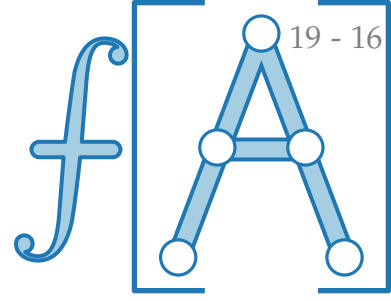
$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

Satz. $G = (\mathbb{M} \cup \mathbb{W}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathbb{M}|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)|$
 $\geq |\mathbb{M}_T| + |N_G(\mathbb{M}_S) \cap \mathbb{W}_T| +$

Beweis des Heiratssatzes



$$G \rightarrow G'$$

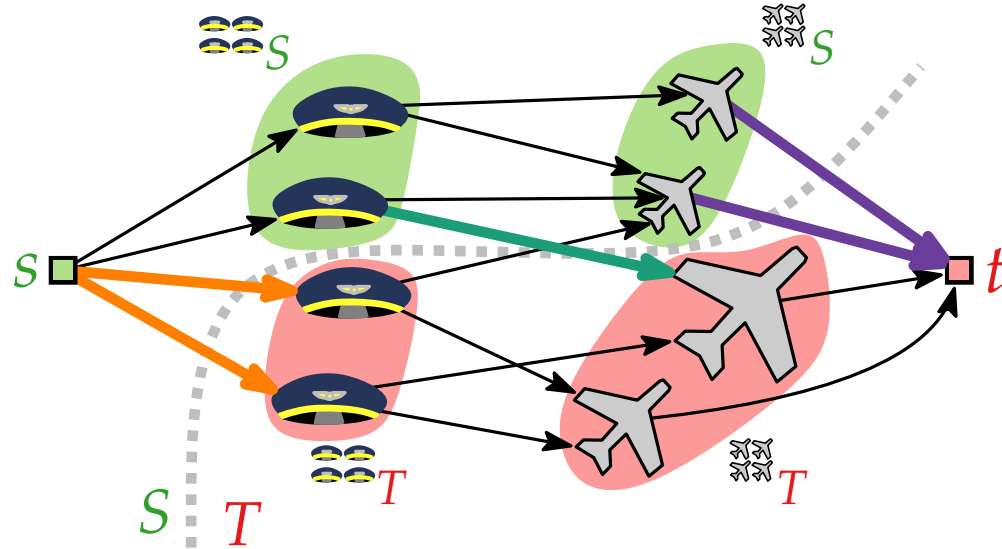
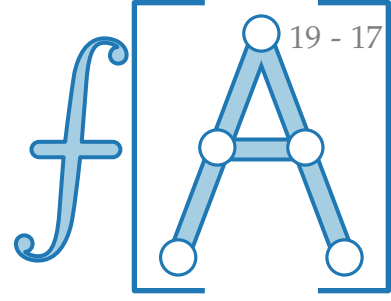
$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

Satz. $G = (\mathbb{M} \cup \mathbb{F}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathbb{M}|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)|$
 $\geq |\mathbb{M}_T| + |N_G(\mathbb{M}_S) \cap \mathbb{F}_T| + |\mathbb{F}_S|$

Beweis des Heiratssatzes



$$G \rightarrow G'$$

$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

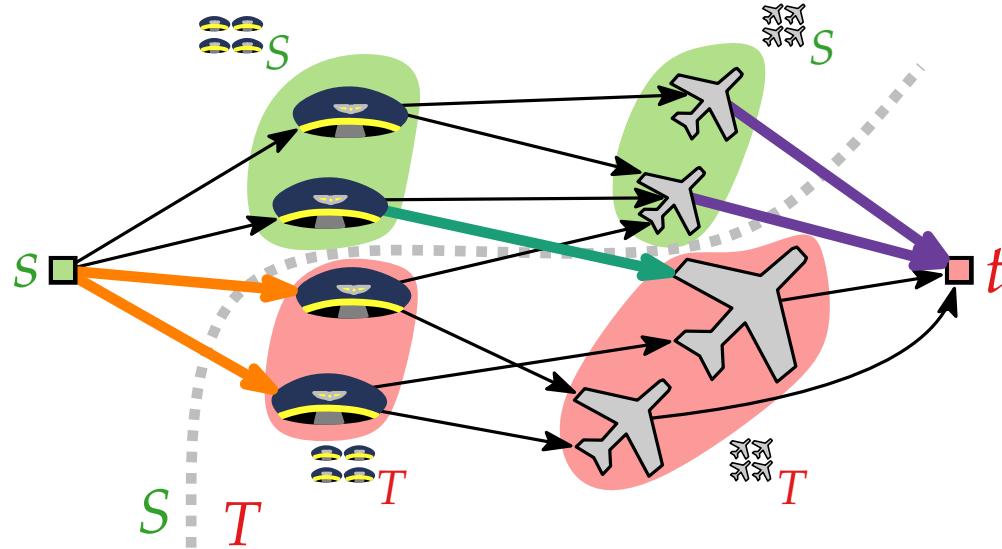
Satz. $G = (\mathbb{M} \cup \mathbb{W}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathbb{M}|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)|$

$$\geq |\mathbb{M}_T| + |N_G(\mathbb{M}_S) \cap \mathbb{W}_T| + |\mathbb{W}_S|$$

$$\geq |\mathbb{M}_T| + |N_G(\mathbb{M}_S) \cap \mathbb{W}_T| + |N_G(\mathbb{M}_S) \cap \mathbb{W}_S|$$

Beweis des Heiratssatzes



$$G \rightarrow G'$$

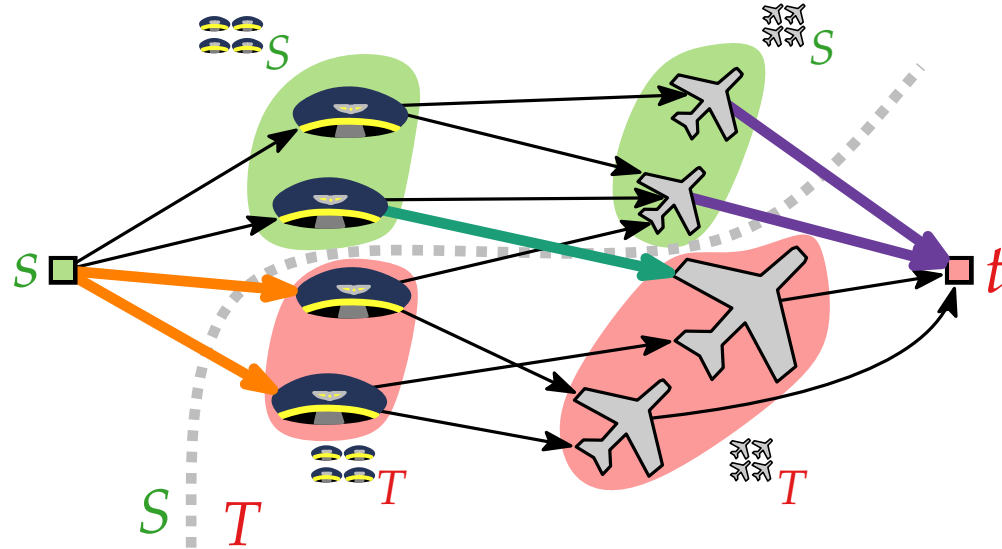
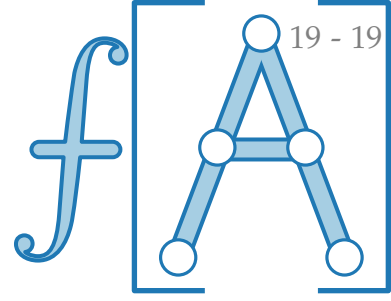
$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

Satz. $G = (\mathcal{A} \cup \mathcal{B}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathcal{A}' \subseteq \mathcal{A}$ gilt $|\mathcal{A}'| \leq |N(\mathcal{A}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathcal{A}'|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)|$
 $\geq |\mathcal{A}'_T| + |N_G(\mathcal{A}'_S) \cap \mathcal{B}_T| + |\mathcal{B}'_S|$
 $\geq |\mathcal{A}'_T| + |N_G(\mathcal{A}'_S) \cap \mathcal{B}_T| + |N_G(\mathcal{A}'_S) \cap \mathcal{B}'_S|$
 $= |\mathcal{A}'_T| + |N_G(\mathcal{A}'_S) \cap \mathcal{B}|$

Beweis des Heiratssatzes



$$G \rightarrow G'$$

$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

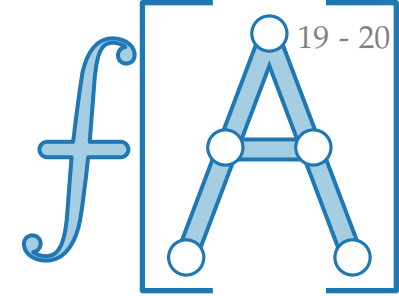
Satz. $G = (\mathcal{C} \cup \mathcal{A}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathcal{C}' \subseteq \mathcal{C}$ gilt $|\mathcal{C}'| \leq |N(\mathcal{C}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathcal{C}|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)|$

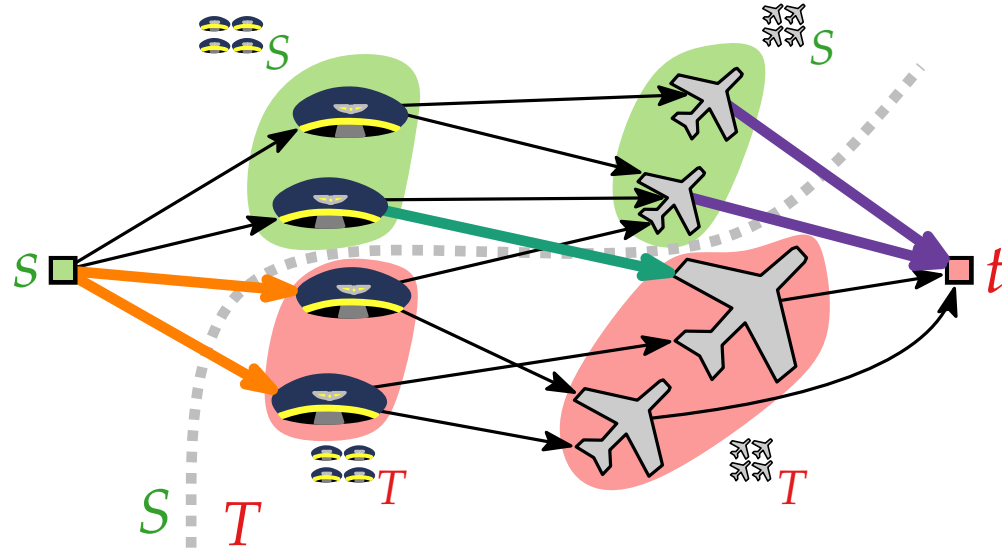
$$\geq |\mathcal{C}_T| + |N_G(\mathcal{C}_S) \cap \mathcal{A}_T| + |\mathcal{A}_S|$$

$$\geq |\mathcal{C}_T| + |N_G(\mathcal{C}_S) \cap \mathcal{A}_T| + |N_G(\mathcal{C}_S) \cap \mathcal{A}_S|$$

$$= |\mathcal{C}_T| + |N_G(\mathcal{C}_S) \cap \mathcal{A}| = |\mathcal{C}_T| + |N_G(\mathcal{C}_S)|$$



Beweis des Heiratssatzes



$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

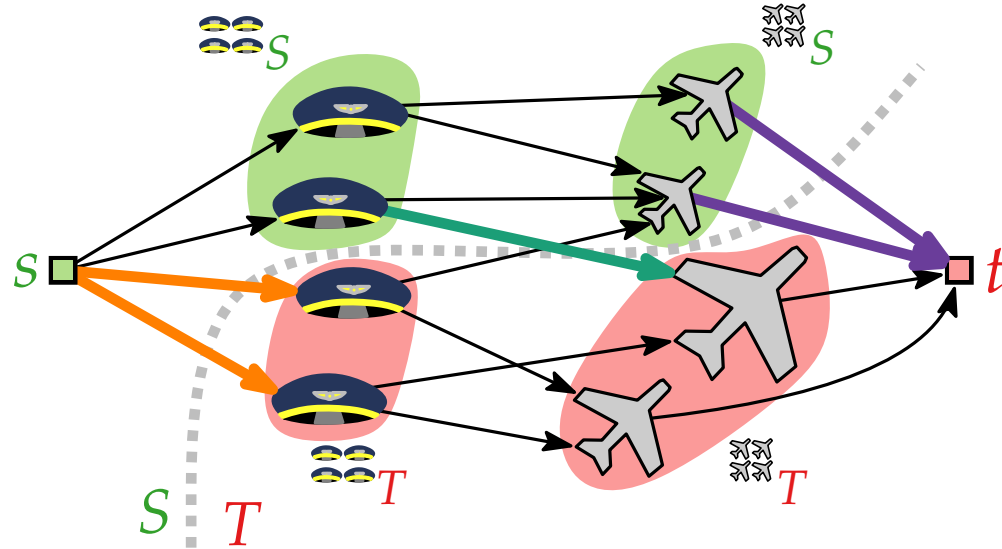
Satz. $G = (\mathcal{M} \cup \mathcal{W}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathcal{M}' \subseteq \mathcal{M}$ gilt $|\mathcal{M}'| \leq |N(\mathcal{M}')|$.

Beweis.
 „ $\Leftarrow$ “

$$\begin{aligned}
 \text{z.Z.: } (S, T) \text{ s-t-Schnitt in } G' &\Rightarrow c(S) \geq |\mathcal{M}| \\
 \text{Es gilt } c(S) &= c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)| \\
 &\geq |\mathcal{M}_T| + |N_G(\mathcal{M}_S) \cap \mathcal{W}_T| + |\mathcal{W}_S| \\
 &\geq |\mathcal{M}_T| + |N_G(\mathcal{M}_S) \cap \mathcal{W}_T| + |N_G(\mathcal{M}_S) \cap \mathcal{W}_S| \\
 &= |\mathcal{M}_T| + |N_G(\mathcal{M}_S) \cap \mathcal{W}| = |\mathcal{M}_T| + |N_G(\mathcal{M}_S)|
 \end{aligned}$$

$N_G(\mathcal{M}_S) \subseteq \mathcal{W}$

Beweis des Heiratssatzes



$$G \rightarrow G'$$

$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

Satz. $G = (\mathbb{M} \cup \mathbb{W}, E)$ bipartit hat eine perfekte Paarung $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis. z.Z.: (S, T) s - t -Schnitt in $G' \Rightarrow c(S) \geq |\mathbb{M}'|$
 „ $\Leftarrow$ “
 Es gilt $c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)|$

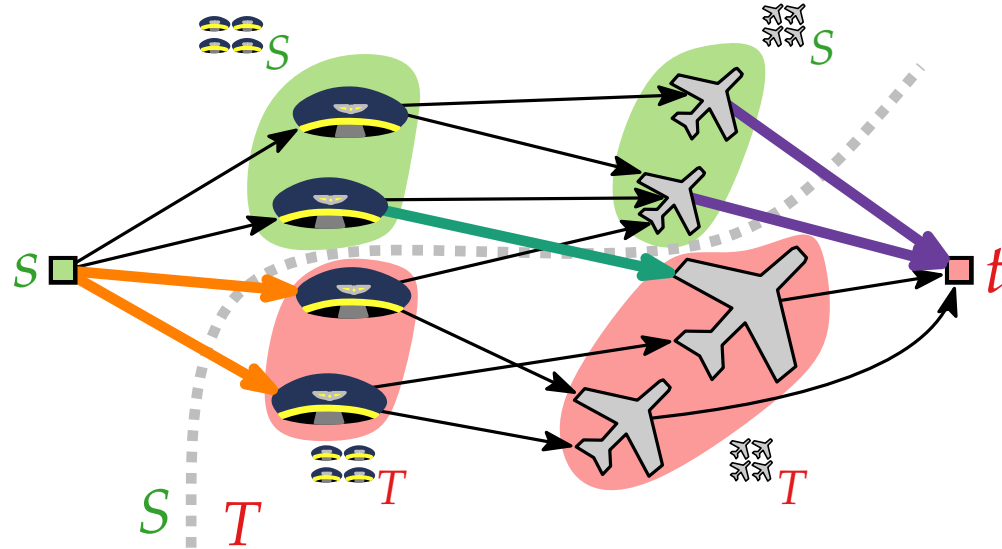
$$\geq |\mathbb{M}'_T| + |N_G(\mathbb{M}'_S) \cap \mathbb{W}_T| + |\mathbb{W}'_S|$$

$$\geq |\mathbb{M}'_T| + |N_G(\mathbb{M}'_S) \cap \mathbb{W}_T| + |N_G(\mathbb{M}'_S) \cap \mathbb{W}'_S|$$

$$= |\mathbb{M}'_T| + |N_G(\mathbb{M}'_S) \cap \mathbb{W}| = |\mathbb{M}'_T| + |N_G(\mathbb{M}'_S)|$$

$$\geq |\mathbb{M}'_T| + |\mathbb{M}'_S|$$

Beweis des Heiratssatzes



$$G \rightarrow G'$$

$$V \rightarrow V' = V \cup \{s, t\}$$

$$c: E' \rightarrow \{1\}$$

Satz. $G = (\mathbb{M} \cup \mathbb{W}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\mathbb{M}' \subseteq \mathbb{M}$ gilt $|\mathbb{M}'| \leq |N(\mathbb{M}')|$.

Beweis.
 „ $\Leftarrow$ “

$$\text{z.Z.: } (S, T) \text{ s-t-Schnitt in } G' \Rightarrow c(S) \geq |\mathbb{M}|$$

$$\text{Es gilt } c(S) = c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)|$$

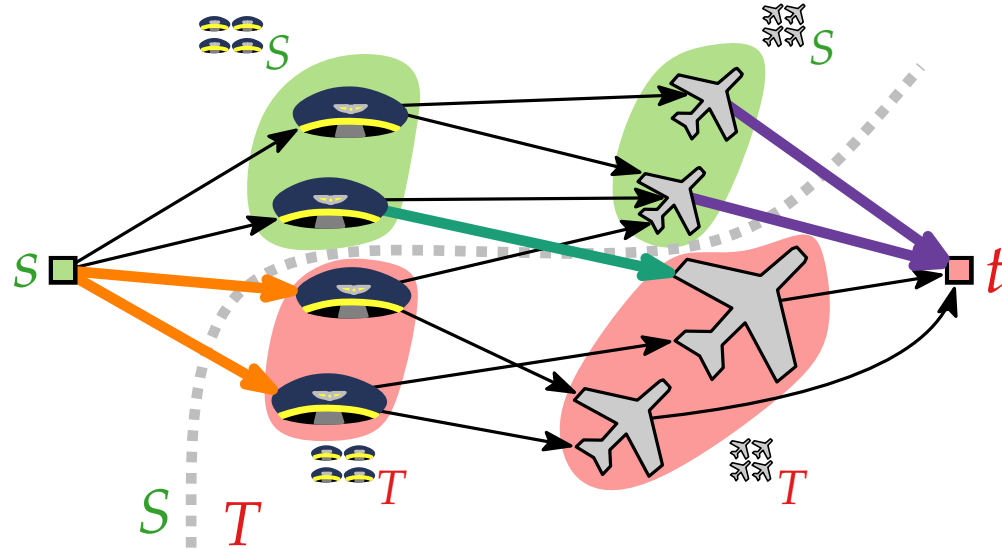
$$\geq |\mathbb{M}_T| + |N_G(\mathbb{M}_S) \cap \mathbb{W}_T| + |\mathbb{W}_S|$$

$$\geq |\mathbb{M}_T| + |N_G(\mathbb{M}_S) \cap \mathbb{W}_T| + |N_G(\mathbb{M}_S) \cap \mathbb{W}_S|$$

$$= |\mathbb{M}_T| + |N_G(\mathbb{M}_S) \cap \mathbb{W}| = |\mathbb{M}_T| + |N_G(\mathbb{M}_S)|$$

$$\geq |\mathbb{M}_T| + |\mathbb{M}_S|$$

Beweis des Heiratssatzes

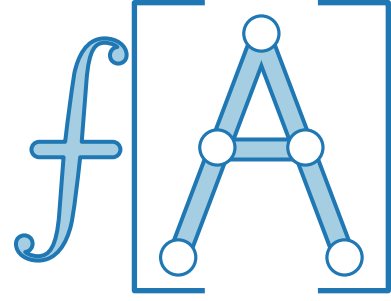


$$\begin{aligned}
 G &\rightarrow G' \\
 V &\rightarrow V' = V \cup \{s, t\} \\
 c: E' &\rightarrow \{1\}
 \end{aligned}$$

Satz. $G = (\mathcal{C} \cup \mathcal{A}, E)$ bipartit hat eine perfekte Paarung
 $\Leftrightarrow$ für jedes $\mathcal{C}' \subseteq \mathcal{C}$ gilt $|\mathcal{C}'| \leq |N(\mathcal{C}')|$.

Beweis.
 „ $\Leftarrow$ “

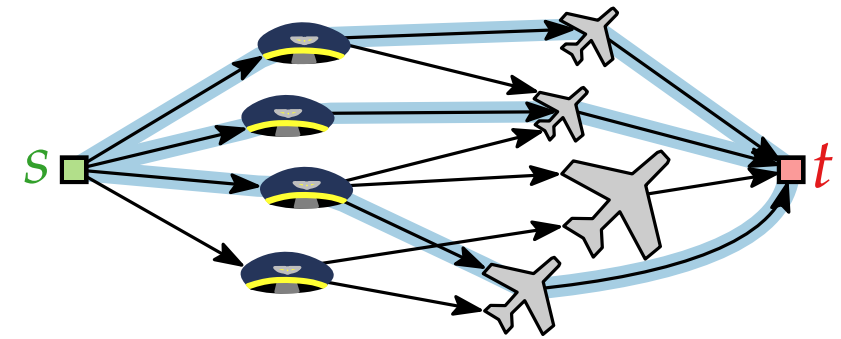
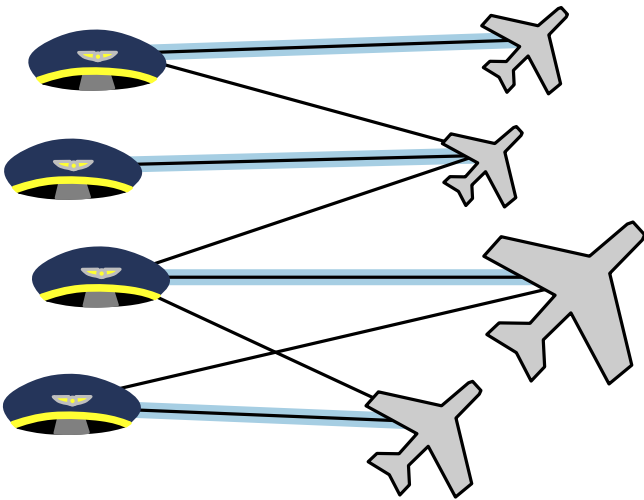
$$\begin{aligned}
 \text{z.Z.: } (S, T) \text{ s-t-Schnitt in } G' &\Rightarrow c(S) \geq |\mathcal{C}| \\
 \text{Es gilt } c(S) &= c(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} c(e) = |\text{Raus}(S)| \\
 &\geq |\mathcal{C}_T| + |N_G(\mathcal{C}_S) \cap \mathcal{A}_T| + |\mathcal{A}_S| \\
 &\geq |\mathcal{C}_T| + |N_G(\mathcal{C}_S) \cap \mathcal{A}_T| + |N_G(\mathcal{C}_S) \cap \mathcal{A}_S| \\
 &= |\mathcal{C}_T| + |N_G(\mathcal{C}_S) \cap \mathcal{A}| = |\mathcal{C}_T| + |N_G(\mathcal{C}_S)| \\
 &\geq |\mathcal{C}_T| + |\mathcal{C}_S| = |\mathcal{C}| \quad \square
 \end{aligned}$$



Fortgeschrittene Algorithmen

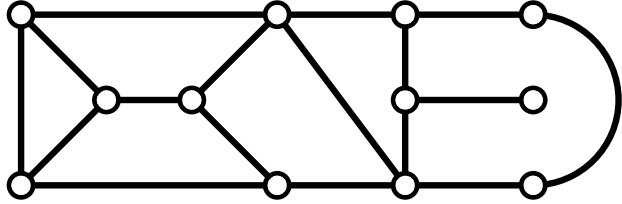
9. Vorlesung Graphalgorithmen III: Paarungen / Matchings

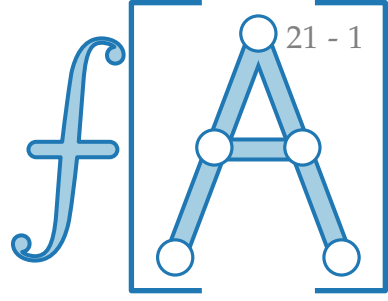
Teil V: Algorithmen ohne Flüsse



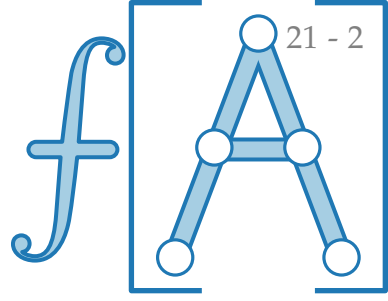
Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

Bsp.  $G = (V, E)$ unger. Graph

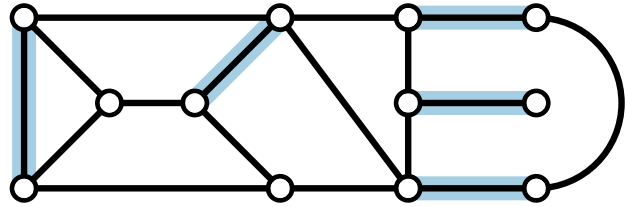


Alternierende und augmentierende Wege



Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

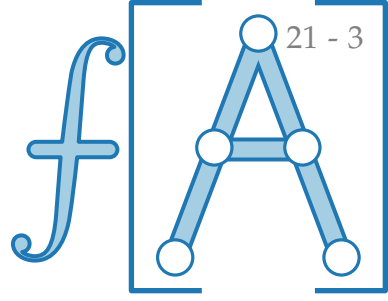
Bsp.



$G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

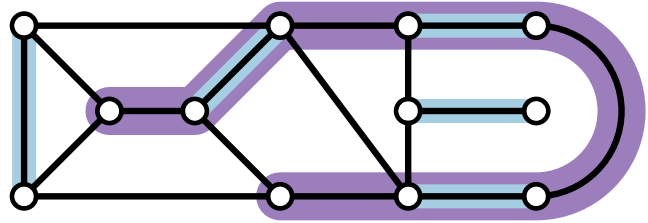
Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

Alternierende und augmentierende Wege



Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

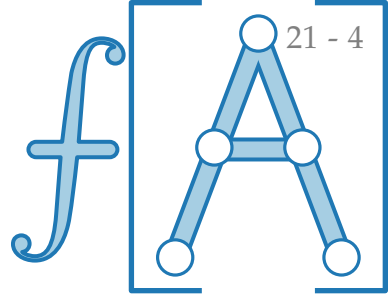
Bsp.



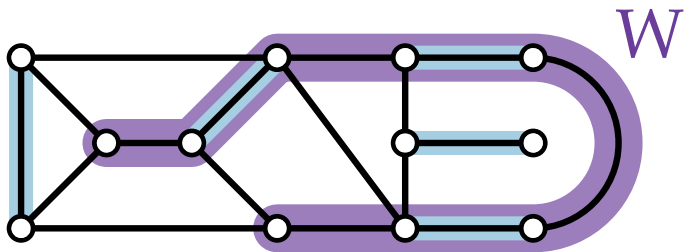
$G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

Alternierende und augmentierende Wege



Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

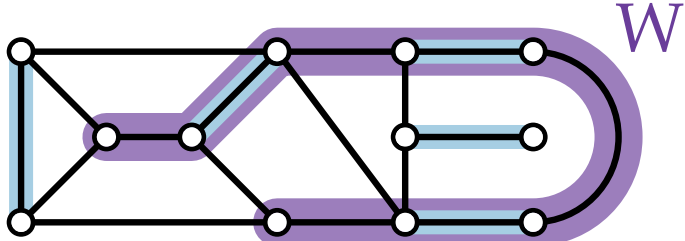
Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

- Finde einen (P -)alternierenden Weg W ,

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

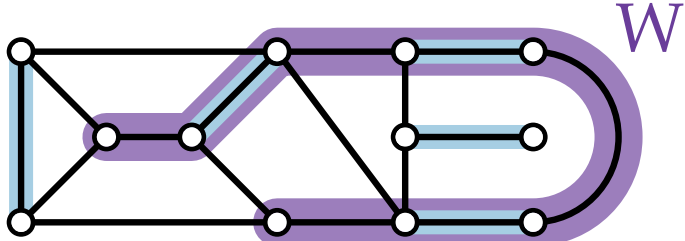
Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

enthält abwechselnd P - und nicht- P -Kanten

- Finde einen (P -)alternierenden Weg W ,

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

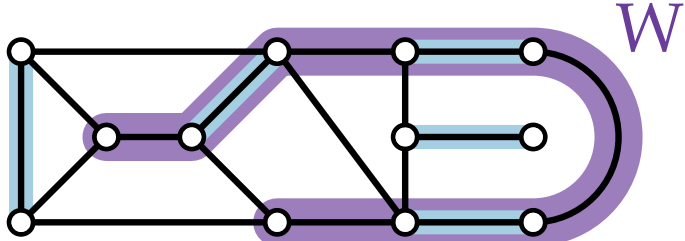
Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

enthält abwechselnd P - und nicht- P -Kanten

- Finde einen (P -)alternierenden Weg W , dessen Endknoten P -frei sind.

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

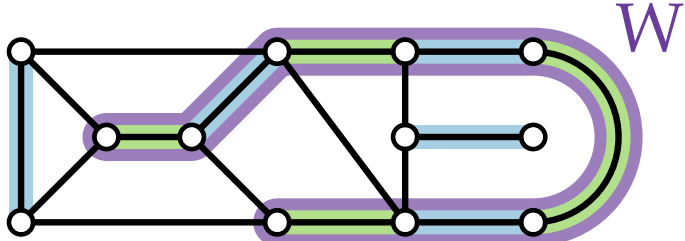
Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

enthält abwechselnd P - und nicht- P -Kanten

- Finde einen (P -)**alternierenden Weg** W , dessen Endknoten P -frei sind.
 So ein Weg heißt (P -)**augmentierend**.

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

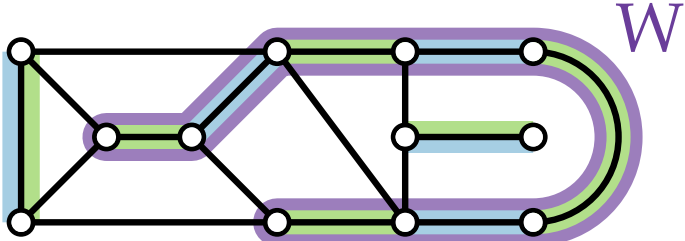
Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

enthält abwechselnd P - und nicht- P -Kanten

- Finde einen (P -)**alternierenden Weg** W , dessen Endknoten P -frei sind.
 So ein Weg heißt (P -)**augmentierend**.

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

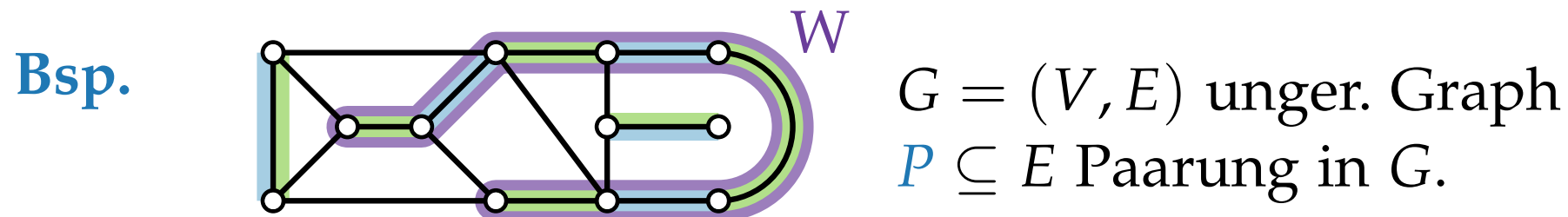
Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

enthält abwechselnd P - und nicht- P -Kanten

- Finde einen (P -)**alternierenden Weg** W , dessen Endknoten **P -frei** sind.
 So ein Weg heißt (P -)**augmentierend**.

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

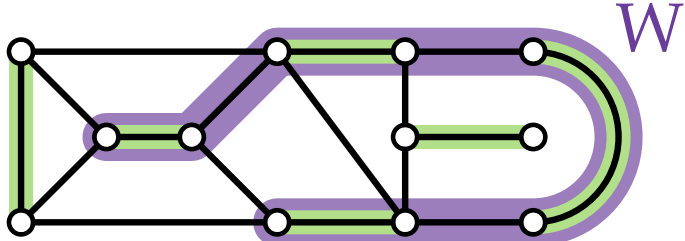


Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

- Finde einen (P) -**alternierenden Weg** W , dessen Endknoten P -**frei** sind.
 So ein Weg heißt (P) -**augmentierend**.
 enthält abwechselnd P - und nicht- P -Kanten
- Setze $P' = W \Delta P$ $A \Delta B = (A \cup B) \setminus (A \cap B)$ ist die *symm. Differenz* von A und B

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

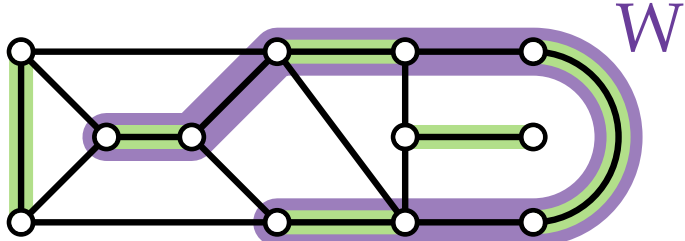
Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

- Finde einen (P -)**alternierenden Weg** W , dessen Endknoten P -frei sind.
 So ein Weg heißt (P -)**augmentierend**.
 enthält abwechselnd P - und nicht- P -Kanten
- Setze $P' = W \Delta P$ $A \Delta B = (A \cup B) \setminus (A \cap B)$ ist die *symm. Differenz* von A und B

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

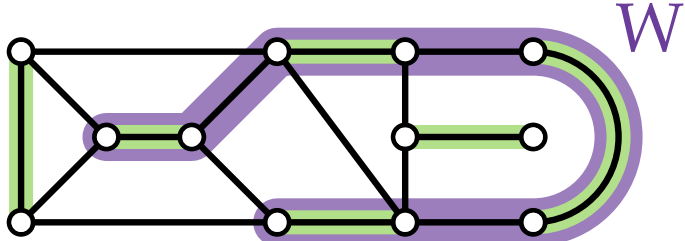
Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

- Finde einen (P) -**alternierenden Weg** W , dessen Endknoten P -**frei** sind.
 So ein Weg heißt (P) -**augmentierend**.
 enthält abwechselnd P - und nicht- P -Kanten
- Setze $P' = W \Delta P$ $A \Delta B = (A \cup B) \setminus (A \cap B)$ ist die *symm. Differenz* von A und B

Beob. ■ Augmentierende Wege haben ungerade Länge.

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

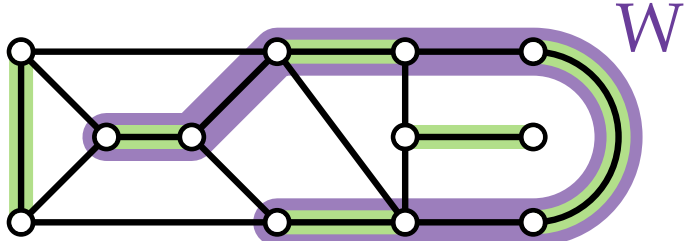
- Finde einen (P) -**alternierenden Weg** W , dessen Endknoten P -**frei** sind.
 So ein Weg heißt (P) -**augmentierend**.
 enthält abwechselnd P - und nicht- P -Kanten
- Setze $P' = W \Delta P$ $A \Delta B = (A \cup B) \setminus (A \cap B)$ ist die *symm. Differenz* von A und B

Beob.

- Augmentierende Wege haben ungerade Länge.
- P' ist um 1 Kante größer als P

Alternierende und augmentierende Wege

Ziel: Besseres Problemverständnis $\rightarrow$ kombinatorische (d.h. nicht flussbasierte) Algorithmen für größte Paarungen.

Bsp.  $G = (V, E)$ unger. Graph
 $P \subseteq E$ Paarung in G .

Wie können wir eine gegebene (nicht-größte) Paarung vergrößern?

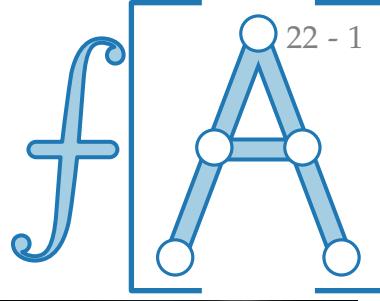
- Finde einen (P) -**alternierenden Weg** W , dessen Endknoten P -**frei** sind.
 So ein Weg heißt (P) -**augmentierend**.
 enthält abwechselnd P - und nicht- P -Kanten
- Setze $P' = W \Delta P$ $A \Delta B = (A \cup B) \setminus (A \cap B)$ ist die *symm. Differenz* von A und B

Beob.

- Augmentierende Wege haben ungerade Länge.
- P' ist um 1 Kante größer als P
 (da P' zusätzlich die Endknoten von W paart).

Satz von Berge

Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .

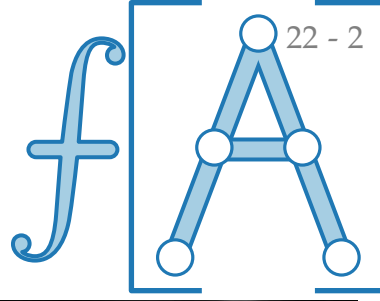


Claude Berge
*1926 Paris
†2002 Paris



Satz von Berge

Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.



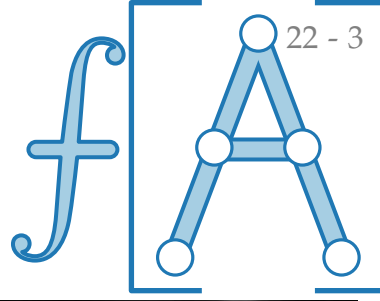
Claude Berge
*1926 Paris
†2002 Paris



Satz von Berge

Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar:



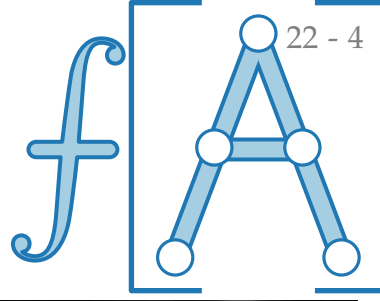
Claude Berge
*1926 Paris
†2002 Paris



Satz von Berge

Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡



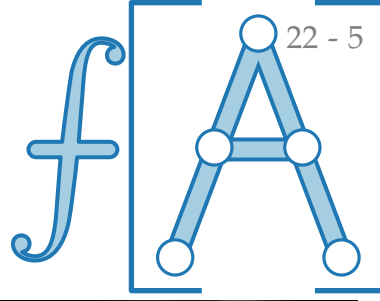
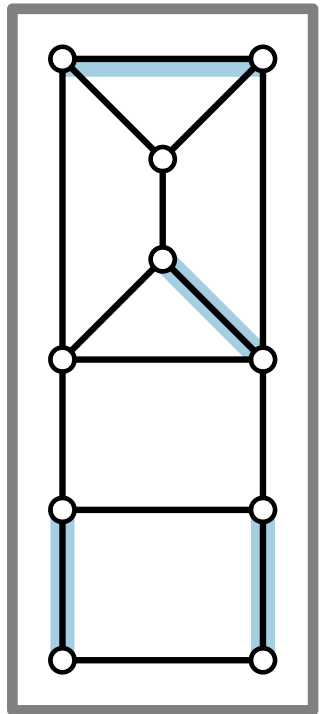
Claude Berge
*1926 Paris
†2002 Paris



Satz von Berge

Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

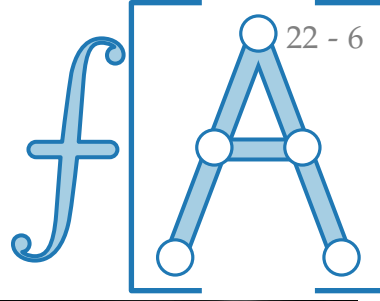
Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.



Claude Berge
*1926 Paris
†2002 Paris

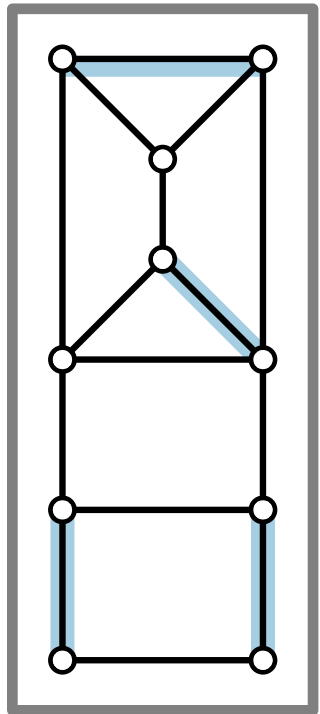


Satz von Berge



Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

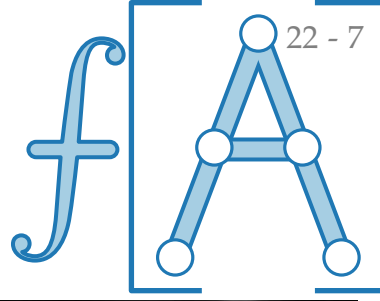
Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.
 $\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.



Claude Berge
*1926 Paris
†2002 Paris

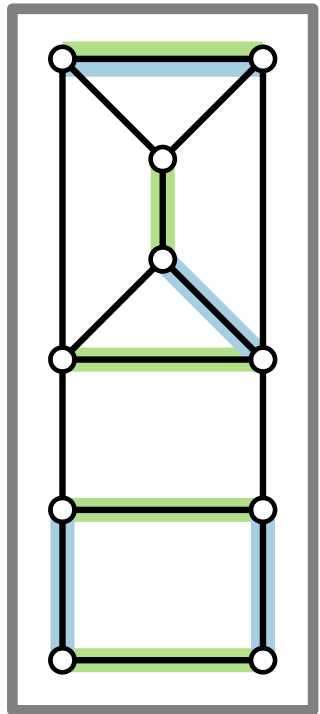


Satz von Berge



Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

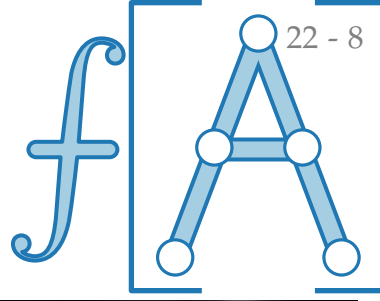
Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.
 $\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.



Claude Berge
*1926 Paris
†2002 Paris



Satz von Berge

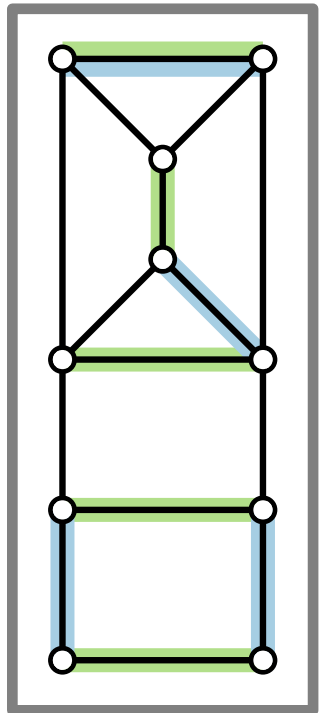


Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.

$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

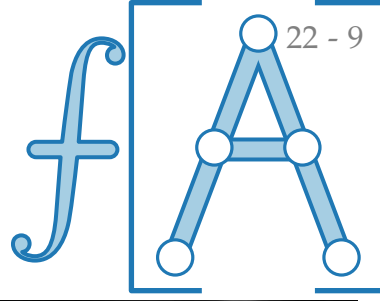
Betrachte $G_{\Delta} = (V, P \Delta P')$.



Claude Berge
*1926 Paris
†2002 Paris



Satz von Berge

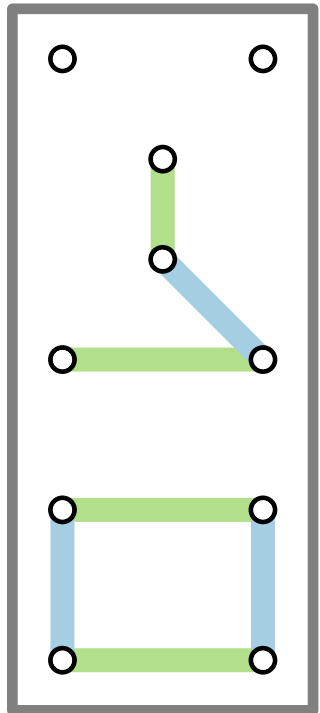


Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.

$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

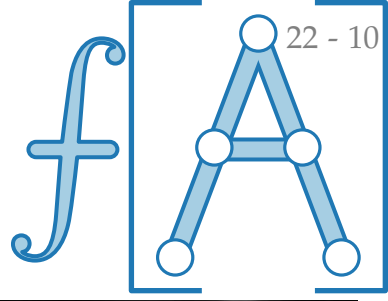
Betrachte $G_{\Delta} = (V, P \Delta P')$.



Claude Berge
*1926 Paris
†2002 Paris



Satz von Berge



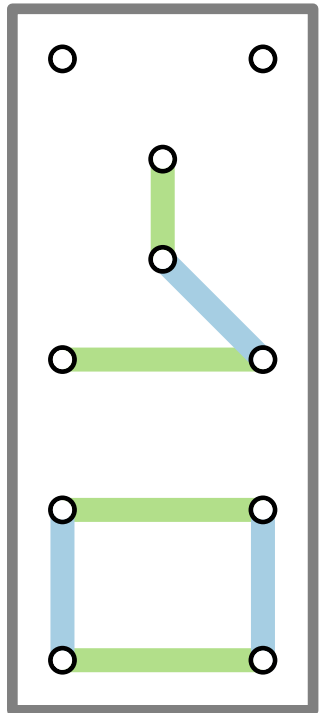
Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.

$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

Betrachte $G_{\Delta} = (V, P \Delta P')$.

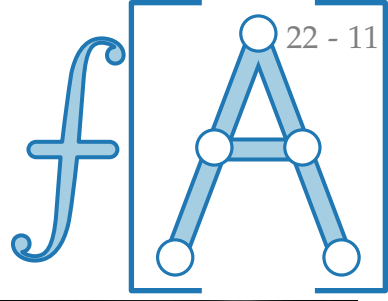
Beobachtung: $\max_{v \in V} \deg_{G_{\Delta}}(v) =$



Claude Berge
*1926 Paris
†2002 Paris



Satz von Berge



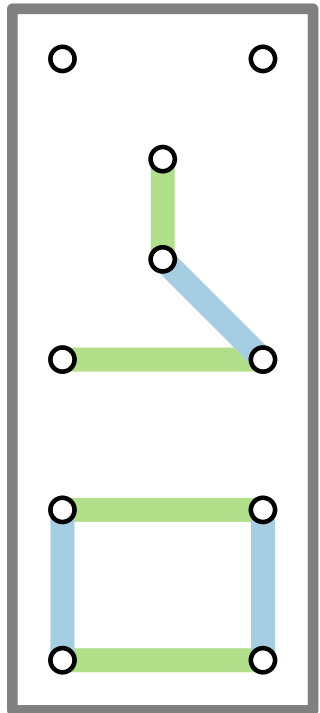
Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.

$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

Betrachte $G_{\Delta} = (V, P \Delta P')$.

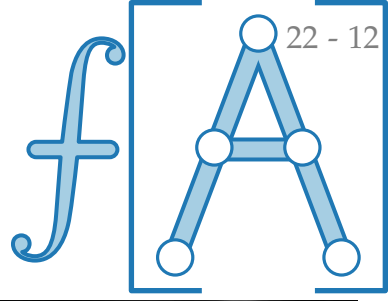
Beobachtung: $\max_{v \in V} \deg_{G_{\Delta}}(v) = 2$.



Claude Berge
*1926 Paris
†2002 Paris



Satz von Berge



Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

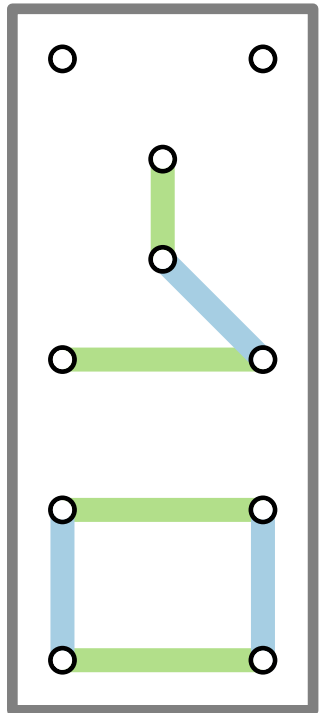
Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.

$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

Betrachte $G_{\Delta} = (V, P \Delta P')$.

Beobachtung: $\max_{v \in V} \deg_{G_{\Delta}}(v) = 2$.

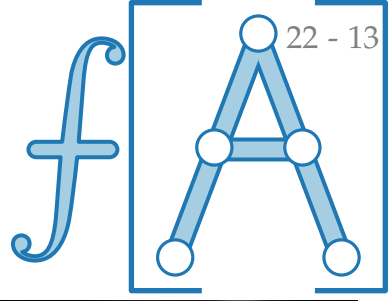
$\Rightarrow G_{\Delta}$ besteht aus einfachen alt. Wegen & Kreisen.



Claude Berge
*1926 Paris
†2002 Paris

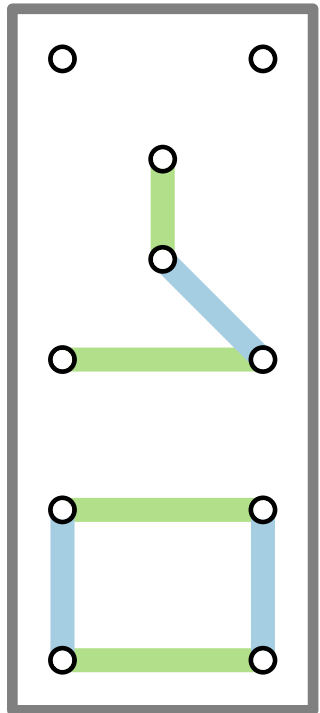


Satz von Berge



Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.



$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

Betrachte $G_{\Delta} = (V, P \Delta P')$.

Beobachtung: $\max_{v \in V} \deg_{G_{\Delta}}(v) = 2$.

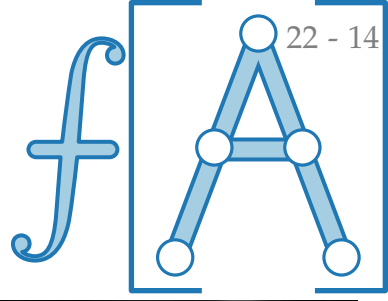
$\Rightarrow G_{\Delta}$ besteht aus einfachen alt. Wegen & Kreisen.

Alle Kreise in G_{Δ} haben gerade Länge.

Claude Berge
*1926 Paris
†2002 Paris

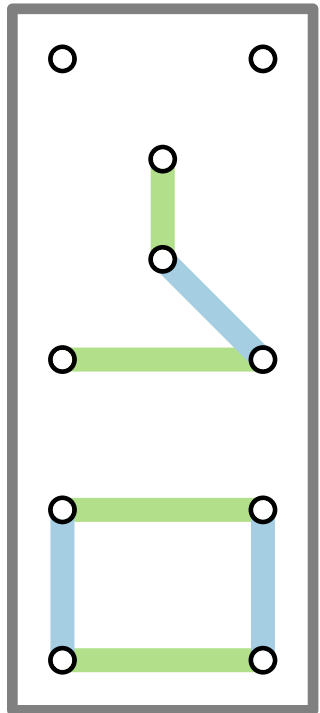


Satz von Berge



Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.



$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

Betrachte $G_{\Delta} = (V, P \Delta P')$.

Beobachtung: $\max_{v \in V} \deg_{G_{\Delta}}(v) = 2$.

$\Rightarrow G_{\Delta}$ besteht aus einfachen alt. Wegen & Kreisen.

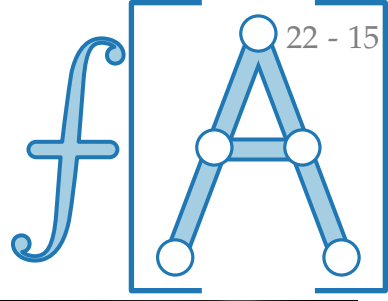
Alle Kreise in G_{Δ} haben gerade Länge.

$\Rightarrow$ Ex. alt. Weg mit mehr Kanten aus P' als aus P .

Claude Berge
*1926 Paris
†2002 Paris

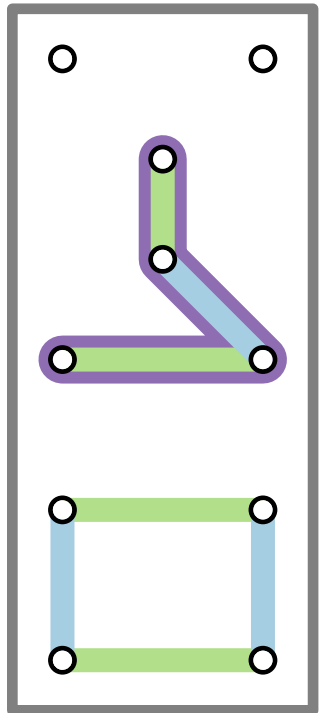


Satz von Berge



Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.



$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

Betrachte $G_{\Delta} = (V, P \Delta P')$.

Beobachtung: $\max_{v \in V} \deg_{G_{\Delta}}(v) = 2$.

$\Rightarrow G_{\Delta}$ besteht aus einfachen alt. Wegen & Kreisen.

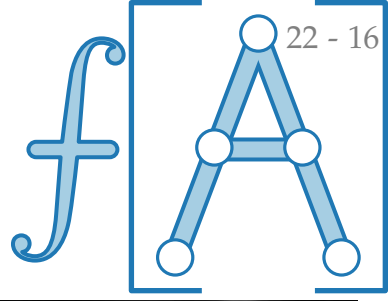
Alle Kreise in G_{Δ} haben gerade Länge.

$\Rightarrow$ Ex. alt. Weg mit mehr Kanten aus P' als aus P .

Claude Berge
*1926 Paris
†2002 Paris

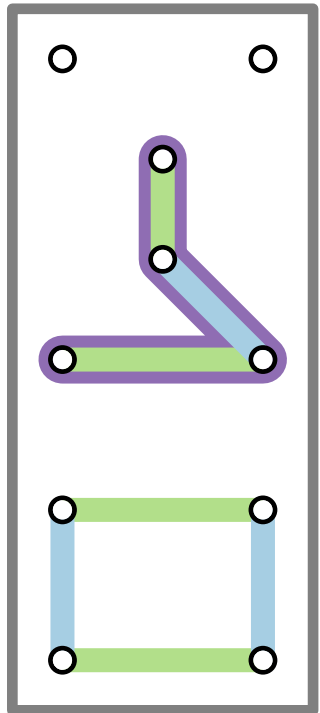


Satz von Berge



Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.



$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

Betrachte $G_\Delta = (V, P \Delta P')$.

Beobachtung: $\max_{v \in V} \deg_{G_\Delta}(v) = 2$.

$\Rightarrow G_\Delta$ besteht aus einfachen alt. Wegen & Kreisen.

Alle Kreise in G_Δ haben gerade Länge.

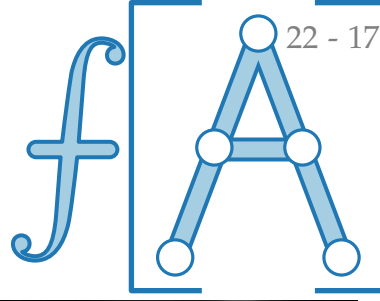
$\Rightarrow$ Ex. alt. Weg mit mehr Kanten aus P' als aus P .

$\Rightarrow$ Dieser Weg ist P -augm.

Claude Berge
*1926 Paris
†2002 Paris

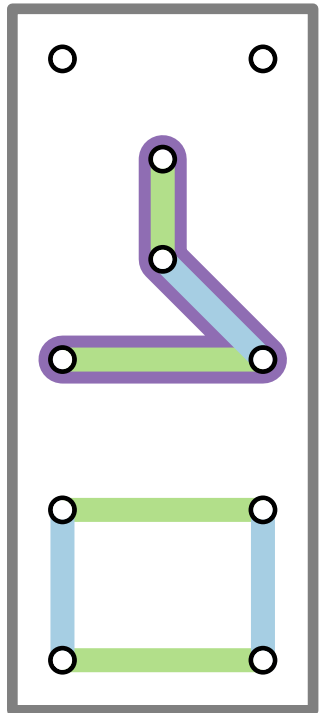


Satz von Berge



Satz. Sei $G = (V, E)$ Graph, $P \subseteq E$ Paarung in G .
 P ist eine größte Paarung in G
 $\Leftrightarrow$ es gibt keinen P -augmentierenden Weg.

Beweis. „ $\Rightarrow$ “ Klar: jeder augm. Weg würde P vergrößern ⚡
„ $\Leftarrow$ “ Annahme: P ist keine größte Paarung.



$\Rightarrow$ Es gibt eine Paarung P' mit $|P'| > |P|$.

Betrachte $G_{\Delta} = (V, P \Delta P')$.

Beobachtung: $\max_{v \in V} \deg_{G_{\Delta}}(v) = 2$.

$\Rightarrow G_{\Delta}$ besteht aus einfachen alt. Wegen & Kreisen.

Alle Kreise in G_{Δ} haben gerade Länge.

$\Rightarrow$ Ex. alt. Weg mit mehr Kanten aus P' als aus P .

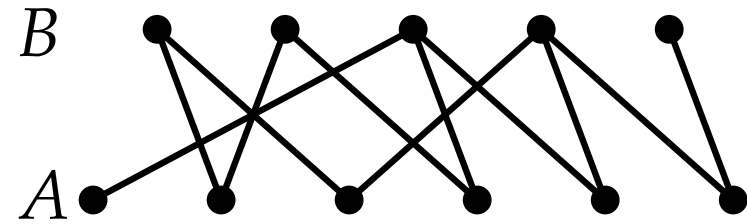
$\Rightarrow$ Dieser Weg ist P -augment. ⚡ zur Annahme. $\square$

Claude Berge
*1926 Paris
†2002 Paris

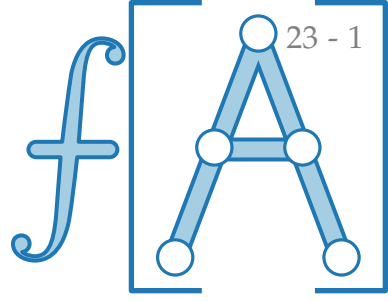


Zurück zu: größte Matchings in bip. Graphen

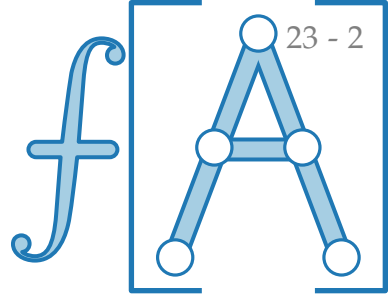
Frage: Wie finden wir augmentierende Wege?



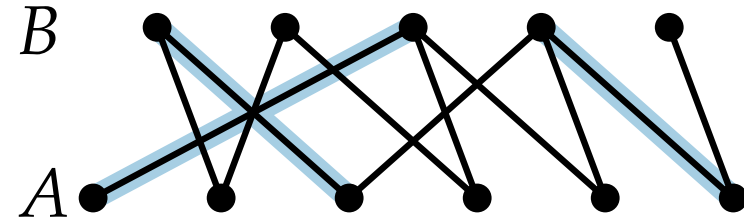
$G = (A \cup B, E)$ bipartit,



Zurück zu: größte Matchings in bip. Graphen

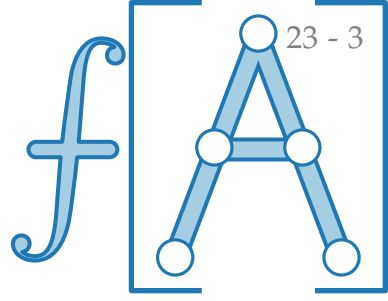


Frage: Wie finden wir augmentierende Wege?

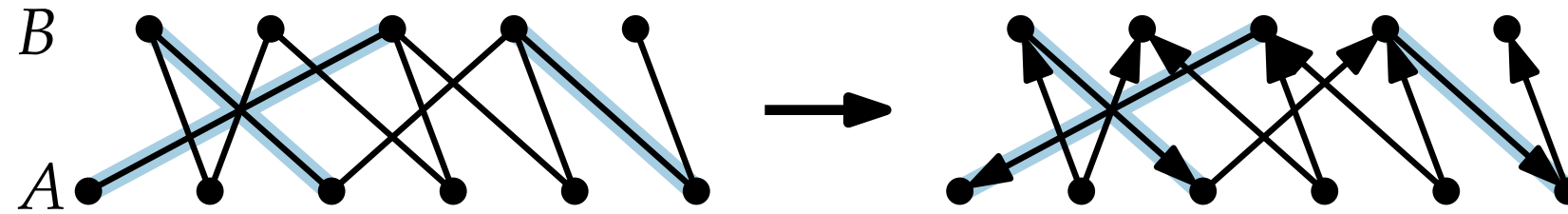


$G = (A \cup B, E)$ bipartit, $P \subseteq E$

Zurück zu: größte Matchings in bip. Graphen



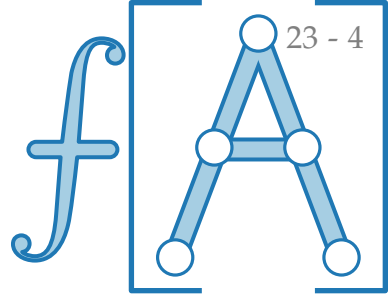
Frage: Wie finden wir augmentierende Wege?



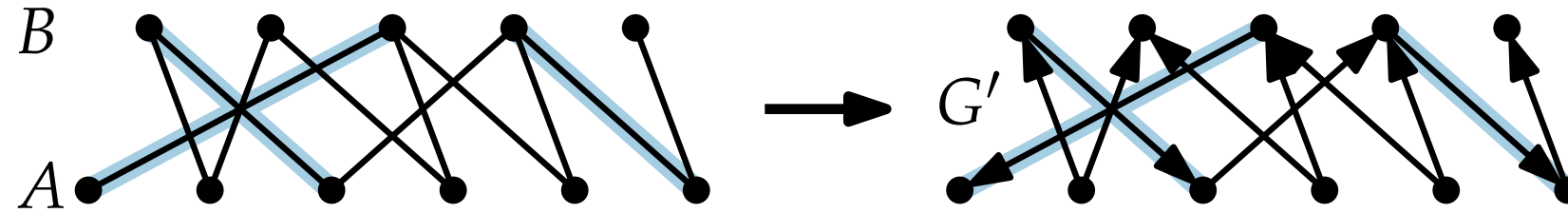
$G = (A \cup B, E)$ bipartit, $P \subseteq E$

Idee: Richte P -Kanten nach unten,
nicht- P -Kanten nach oben.

Zurück zu: größte Matchings in bip. Graphen



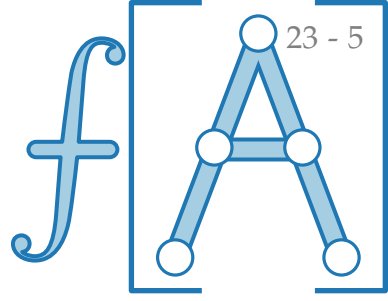
Frage: Wie finden wir augmentierende Wege?



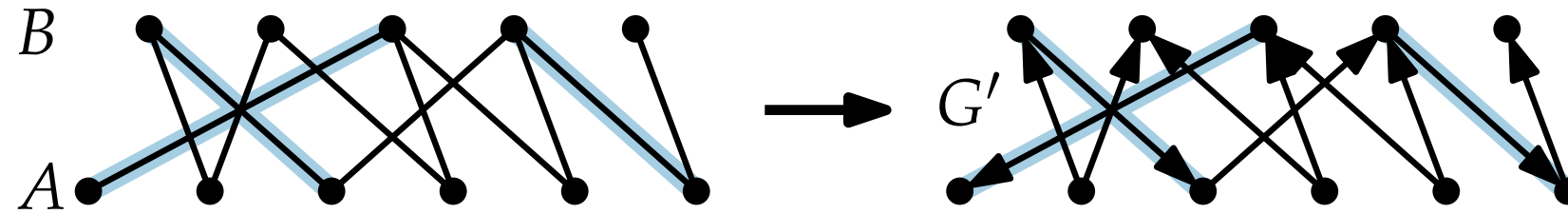
$G = (A \cup B, E)$ bipartit, $P \subseteq E$

Idee: Richte P -Kanten nach unten,
nicht- P -Kanten nach oben. $\} \longrightarrow G' = (A \cup B, E')$

Zurück zu: größte Matchings in bip. Graphen



Frage: Wie finden wir augmentierende Wege?

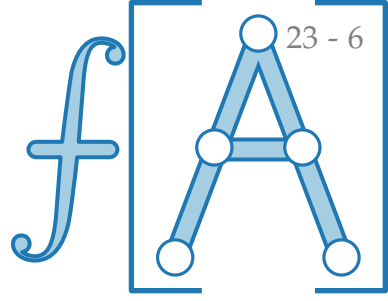


$G = (A \cup B, E)$ bipartit, $P \subseteq E$

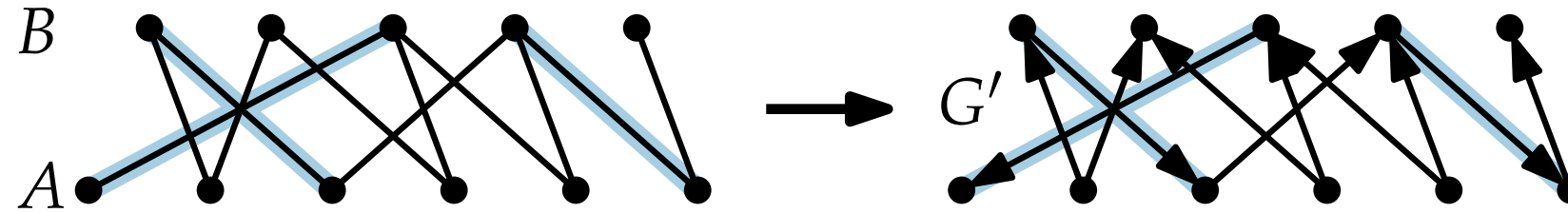
Idee: Richte P -Kanten nach unten,
nicht- P -Kanten nach oben. $\} \longrightarrow G' = (A \cup B, E')$

Augmentierende
Wege in G $\longleftrightarrow$

Zurück zu: größte Matchings in bip. Graphen



Frage: Wie finden wir augmentierende Wege?

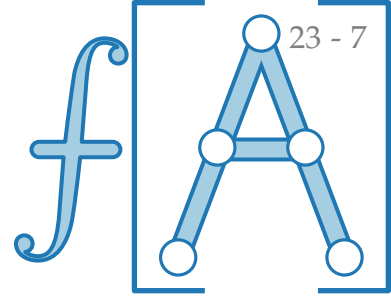


$G = (A \cup B, E)$ bipartit, $P \subseteq E$

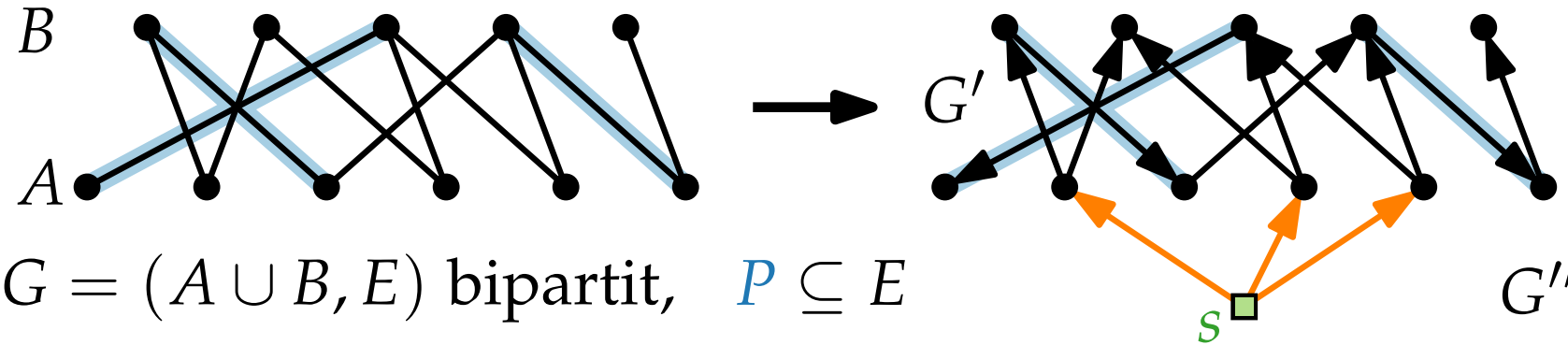
Idee: Richte P -Kanten nach unten,
nicht- P -Kanten nach oben. $\} \longrightarrow G' = (A \cup B, E')$

Augmentierende
Wege in G $\longleftrightarrow$ gerichtete Wege mit
 P -freien Endknoten in G'

Zurück zu: größte Matchings in bip. Graphen



Frage: Wie finden wir augmentierende Wege?

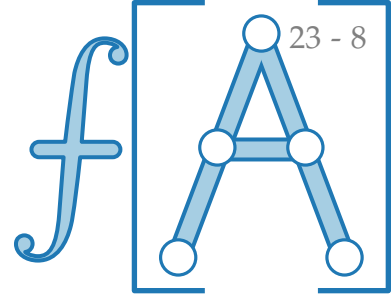


Idee: Richte P -Kanten nach unten,
nicht- P -Kanten nach oben. $\} \longrightarrow G' = (A \cup B, E')$

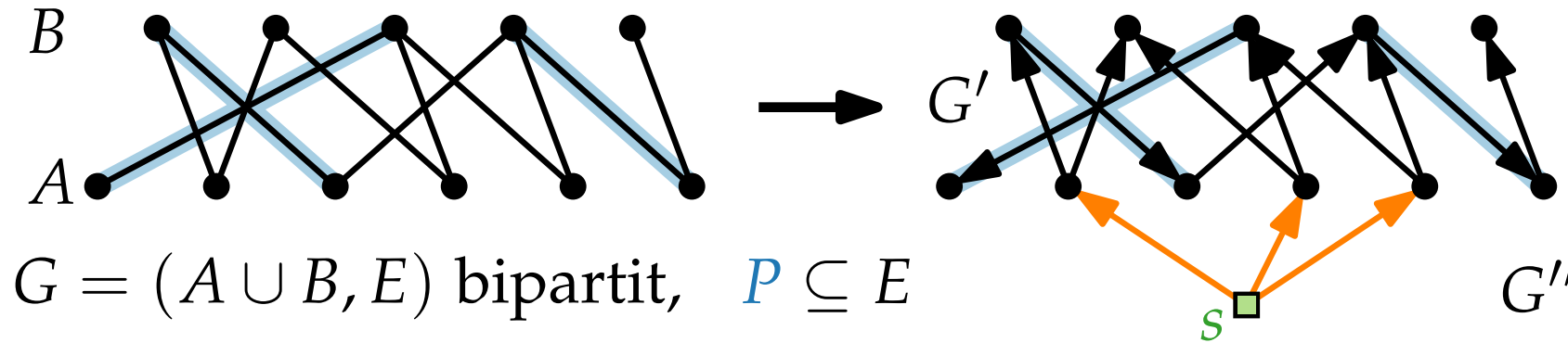
Augmentierende Wege in $G \longleftrightarrow$ gerichtete Wege mit P -freien Endknoten in G'

Definiere $G'' = (A \cup B \cup \{s\}, E' \cup \{sa \mid a \in A, P\text{-frei}\})$

Zurück zu: größte Matchings in bip. Graphen



Frage: Wie finden wir augmentierende Wege?



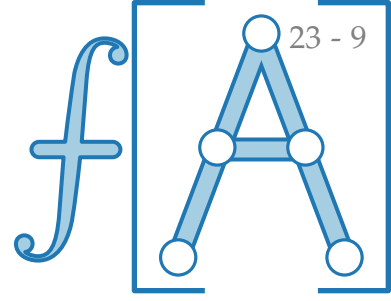
Idee: Richte P -Kanten nach unten,
nicht- P -Kanten nach oben. $\} \rightarrow G' = (A \cup B, E')$

Augmentierende Wege in $G \iff$ gerichtete Wege mit P -freien Endknoten in G'

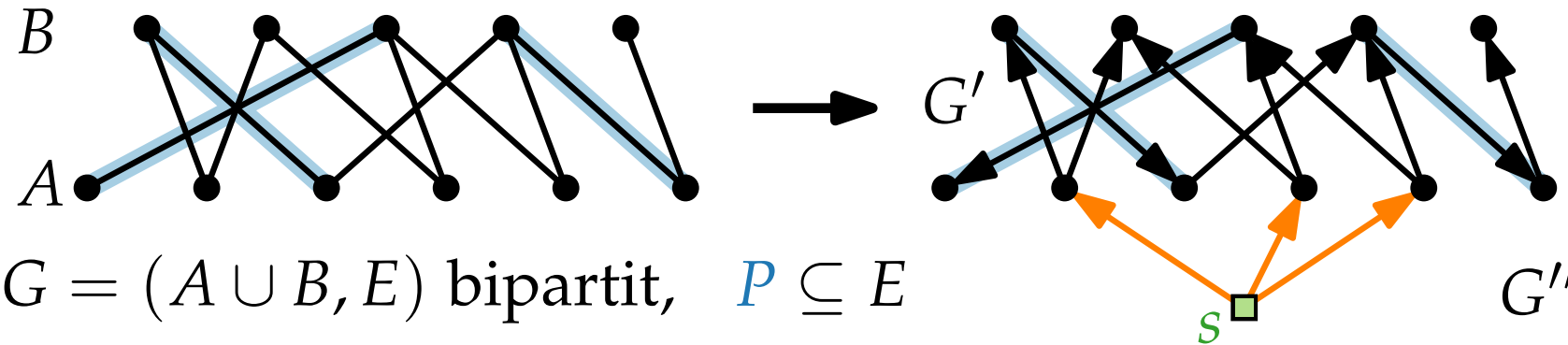
Definiere $G'' = (A \cup B \cup \{s\}, E' \cup \{sa \mid a \in A, P\text{-frei}\})$

Breitensuche $\text{BFS}(G'', s)$ erreicht einen P -freien Endknoten in $B \iff$

Zurück zu: größte Matchings in bip. Graphen



Frage: Wie finden wir augmentierende Wege?



$G = (A \cup B, E)$ bipartit, $P \subseteq E$

Idee: Richte P -Kanten nach unten,
nicht- P -Kanten nach oben. $\} \rightarrow G' = (A \cup B, E')$

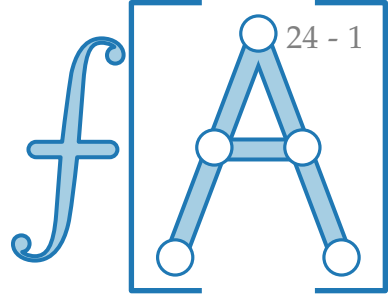
Augmentierende Wege in $G \iff$ gerichtete Wege mit P -freien Endknoten in G'

Definiere $G'' = (A \cup B \cup \{s\}, E' \cup \{sa \mid a \in A, P\text{-frei}\})$

Breitensuche $\text{BFS}(G'', s)$ erreicht einen P -freien Endknoten in $B \iff G$ hat P -augm. Weg.

Ergebnis

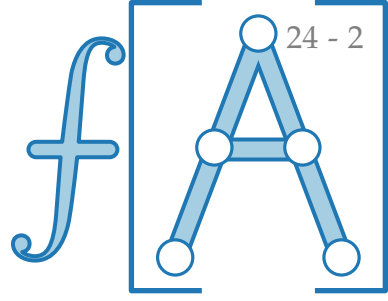
Algo: $P := \emptyset$.



Ergebnis

Algo: $P := \emptyset$.

Führe $\text{BFS} \leq n/2$ mal aus –
bis kein freier Knoten in B mehr gefunden wird.

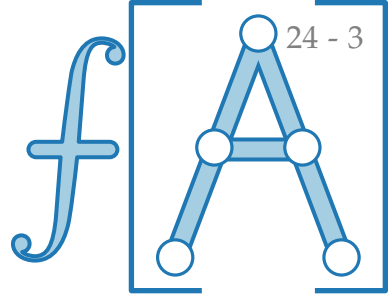


Ergebnis

Algo: $P := \emptyset$.

Führe $\text{BFS} \leq n/2$ mal aus –
bis kein freier Knoten in B mehr gefunden wird.

Gib größte Paarung zurück.



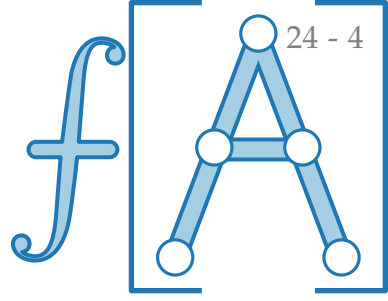
Ergebnis

Algo: $P := \emptyset$.

Führe $\text{BFS} \leq n/2$ mal aus –
bis kein freier Knoten in B mehr gefunden wird.

Gib größte Paarung zurück.

Satz. In einem bipartiten Graphen $G = (V, E)$ lässt sich in $\mathcal{O}(nm)$ Zeit eine größte Paarung bestimmen.



Ergebnis

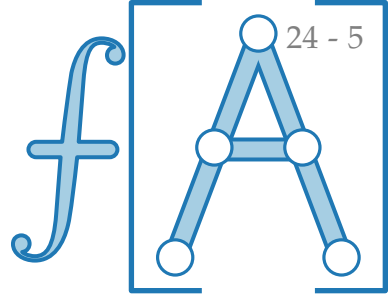
Algo: $P := \emptyset$.

Führe $\text{BFS} \leq n/2$ mal aus –
bis kein freier Knoten in B mehr gefunden wird.

Gib größte Paarung zurück.

Satz. In einem bipartiten Graphen $G = (V, E)$ lässt sich in $\mathcal{O}(nm)$ Zeit eine größte Paarung bestimmen.

Satz. [Edmonds 1965, Gabow & Lawler 1976]
In einem allgemeinen Graphen $G = (V, E)$ lässt sich mit Hilfe P -augmentierender Wege in $\mathcal{O}(n^3)$ Zeit eine größte Paarung bestimmen.



Jack Edmonds
★1934